

Programación aplicada a la sensórica industrial y automotriz

ESP32, MicroPython y monitoreo IoT

Javier Marcelo Flores Monrroy

Programación aplicada a la sensorica industrial y automotriz

ESP32, MicroPython y monitoreo IoT

© 2026 Javier Marcelo Flores Monrroy

Primera edición.

Depósito legal: 4-4-115-2026

Los programas de este libro se publican bajo licencia MIT. Pueden copiarse, modificarse y usarse en clase o en trabajo profesional, con la sola condición de conservar el aviso de licencia.

El texto, las figuras y los diagramas de conexión se publican bajo Creative Commons BY-NC-SA 4.0: pueden reproducirse citando la fuente, sin uso comercial y compartiendo igual.

Todos los programas, los módulos que se copian a la placa y los archivos de las figuras están en:

github.com/Chelo-umsa/sensorica-esp32

Los nombres de marcas y productos mencionados —ESP32, Adafruit IO, Grafana, InfluxDB, Thonny, Wokwi— pertenecen a sus respectivos titulares y se citan con fines didácticos.

Cada programa de este libro se comprobó contra un simulador de MicroPython escrito para ese fin, que verifica la aritmética, los rangos y el comportamiento ante fallas de cableado. Las constantes de calibración corresponden a los sensores con los que se escribió el libro y deben ajustarse a los de cada laboratorio, con el procedimiento del apartado 3.5.

Índice

1. El ESP32 y MicroPython	1
1.1 La placa y sus pines	1
1.2 Instalar MicroPython y Thonny	7
1.3 El simulador Wokwi	11
1.4 El lenguaje: lo que conviene tener fresco	12
1.5 El primer programa	16
2. Entradas y salidas digitales	19
2.1 Gobernar una salida digital	19
2.2 Coordinar varias salidas: el semáforo	23
2.3 Leer una entrada digital: el pulsador	27
2.4 Una entrada que gobierna una salida: alarma con sensor PIR	32
2.5 Una carga que no puede colgar del pin: el relé	37
3. Señales analógicas	47
3.1 Leer una tensión: el potenciómetro	47
3.2 Señales de 5 V: el divisor de tensión	51
3.3 Un sensor lineal: el LM35	56
3.4 Un sensor que no es lineal: el termistor NTC	60
3.5 Calibración de dos puntos	65
4. Señales de pulso	71
4.1 Contar sin perder: consulta contra interrupción	71
4.2 Frecuencímetro por conteo	77
4.3 Medición por período	80
4.4 Tacómetro: del pulso a las revoluciones	83
5. Salidas moduladas	91
5.1 Qué se controla y qué no	91

5.2 Brillo de un LED gobernado por un potenciómetro	93
5.3 El servomotor: la posición va en el ancho del pulso	97
5.4 Barrido continuo	101
5.5 Tonos y sirena con zumbador pasivo	104
6. Sensores con bus y protocolo	109
6.1 Explorar el bus I ² C	110
6.2 La pantalla OLED	113
6.3 DHT11: un hilo y un acuerdo propio	116
6.4 HX710B: un protocolo con reloj propio	118
6.5 PZEM-004T: conversar con un instrumento	122
6.6 Tablero local: leer, decidir y mostrar	126
7. Monitoreo remoto	133
7.1 Conectar a la red sin publicar las claves	135
7.2 La placa como servidor web	137
7.3 La misma página con indicadores de barra	144
7.4 El celular por Bluetooth	145
7.5 De placa a placa, sin red: ESP-NOW	151
7.6 Publicar en internet: MQTT y Adafruit IO	158
7.7 Un tablero propio: InfluxDB y Grafana	163
7.8 Una página que manda, no que informa	168
7.9 El botón del tablero: suscribirse en lugar de publicar	174
Anexo A. Crear la cuenta y el tablero en Adafruit IO	181
A.1 La cuenta y la clave	181
A.2 Los feeds	182
A.3 El tablero	183
Anexo B. Los módulos que se copian a la placa	185
Anexo C. Los pines de cada laboratorio	187

Prefacio

Este libro nació de un problema de aula. Dicto **Programación** en mecánica automotriz e **Informática Industrial** en electricidad: dos materias que enseñan lo mismo con nombres distintos. En una se llama monitoreo de procesos y en la otra, sensores del vehículo. Los estudiantes de cada carrera creen que lo suyo no le sirve a la otra, y no es así: un termistor que mide el bobinado de un motor y uno que mide el refrigerante de un vehículo se leen con el mismo programa.

De ahí sale la única decisión de fondo del libro: los capítulos están ordenados por **tipo de señal** y no por aplicación. Un capítulo para las señales digitales, otro para las analógicas, otro para los pulsos, otro para los sensores que hablan por bus. Así el mismo laboratorio sirve a las dos áreas, y cada uno cierra con un recuadro que muestra el otro uso del mismo algoritmo.

Por eso el libro no está escrito para una carrera sino para dos familias de carreras. En el **área eléctrica y electrónica** —electricidad, electromecánica, electrónica y telecomunicaciones— porque un sensor es, antes que nada, una señal que hay que acondicionar, medir y transmitir. Y en el **área mecánica** —mecánica automotriz, mecánica industrial, mantenimiento— porque el motor, la caja y el sistema hidráulico de hoy se diagnostican leyendo esos mismos sensores. Quien dicte una materia de sensores, de instrumentación, de automatización o de diagnóstico electrónico va a encontrar aquí sus contenidos, con el acomodo de que los laboratorios no hay que rehacerlos para cada curso.

El método de cada laboratorio es el del volumen anterior de esta serie: primero el razonamiento, después el circuito y recién entonces el programa. Un programa que se copia sin haber entendido qué problema resuelve no enseña nada, y cuando falla no hay por dónde empezar.

Los errores que aparecen en estas páginas no son hipotéticos. Casi todos salieron de las guías que vengo usando en clase: un LED cableado al revés que hacía que `on()` lo apagara, un servomotor mandado contra su tope mecánico, un pulsador que ponía la fuente en cortocircuito, un servidor web que moría con la primera trama defectuosa del sensor. Están

señalados con su síntoma y su causa porque un error explicado enseña más que un programa que siempre funcionó.

Los programas están verificados uno por uno. No contra el criterio de quien los escribió, sino contra un simulador que comprueba la aritmética, los rangos y lo que ocurre cuando un sensor se desconecta. Aun así, la placa siempre tiene la última palabra: donde el libro da una constante de calibración, es la del sensor con que se escribió, y el apartado 3.5 explica cómo obtener la del suyo.

Está pensado para que un estudiante pueda armar cada montaje con lo que hay en un laboratorio de una universidad pública. Ningún laboratorio necesita instrumental que no sea un multímetro, y los tres primeros del capítulo 4 no necesitan ni un sensor: la placa se genera las señales a sí misma.

El ESP32 y MicroPython

Un sensor, por sí solo, no informa nada. Entrega una tensión, un tren de pulsos o una trama de bits, y hace falta algo que lo interroge, interprete lo que devuelve y lo convierta en una magnitud con unidades. Ese trabajo lo hace el microcontrolador, y a lo largo de este libro lo hará siempre el mismo: una placa ESP32 programada en MicroPython.

Este capítulo prepara la mesa de trabajo. No hay todavía ningún sensor conectado: se trata de conocer la placa, saber qué pin se puede usar y cuál no, dejar MicroPython instalado y escribir el primer programa. Es el único capítulo del libro que puede leerse sin tener nada en la mano, y conviene no saltarlo, porque buena parte de los tropiezos que aparecen más adelante —una lectura que no varía, una placa que no arranca, un sensor que mide de menos— se explican por algo que está dicho en estas páginas.

1.1 La placa y sus pines

El ESP32 es un microcontrolador de 32 bits con WiFi y Bluetooth integrados. Para lo que interesa en este libro, sus características relevantes son cuatro: trabaja a **3,3 V**, tiene un convertidor analógico-digital de **12 bits**, maneja todos los protocolos de comunicación que usan los sensores habituales (I²C, SPI, UART) y puede conectarse a una red sin hardware adicional. Esa última capacidad es la que permite que el mismo montaje que mide sobre la mesa termine publicando en internet, que es el recorrido completo del libro.

Las placas de desarrollo más difundidas son la **ESP32 DevKit V1** y sus equivalentes de 30 o 38 pines. Todas exponen el mismo chip, y las diferencias están en cuántos pines dejan accesibles. Los laboratorios de este libro se escribieron sobre una DevKit V1 de 30 pines.

Dos alimentaciones que no son intercambiables

La placa tiene un pin 3V3 y un pin VIN. El primero entrega los 3,3 V con que trabaja el propio microcontrolador; el segundo devuelve los 5 V que entran por el conector USB. Muchos sensores funcionan con cualquiera de los dos, pero algunos no: el LM35 del capítulo 3 necesita al menos 4 V y alimentado con 3,3 V mide de menos sin dar ningún síntoma de falla. Cada laboratorio indica de cuál de los dos pines debe colgarse el sensor, y esa indicación no es intercambiable.

Masa, tierra y ground no son la misma cosa

En el taller las tres palabras se usan como sinónimos, y a la hora de conectar un cable da igual cuál se diga. Este libro dice **masa** siempre, y conviene saber por qué, porque llega un momento —en instalaciones, en protecciones— en que la diferencia deja de ser un capricho de vocabulario.

La **masa** es el punto de 0 V que todo el montaje toma como referencia. Medir una tensión es medir una diferencia, de modo que sin una referencia común no hay nada que medir: por eso el sensor y la placa tienen que compartir ese cable aunque cada uno tenga su propia fuente, y por eso la mitad de las fallas de un montaje que «lee cualquier cosa» son una masa que quedó suelta. En la placa es el pin **GND**, y hay varios porque hacen falta varios.

La **tierra** de una instalación es otra cosa. Es el conductor que llega al electrodo enterrado, y su trabajo no es servir de referencia sino desviar una corriente de falla para que no pase por una persona. En funcionamiento normal no conduce nada, y no es donde se conecta un sensor.

Las dos áreas lo muestran bien. En un vehículo la masa es el **chasis**, unido al borne negativo de la batería, y no hay ninguna tierra: el auto está parado sobre cuatro ruedas de caucho. En un tablero industrial conviven las dos y son bornes distintos, con colores distintos y funciones que no se reemplazan. De ahí la decisión del libro: *tierra* queda reservada para la instalación y *masa* nombra el 0 V de los montajes.

En los diagramas el pin va rotulado **GND**, que es lo que dice la serigrafía de la placa y lo que el estudiante va a leer con el montaje en la mano.

QUÉ PIN SE PUEDE USAR

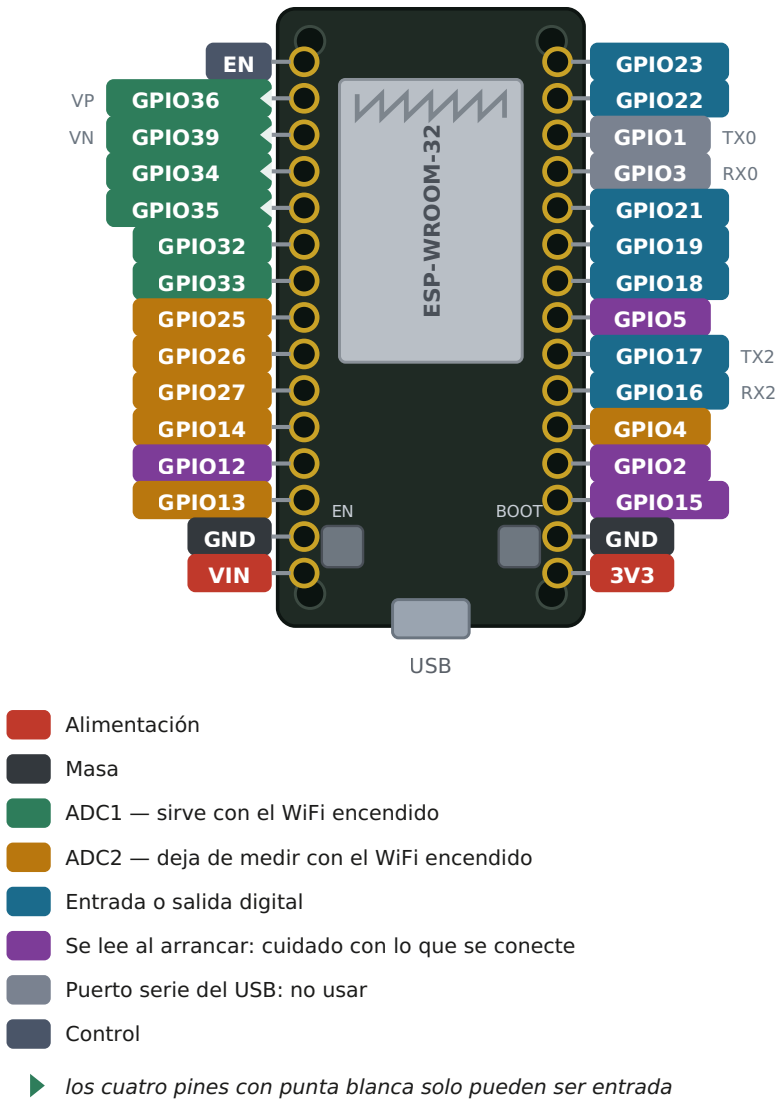


Figura 1.1 — La placa vista desde arriba, con el conector USB abajo. El rótulo de color da el número de GPIO, que es el que usa el programa; a su lado, en gris, lo que dice la serigrafía cuando no coinciden.

Ésta es la placa de 30 pines, y hay otra de 38

Todas las figuras de este libro muestran la **DevKit de 30 pines**: quince agujeros por lado. Es la más difundida y la más barata, y es la que se toma como referencia de aquí en adelante.

En el mercado también se consigue una versión de **38 pines**, con diecinueve por lado. Es más larga, saca al conector los pines de la memoria flash —los GPIO 6 a 11, que igual no se pueden usar— y, sobre todo, **lleva los pines en otro orden**. El microcontrolador es el mismo y los programas de este libro corren idénticos en las dos; lo que no coincide es el dibujo.

De ahí una recomendación que vale para todo el libro: **conecte leyendo el nombre del pin, nunca contando posiciones desde la esquina**. Contar funciona mientras la placa sea la del dibujo, y falla en silencio apenas alguien trae la otra. Para saber cuál tiene, cuente los agujeros de un lado: quince es ésta, diecinueve es la de 38.

No todos los pines del ESP32 sirven para todo. Algunas están comprometidas con la memoria interna, otras sólo leen, y otras se consultan durante el arranque y cambian el comportamiento de la placa según el nivel que encuentren. Conviene tener presente esta tabla desde el principio, porque explica una cantidad desproporcionada de fallas.

Pines	Se pueden usar	Advertencia
GPIO 6 - 11	No	Están conectados a la memoria flash del propio módulo. Usarlos impide que la placa arranque. En muchas placas ni siquiera están expuestos.
GPIO 34, 35, 36, 39	Sólo como entrada	No tienen resistencias internas de elevación ni de descenso, y no pueden configurarse como salida. Son excelentes entradas analógicas.
GPIO 0, 2, 12, 15	Sí, con cuidado	Se leen durante el arranque. Si tienen algo conectado que fuerce un nivel, la placa puede quedarse sin arrancar o entrar en modo de carga de firmware.
GPIO 1, 3	Sí, con cuidado	Son el puerto serie que usa el USB. Al utilizarlos se pierde la consola de Thonny.
Los demás	Sí	Entrada o salida, sin restricciones particulares.

El error de los dos convertidores analógicos

El ESP32 tiene dos convertidores. El **ADC1** ocupa los pines **GPIO 32 a 39**; el **ADC2**, los GPIO 0, 2, 4, 12 a 15 y 25 a 27. La diferencia es decisiva: **el ADC2 deja de funcionar cuando el WiFi está encendido**, porque la radio lo usa internamente.

En la práctica esto significa que un montaje que mide bien sobre la mesa deja de medir en cuanto se le agrega la conexión a la red, y el síntoma es un valor congelado o un error difícil de interpretar. Por eso, en todo este libro, **cualquier sensor analógico va a un pin del ADC1**, esté o no previsto conectarlo a internet. Es una disciplina barata que evita rehacer el cableado al llegar al capítulo 7.

CUÁNTA CORRIENTE PUEDE ENTREGAR UN PIN

Una salida del ESP32 puede entregar unos 12 mA con comodidad, y no debería pedírsele más de 40 mA en ningún caso. Alcanza de sobra para un

LED con su resistencia, pero no para un relé, un motor ni una tira de LED. Todo lo que consume más que un indicador luminoso se maneja con un transistor, un módulo de relé o un controlador, nunca colgado directamente del pin. Es la diferencia entre un montaje que dura el semestre y uno que se lleva la placa puesta.

1.2 Instalar MicroPython y Thonny

MicroPython es una implementación reducida de Python pensada para microcontroladores. Se graba una sola vez en la placa y a partir de ahí la placa «habla» Python: se le pueden mandar instrucciones sueltas y ver la respuesta en el momento, sin compilar ni cargar nada. Esa inmediatez es la razón principal para elegirlo frente a C en un curso.

El entorno que se usará es **Thonny**, gratuito y disponible para Windows, Linux y macOS.

PROCEDIMIENTO

1. Instalar Thonny. Descargarlo de thonny.org y ejecutar el instalador. No requiere ninguna opción especial.

2. Descargar el firmware. En micropython.org/download/, buscar la placa ESP32 genérica y bajar el archivo `.bin` más reciente.

3. Instalar el controlador del puerto. La placa no se comunica por USB por su cuenta: lleva un segundo integrado que traduce entre el USB del computador y el puerto serie del ESP32, y Windows necesita su controlador para verlo. Es el paso que más veces falla en la primera clase y el que decide todo lo que viene después; el apartado siguiente explica cuál corresponde.

4. Conectar la placa al computador con un cable USB. Debe ser un cable de datos: los cables de sólo carga, muy comunes, no establecen comunicación y la placa no aparece en ningún puerto.

5. Grabar el firmware. En Thonny, ir a *Herramientas* → *Opciones* → *Intérprete*, elegir *MicroPython (ESP32)* y el puerto COM correspondiente, y pulsar *Instalar o actualizar firmware*. Seleccionar el archivo descargado y confirmar. El proceso tarda menos de un minuto.

6. Comprobar. De vuelta en la ventana principal, el panel inferior —la *Shell*— debe mostrar el indicador de MicroPython:

```
>>> print("hola")
hola
>>> import machine
>>> machine.freq()
160000000
```

Si esas tres líneas responden, la placa está lista y no hay que volver a grabar el firmware nunca más.

CUÁL CONTROLADOR, Y POR QUÉ NO ES SIEMPRE EL MISMO

El ESP32 no habla USB. Junto al conector hay un segundo integrado, el **conversor USB-serie**, que traduce entre el cable y el puerto serie del microcontrolador. Ese chip es el que el computador ve, y ése es el que necesita controlador.

El problema es que no todas las placas llevan el mismo. Dos placas que se ven iguales, compradas el mismo día, pueden traer conversores de fabricantes distintos, y el controlador de una no le sirve a la otra. De ahí que en la primera clase medio curso esté trabajando y la otra mitad no vea ningún puerto, con el mismo instalador ejecutado.

Hay dos maneras de saber cuál toca. Con la placa en la mano, leer la inscripción del integrado cuadrado que está al lado del conector USB. Con la placa conectada, abrir el **Administrador de dispositivos** de Windows y mirar qué nombre aparece —o qué dispositivo aparece con un signo de admiración, que es justamente el que se quedó sin controlador.

Dice el integrado	Windows lo llama	Controlador que corresponde
CP2102 , CP2102N	Silicon Labs CP210x USB to UART Bridge	El VCP de Silicon Labs , en su página de <i>CP210x USB to UART Bridge VCP Drivers</i> .
CH340 , CH340C , CH9102	USB-SERIAL CH340 o USB Enhanced SERIAL	El VCP de WCH , que en su sitio figura como <i>CH341SER</i> .

Los dos son gratuitos y no piden registro. En Linux no hace falta ninguno —el núcleo ya los trae— y en las versiones recientes de macOS tampoco. Y en Windows 11 al día, el de Silicon Labs suele instalarse solo por Windows Update apenas se conecta la placa: por eso hay estudiantes a los que nunca les falló, y conviene saberlo antes de que expliquen que ellos no hicieron nada distinto.

El controlador va en el computador, no en la placa

Instalarlo no le graba nada al ESP32, no le borra el programa y no tiene relación con el firmware de MicroPython. Es un archivo que se instala en Windows para que reconozca un chip.

Vale la pena decirlo en voz alta porque el reflejo, cuando una placa no aparece, es volver a grabarle el firmware. No puede funcionar: para grabar el firmware hace falta el puerto, que es exactamente lo que falta. Y una placa que no aparece casi nunca está dañada —lo que falta está del lado del computador.

CUANDO LA PLACA SIGUE SIN APARECER

Síntoma	Causa habitual
No hay ningún puerto COM en la lista	Falta el controlador del conversor USB-serie, o se instaló el del otro fabricante. Ver el apartado anterior.
Aparece un dispositivo con signo de admiración	El controlador está, pero no es el de ese chip. Desinstalarlo desde el Administrador de dispositivos e instalar el que corresponda.
El controlador está bien y aun así no hay puerto	El cable. Los de sólo carga no llevan las líneas de datos y no hay manera de distinguirlos por fuera; la prueba es cambiarlo por uno que se sepa bueno.
El puerto aparece pero la conexión falla	Otro programa lo tiene tomado. Un monitor serie abierto, o una segunda ventana de Thonny, bloquean el puerto.
La grabación se interrumpe a la mitad	En algunas placas hay que mantener presionado el botón BOOT mientras comienza la grabación, y soltarlo cuando la barra empieza a avanzar.
La placa se reinicia sola todo el tiempo	Alimentación insuficiente. Un puerto USB de bajo consumo, o un cable largo y delgado, no sostienen los picos de corriente del WiFi.

Guardar en la placa o en el computador

Thonny distingue dos lugares. Al guardar un programa pregunta si va *en este computador* o *en el dispositivo MicroPython*. Un programa guardado en el computador se ejecuta en la placa pero desaparece de ella al desconectarla; uno guardado en la placa queda allí. Y hay un nombre especial: el archivo llamado `main.py` dentro de la placa se ejecuta solo cada vez que se la alimenta, sin computador de por medio. Es lo que convierte un ejercicio de laboratorio en un equipo que funciona en el tablero.

1.3 El simulador Wokwi

Wokwi es un simulador de electrónica que funciona en el navegador, en `wokwi.com`. Permite armar un circuito con ESP32, escribir el programa en MicroPython y verlo funcionar sin tener la placa delante. Para un curso numeroso resuelve un problema concreto: los estudiantes pueden practicar entre clases, sin esperar turno de laboratorio y sin riesgo de dañar componentes.

Sus ventajas para este libro son tres. La primera es que no hay nada que instalar. La segunda es que el hardware virtual no se quema, de modo que el estudiante puede equivocarse cuantas veces necesite. La tercera, menos evidente, es que separa los problemas: si el programa funciona en el simulador y no en la placa, el defecto está en el montaje y no en el código, lo cual acorta enormemente la búsqueda.

Wokwi simula además el WiFi, de modo que los laboratorios de red del capítulo 7 pueden ensayarse allí antes de armarlos, y dispone de un analizador lógico virtual con el que se pueden observar las tramas de I²C o UART que en la placa real son invisibles.

CÓMO EMPEZAR

En `wokwi.com` se elige *New Project* → *ESP32* → *MicroPython*. Aparecen dos paneles: el diagrama del circuito a la derecha y el editor a la izquierda. Los componentes se agregan con el botón **+** y se conectan arrastrando el ratón de una pata a otra. El botón de arranque pone a correr la simulación, y **Ctrl+C** sobre la terminal entra al intérprete interactivo, igual que en Thonny.

Del simulador a la mesa

El circuito que se arma en Wokwi y el que se arma en la protoboard no siempre se dibujan igual. El simulador resuelve internamente cosas que en el montaje real hay que poner —una resistencia de elevación, por ejemplo— y admite conexiones que sobre la mesa conviene hacer de otra manera. Por eso cada laboratorio de este libro trae su propia sección de circuito, y es **esa** la que debe seguirse al cablear sobre protoboard. El simulador sirve para probar la lógica del programa; el cableado del libro, para que el montaje funcione y dure.

1.4 El lenguaje: lo que conviene tener fresco

Este libro no enseña a programar. Da por sabidos los tipos de datos, las estructuras de decisión y repetición, las listas y las funciones, que son materia de un primer curso de programación.¹ Lo que sí hace falta es señalar en qué se aparta MicroPython del Python que se usa sobre un computador, porque son pocas diferencias pero todas aparecen en los laboratorios.

EL MÓDULO MACHINE

Todo el acceso al hardware pasa por el módulo `machine`. Es la primera línea de casi todos los programas del libro:

```
from machine import Pin, ADC, PWM, SoftI2C, UART
```

Clase	Para qué sirve	Capítulo
<code>Pin</code>	Entradas y salidas digitales	2
<code>ADC</code>	Leer una tensión continua	3
<code>Pin.irq</code>	Atender un pulso sin estar esperándolo	4
<code>PWM</code>	Salidas moduladas: brillo, velocidad, servomotores	5
<code>SoftI2C</code> , <code>UART</code>	Sensores que hablan por bus	6

EL TIEMPO SE MIDE DISTINTO

En un computador el tiempo se maneja con `time.sleep()` y con la hora del sistema. En un microcontrolador no hay reloj de calendario al encender, y las esperas se cuentan en milisegundos o microsegundos. MicroPython agrega para eso una familia de funciones:

Función	Qué hace
<code>time.sleep_ms(n)</code>	Espera n milisegundos.
<code>time.sleep_us(n)</code>	Espera n microsegundos.
<code>time.ticks_ms()</code>	Devuelve un contador de milisegundos desde que arrancó la placa.
<code>time.ticks_diff(a, b)</code>	Calcula <code>a - b</code> entre dos lecturas del contador.

Por qué no se restan los contadores directamente

El contador de `ticks_ms()` no crece indefinidamente: llega a un máximo y vuelve a cero. Si se restan dos lecturas con el signo menos y el vuelco ocurre entre ambas, el resultado es un número negativo enorme y el programa se comporta de manera inexplicable. Un equipo que anduvo bien toda la tarde falla una vez por día, siempre a la misma hora de encendido.

`ticks_diff()` existe precisamente para eso: hace la resta teniendo en cuenta el vuelco. En este libro **toda** diferencia de tiempos se calcula con `ticks_diff()`, sin excepción.

ESPERAR O PREGUNTAR SI YA PASÓ EL TIEMPO

Casi todo este libro se escribe con `sleep`, que es lo natural y lo más legible. Pero hay programas que no lo pueden usar, y conviene ver desde ahora por qué, porque es la única diferencia de estructura que separa a los programas simples de los que vienen después.

Tomemos un LED que parpadea cada medio segundo. Escrito con esperas es lo más corto que puede ser:

```
while True:
    led.value(1)
    time.sleep(0.5)
    led.value(0)
    time.sleep(0.5)
```

Y escrito preguntando por el reloj hace exactamente lo mismo, con el doble de líneas:

```
encendido = False
cambio = time.ticks_ms()

while True:
    if time.ticks_diff(time.ticks_ms(), cambio) >= 500:
        encendido = not encendido
        led.value(1 if encendido else 0)
        cambio = time.ticks_ms()
```

Si los dos hacen lo mismo y el segundo es más largo, la pregunta es para qué sirve. La respuesta está en lo que pasa **mientras tanto**.

En la primera versión, durante esos dos medios segundos el programa está detenido. No lee un pulsador, no atiende un sensor, no contesta a nadie: está esperando y nada más. En la segunda, la repetición gira miles de veces por segundo y en cada vuelta vuelve a mirar el reloj; el LED cambia cuando corresponde, y entre cambio y cambio **queda sitio para hacer otra cosa**.

De ahí la regla que vale para todo el libro, y que es la única que hace falta recordar:

Cuándo cada una

Se usa `sleep` cuando el programa **no tiene nada más que hacer** mientras espera. Es el caso del primer parpadeo, del semáforo, y de cualquier programa que mide, muestra y vuelve a medir: ahí la espera no estorba a nadie y escribirlo con el reloj sólo lo haría más largo.

Se usa `ticks_diff` cuando **sí**. Un pulsador que hay que seguir leyendo mientras el LED hace lo suyo, una sirena que no puede dejar de vigilar al sensor, un servidor web que tiene que atender visitas sin dejar de medir. En todos esos casos la espera dejaría ciego al programa justo cuando tiene que estar mirando.

La cuenta vale la pena hacerla una vez. Una alarma que alterna dos tonos cada doscientos milisegundos escrita con esperas está **detenida cuatro quintas partes del tiempo** si además tiene que leer un sensor cada

cincuenta milisegundos. No es que reaccione lento: es que durante ese rato el movimiento puede empezar y terminar sin que el programa se entere. Eso es exactamente lo que ocurre en el laboratorio 2.4, y es donde esta sección deja de ser teoría.

El error de escribirlo a medias

La tentación, al descubrir el problema, es acortar las esperas: cambiar `sleep(0.5)` por diez `sleep(0.05)` con la lectura del sensor en el medio. Funciona, y es por donde todos pasamos.

Pero el programa queda con la estructura del tiempo metida dentro de la lógica, y agregar una segunda cosa que atender obliga a rehacerlo entero. Con `ticks_diff` cada cosa lleva su propia marca de tiempo y agregar una más son tres líneas, no una reescritura. Es la diferencia entre un programa de laboratorio y uno que se puede seguir usando.

OTRAS DIFERENCIAS QUE SE NOTAN

Asunto	En la placa
Memoria	Hay unos pocos cientos de kilobytes. Un programa que acumula lecturas en una lista sin límite termina agotándola y deteniéndose.
Números reales	Son de precisión simple, no doble. Alcanza para cualquier medición, pero no conviene apoyarse en la decimoquinta cifra.
Biblioteca estándar	Está recortada. Módulos como <code>os</code> o <code>json</code> existen en versión reducida; muchos otros directamente no están.
Módulos adicionales	Los controladores de sensores (<code>ssd1306</code> , <code>umqtt</code>) se copian a la placa como un archivo <code>.py</code> más.

¹ Para el lector que necesite repasar esos fundamentos, el volumen anterior de esta serie, *Algoritmos resueltos con Python*, desarrolla el razonamiento, el diagrama de flujo y el programa de cada construcción del lenguaje con el mismo método que se usa aquí.

1.5 El primer programa

Antes de conectar nada conviene comprobar que la cadena completa funciona: que Thonny escribe en la placa, que la placa ejecuta y que se puede detener un programa en marcha. Para eso alcanza con el LED que la propia placa trae soldado, en el **GPIO2** de la mayoría de las DevKit V1.

OBJETIVO

Hacer parpadear el LED incorporado y aprender a interrumpir un programa que no termina solo.

MATERIALES

Cantidad	Componente
1	Placa ESP32 DevKit V1
1	Cable USB de datos

RAZONAMIENTO

Encender y apagar alternadamente es una repetición que no termina: mientras la placa esté alimentada, debe seguir. Dentro de esa repetición hay cuatro pasos —encender, esperar, apagar, esperar— y las dos esperas son las que hacen visible el cambio. Sin ellas el LED conmutaría millones de veces por segundo y el ojo vería un brillo constante.

El pin se declara una sola vez, antes de la repetición, porque configurarlo es algo que se hace una vez y no en cada vuelta.

PROGRAMA

Archivo `lab_1_5_parpadeo.py`

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)          # el LED que la placa trae soldado

while True:
    led.value(1)               # encender
    print("encendido")
```

```
time.sleep(0.5)

led.value(0)           # apagar
print("apagado")
time.sleep(0.5)
```

QUÉ DEBE OBSERVARSE

El LED azul de la placa parpadea dos veces por segundo y la consola de Thonny escribe alternadamente las dos palabras. El programa no se detiene por sí mismo: se interrumpe con el botón de parada de Thonny o con **Ctrl+C** sobre la consola. Conviene practicarlo ahora, porque todos los programas del libro son repeticiones sin fin y se detienen de la misma manera.

value(1) o on()

`led.value(1)` y `led.on()` hacen exactamente lo mismo. En este libro se usa `value()` de manera sistemática, por una razón práctica: permite escribir `led.value(condicion)` y decidir el estado con una sola línea, en lugar de un `if` con dos ramas. Se verá aprovechado desde el laboratorio siguiente.

SI ALGO NO SALE

Síntoma	Causa habitual
El LED no parpadea pero la consola sí escribe	Esa placa tiene el LED en otro pin, o no lo trae. Probar con el GPIO 5, o seguir directamente al capítulo 2 con un LED externo.
<code>OSError</code> al ejecutar	El programa quedó guardado en el computador y no en la placa, o el intérprete seleccionado en Thonny es el del computador y no el de MicroPython.
El botón de parada no responde	Mantener Ctrl+C presionado sobre la consola. Si tampoco, el botón EN de la placa reinicia el microcontrolador.

El mismo algoritmo en...

Este primer programa es idéntico en las dos aplicaciones, y conviene señalarlo: una baliza de señalización en una planta y una luz intermitente en un tablero de vehículo son, para el microcontrolador, el mismo problema. Lo que cambia es la etapa de potencia que hay después del pin —un relé, un transistor— y los tiempos. El algoritmo, no.

EJERCICIOS PROPUESTOS

1. Modifique los tiempos para que el LED quede encendido un cuarto de segundo y apagado un segundo y medio. ¿Cambia en algo la estructura del programa?
2. Escriba el programa usando una sola espera por vuelta en lugar de dos. Ayuda: `led.value(not led.value())`.
3. Guarde el programa en la placa con el nombre `main.py`, desconecte el cable USB y alimente la placa con un cargador de teléfono. Explique qué ocurre y por qué.
4. ¿Cuántas veces por segundo parpadearía el LED si se quitaran las dos esperas? Justifique con el dato de frecuencia que devolvió `machine.freq()`.
5. Consultando la tabla del apartado 1.1, indique cuáles de los siguientes pines podrían haberse usado en lugar del GPIO2 y cuáles no: 4, 9, 34, 23, 36.

Entradas y salidas digitales

Una señal digital sólo tiene dos estados. Está o no está, conduce o no conduce, hay presencia o no la hay. Es la más simple de todas las que trata este libro y por eso es la primera, pero conviene no subestimarla: el final de carrera de una máquina, el presostato de un compresor, el contacto de una puerta, el interruptor de nivel de un tanque y el sensor de cinturón de un vehículo son todos señales digitales, y buena parte de la automatización real se hace con ellas.

Los cuatro laboratorios de este capítulo recorren el camino completo: primero gobernar una salida, después coordinar varias, después leer una entrada y finalmente hacer que una entrada gobierne una salida. Al terminarlo, el lector tendrá armado un sistema de alarma que funciona, y lo habrá hecho con un solo tipo de señal.

2.1 Gobernar una salida digital

OBJETIVO

Encender y apagar un LED externo desde un pin de la placa, y entender por qué la resistencia en serie no es opcional.

MATERIALES

Cantidad	Componente	Observación
1	LED de 5 mm	de cualquier color
1	Resistencia de 330 Ω	limita la corriente
—	Cables de conexión	

EL CIRCUITO

El LED tiene una pata larga (**ánodo**, positiva) y una corta (**cátodo**, negativa). La corriente entra por la larga y sale por la corta. La resistencia va en serie, y puede ir de cualquiera de los dos lados:

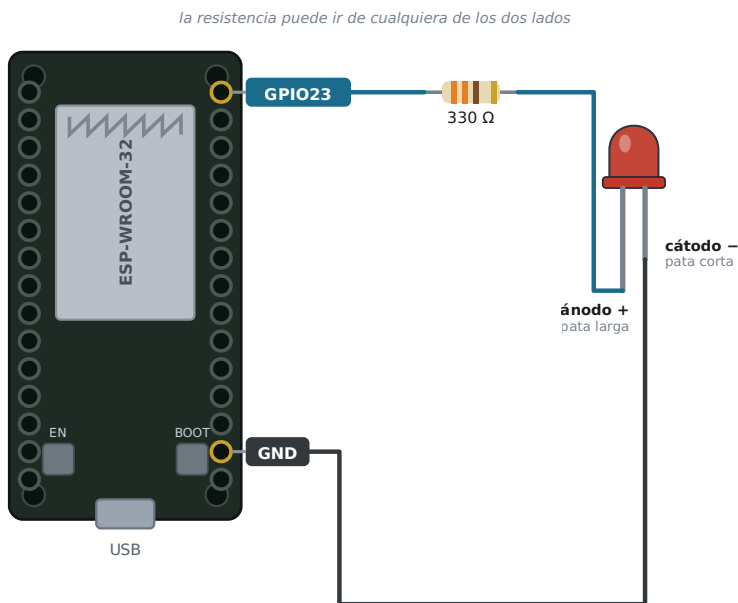


Figura 2.1 — Circuito del laboratorio 2.1. El ánodo va siempre al pin y el cátodo siempre a masa: así `value(1)` significa encendido.

Un detalle que invierte toda la lógica

Existe la tentación de conectar el ánodo a 3V3 y el cátodo al pin, porque el LED igual enciende. Pero entonces enciende cuando el pin está en **bajo**, de modo que `led.value(1)` lo **apaga** y `led.value(0)` lo **enciende**. Todo el programa queda con el significado invertido, y cuando el montaje crece a tres o cuatro luces el error se vuelve muy difícil de rastrear.

En este libro el ánodo va siempre al pin y el cátodo siempre a masa. Así `value(1)` significa encendido, que es lo que cualquiera espera al leer el programa.

Por qué 330 Ω

Un LED rojo cae alrededor de 2 V cuando conduce. Con los 3,3 V del pin quedan 1,3 V sobre la resistencia, y con 330 Ω la corriente resulta de unos 4 mA: suficiente para que se vea bien y muy por debajo de lo que el pin puede entregar. Sin resistencia, la corriente sólo queda limitada por la resistencia interna del LED y del pin, y se destruyen los dos.

RAZONAMIENTO

El programa es el mismo del laboratorio 1.5, cambiando el número de pin. Vale la pena detenerse en eso: para el microcontrolador no hay ninguna diferencia entre el LED soldado a la placa y uno puesto en la protoboard. Lo que cambia es el circuito, no el algoritmo. Esta separación —el programa de un lado, la etapa de potencia del otro— es la que permitirá más adelante reemplazar el LED por un relé sin tocar una sola línea.

PROGRAMA

Archivo `lab_2_1_salida_digital.py`

```
from machine import Pin
import time

led = Pin(23, Pin.OUT)

ENCENDIDO_S = 0.5
APAGADO_S = 0.5

while True:
    led.value(1)
    print("encendido")
    time.sleep(ENCENDIDO_S)

    led.value(0)
    print("apagado")
    time.sleep(APAGADO_S)
```

Los tiempos van en constantes

Los dos tiempos están en constantes con nombre, escritas en mayúsculas al principio del programa, en lugar de repetidos como números sueltos dentro de la repetición. Es una costumbre que este libro sostiene en todos sus programas: cuando haya que ajustar un tiempo, un umbral o un pin, está en un solo lugar visible y no hay que buscarlo entre las líneas.

QUÉ DEBE OBSERVARSE

El LED parpadea una vez por segundo. Si se aumenta `ENCENDIDO_S` y se disminuye `APAGADO_S`, el LED pasa más tiempo encendido sin cambiar la cantidad de parpadeos por minuto.

SI ALGO NO SALE

Síntoma	Causa habitual
El LED no enciende nunca	Está al revés. La pata larga debe mirar hacia el pin. Invertirlo no lo daña: simplemente no conduce.
Enciende cuando el programa dice apagado	El ánodo quedó en 3V3 en lugar del pin. Es el caso del recuadro anterior.
Enciende muy débil	La resistencia es de un valor mucho mayor que $330\ \Omega$. Verificar el código de colores.
El LED enciende fijo y no parpadea	El programa no está corriendo: quedó una versión anterior en la placa, o la ejecución se detuvo con un error que aparece en la consola.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el piloto de señalización de un tablero. Cambiando la resistencia y el LED por un módulo de relé conectado al mismo pin, el programa gobierna una lámpara de 220 V sin que se modifique una sola línea.

En el área mecánica, es una luz testigo del panel de instrumentos. La estructura encender-esperar-apagar-esperar es exactamente la de un intermitente de giro, y sólo hay que ajustar los tiempos a los que fija la norma.

2.2 Coordinar varias salidas: el semáforo

OBJETIVO

Gobernar tres salidas que deben respetar una secuencia, garantizando que en ningún instante haya dos encendidas a la vez.

MATERIALES

Cantidad	Componente
3	LED de 5 mm: rojo, amarillo y verde
3	Resistencias de 330 Ω
—	Cables de conexión

EL CIRCUITO

Los tres LED se conectan igual que el del laboratorio anterior, cada uno a su pin y con su resistencia:

una resistencia de 330 Ω y un pin distinto para cada luz

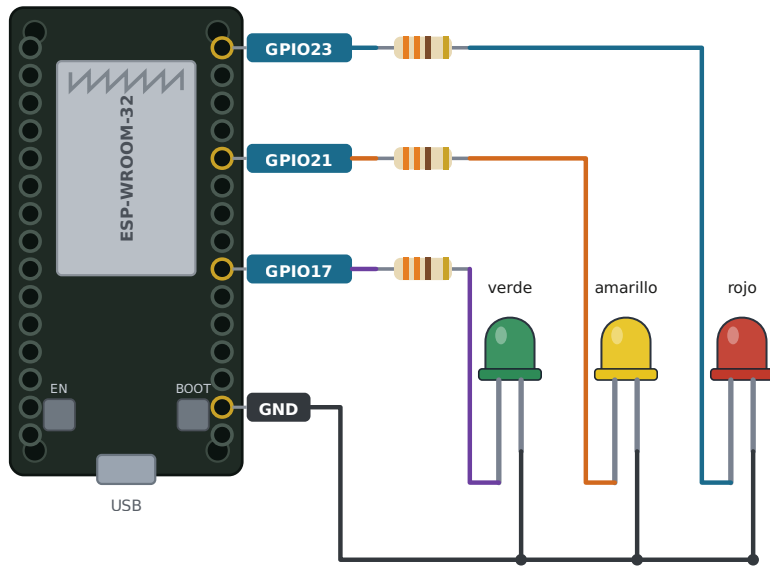


Figura 2.2 — Circuito del semáforo. Cada luz tiene su pin y su resistencia; los tres cátodos comparten la línea de masa.

RAZONAMIENTO

La tentación es escribir la secuencia tal como se la describe en voz alta: «enciendo el verde, espero cinco segundos, apago el verde, enciendo el amarillo...». Ese orden esconde un defecto. Si el apagado del LED anterior se escribe **después** de la espera, durante esos cinco segundos quedan encendidas dos luces: la nueva y la que venía de la fase anterior. En un semáforo de juguete se ve raro; en un enclavamiento real es una falla de seguridad.

La corrección no es agregar apagados, sino cambiar de idea sobre qué se está programando. No hay tres luces que se encienden y se apagan por separado: hay un **estado**, y en cada estado hay exactamente una luz encendida. Escrito así, es imposible que queden dos prendidas, porque cada cambio de estado fija las tres de una vez.

Ese cambio de enfoque se traduce en una función que recibe cuál debe quedar encendida y se ocupa de las otras dos. La repetición principal se vuelve la lectura directa de la secuencia.

PROGRAMA

Archivo `lab_2_2_semaforo.py`

```
from machine import Pin
import time

rojo = Pin(23, Pin.OUT)
amarillo = Pin(21, Pin.OUT)
verde = Pin(17, Pin.OUT)

SEGUNDOS_ROJO = 5
SEGUNDOS_VERDE = 5
SEGUNDOS_AMARILLO = 2

def encender(cual, segundos, mensaje):
    """Deja encendida una sola luz y apaga las otras dos.

    Fijar las tres en la misma instruccion es lo que garantiza que
    en ningun instante haya dos encendidas a la vez.
    """
    rojo.value(1 if cual is rojo else 0)
    amarillo.value(1 if cual is amarillo else 0)
    verde.value(1 if cual is verde else 0)
    print(mensaje)
    time.sleep(segundos)

while True:
    encender(rojo, SEGUNDOS_ROJO, "Rojo - alto")
    encender(verde, SEGUNDOS_VERDE, "Verde - avance")
    encender(amarillo, SEGUNDOS_AMARILLO, "Amarillo - precaucion")
```

Dónde se lee la secuencia

Las tres últimas líneas del programa son la secuencia completa del semáforo, en orden y en castellano. Cualquiera puede leerlas y decir si el ciclo es correcto, sin entender nada de lo que hay más arriba. Cuando un programa se deja escribir así, las modificaciones se vuelven seguras: agregar una fase de intermitente nocturno es agregar una línea, no reescribir la lógica.

QUÉ DEBE OBSERVARSE

El ciclo dura doce segundos y en todo momento hay exactamente una luz encendida. Conviene mirar con atención el instante del cambio: no debe haber ningún momento, por breve que sea, con dos luces prendidas ni con las tres apagadas.

SI ALGO NO SALE

Síntoma	Causa habitual
Quedan dos luces encendidas a la vez	El programa en uso todavía apaga <i>después</i> de la espera. Es el defecto que este laboratorio corrige.
Una luz no enciende nunca	El LED está invertido, o el pin del programa no es aquel en el que se cableó.
Las tres encienden juntas y no cambian	Se cablearon los tres ánodos al mismo pin, o los tres pines quedaron unidos en la misma fila de la protoboard.

El mismo algoritmo en...

En el área eléctrica y electrónica, esta estructura es la de una secuencia de proceso con enclavamiento: tres etapas que deben sucederse y que nunca pueden coincidir, como el arranque estrella-triángulo de un motor, donde energizar los dos contactores a la vez provoca un cortocircuito franco. La función `encender()` es, literalmente, el enclavamiento.

En el área mecánica, es la secuencia de un tablero al dar contacto: testigos que se encienden en orden durante la comprobación inicial, y luces de giro que no pueden estar activas en ambos lados simultáneamente.

2.3 Leer una entrada digital: el pulsador

OBJETIVO

Leer el estado de un pulsador, filtrar el rebote mecánico del contacto y conseguir que cada pulsación cambie el estado del LED y lo mantenga.

MATERIALES

Cantidad	Componente
1	Pulsador de cuatro patas para protoboard
1	LED de 5 mm y resistencia de 330 Ω
—	Cables de conexión

EL CIRCUITO

las patas de un mismo lado están unidas de fábrica: tome una de cada lado

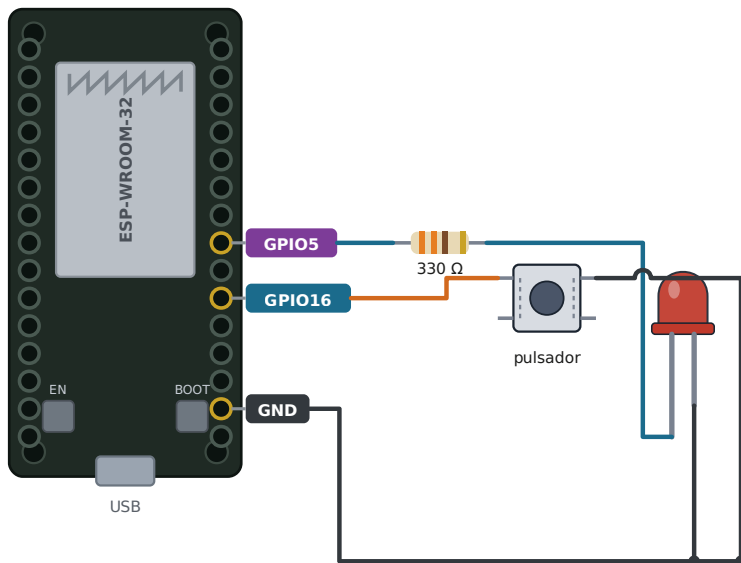


Figura 2.3 — Circuito del pulsador. Un solo polo en el botón: el pin de un lado y la masa del otro, nunca 3V3 y GND enfrentados.

Nunca los dos polos en el pulsador

Un montaje que se ve con frecuencia lleva GND a un lado del pulsador y 3V3 al otro, con el pin tomando la lectura del medio. Al presionar, eso une los 3,3 V con masa: es un **cortocircuito franco** sobre la alimentación de la placa. Puede que el regulador lo soporte un tiempo, y por eso el error sobrevive; pero calienta, provoca reinicios y termina dañando la placa.

La conexión correcta usa **un solo polo**. Una pata al pin, la otra a masa, y nada más. El nivel en reposo lo provee la resistencia interna de la que se habla enseguida. Algunos simuladores admiten el otro montaje sin consecuencias, porque no simulan la corriente; sobre la protoboard sí las hay.

LA RESISTENCIA INTERNA DE ELEVACIÓN

Con una sola pata a masa, el pin queda sin conexión mientras el pulsador está suelto. Un pin así no lee ni 0 ni 1: lee ruido, y cambia de valor solo. Hace falta algo que lo mantenga en un nivel conocido mientras nadie lo toca.

El ESP32 trae ese algo adentro. Declarando el pin con `Pin.PULL_UP` se conecta internamente una resistencia hacia los 3,3 V, de modo que:

Estado del pulsador	Lectura del pin	Por qué
Suelto	1	La resistencia interna lo mantiene elevado.
Presionado	0	El pulsador lo une directamente a masa.

La lectura queda entonces invertida respecto de la intuición: presionado es cero. Por eso el programa empieza con `pulsado = not pulsador.value()`, que traduce la lectura eléctrica al significado que interesa y permite que el resto del programa se lea en castellano.

RAZONAMIENTO

Un pulsador mecánico no cambia de estado limpiamente. Las láminas del contacto rebotan durante unos milisegundos, y en ese lapso el pin lee una ráfaga de ceros y unos. Un programa que actúe ante cada cambio verá diez o quince pulsaciones donde el dedo hizo una sola. Es el motivo por el cual un LED «no responde» o «responde al azar»: en realidad responde de más.

El filtro consiste en exigir que entre dos cambios aceptados haya transcurrido un tiempo mínimo, mayor que la duración del rebote y menor que el intervalo entre dos pulsaciones humanas. Cincuenta milisegundos cumplen ambas condiciones con holgura.

Hay además una segunda decisión. Interesa el instante en que el pulsador **se presiona**, no el que se suelta, porque si no cada pulsación produciría dos cambios y el LED volvería siempre a su estado original. Y el estado

del LED se guarda en una variable propia: el pulsador no manda sobre la luz, manda sobre el **cambio** de la luz. Eso es lo que hace que quede encendida al soltar.

PROGRAMA

Archivo `lab_2_3_pulsador.py`

```
from machine import Pin
import time

pulsador = Pin(16, Pin.IN, Pin.PULL_UP)
led = Pin(5, Pin.OUT)

REBOTE_MS = 50

encendido = False
estaba_pulsado = False
ultimo_cambio = time.ticks_ms()

print("Pulse el boton para encender y apagar el LED.")

while True:
    pulsado = not pulsador.value()      # el pulsador une el pin a masa
    ahora = time.ticks_ms()

    hubo_cambio = pulsado != estaba_pulsado
    paso_el_rebote = time.ticks_diff(ahora, ultimo_cambio) > REBOTE_MS

    if hubo_cambio and paso_el_rebote:
        estaba_pulsado = pulsado
        ultimo_cambio = ahora

        # Solo interesa cuando se presiona, no cuando se suelta.
        if pulsado:
            encendido = not encendido
            led.value(1 if encendido else 0)
            print("LED encendido" if encendido else "LED apagado")

    time.sleep_ms(10)
```

QUÉ DEBE OBSERVARSE

Cada pulsación cambia el estado del LED y lo deja así al soltar. La consola escribe **una sola línea** por pulsación. Vale la pena hacer la prueba

inversa: poner `REBOTE_MS = 0` y volver a ejecutar. Aparecerán varias líneas por cada toque, y a veces el LED quedará en el estado contrario al esperado. Ese es el rebote, visto de frente.

SI ALGO NO SALE

Síntoma	Causa habitual
El LED cambia solo, sin tocar nada	Falta <code>Pin.PULL_UP</code> en la declaración, y el pin está leyendo ruido.
Hay que mantener presionado para que quede encendido	Se está ejecutando la versión sin memoria de estado, la que copia la lectura directamente a la salida.
Una pulsación produce varias líneas en la consola	El tiempo de rebote es demasiado corto para ese pulsador. Subirlo a 80 ms.
No responde nunca	Las dos patas cableadas pertenecen al mismo contacto interno. Un pulsador de cuatro patas tiene dos pares unidos de fábrica: hay que tomar una pata de cada par, es decir, las que están en diagonal.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el pulsador de marcha con enclavamiento: se presiona un instante y el equipo queda en marcha. La variable `encendido` cumple el papel del contacto de retención en un esquema de mando, y el filtro de rebote hace lo mismo que el tiempo de un temporizador antiparasitario en un autómata programable.

En el área mecánica, es el botón de arranque por pulsación: se toca una vez y el sistema queda activo. El mismo filtro es el que impide que la vibración del motor produzca accionamientos falsos en los mandos del volante.

2.4 Una entrada que gobierna una salida: alarma con sensor PIR

OBJETIVO

Detectar movimiento con un sensor infrarrojo pasivo y hacer sonar una sirena, sin que el sonido impida seguir vigilando.

MATERIALES

Cantidad	Componente	Observación
1	Sensor PIR HC-SR501	se alimenta con 5 V
1	Buzzer pasivo	el activo no sirve, ver nota
—	Cables de conexión	

EL CIRCUITO

el PIR se alimenta con 5 V (VIN), que está del otro lado de la placa

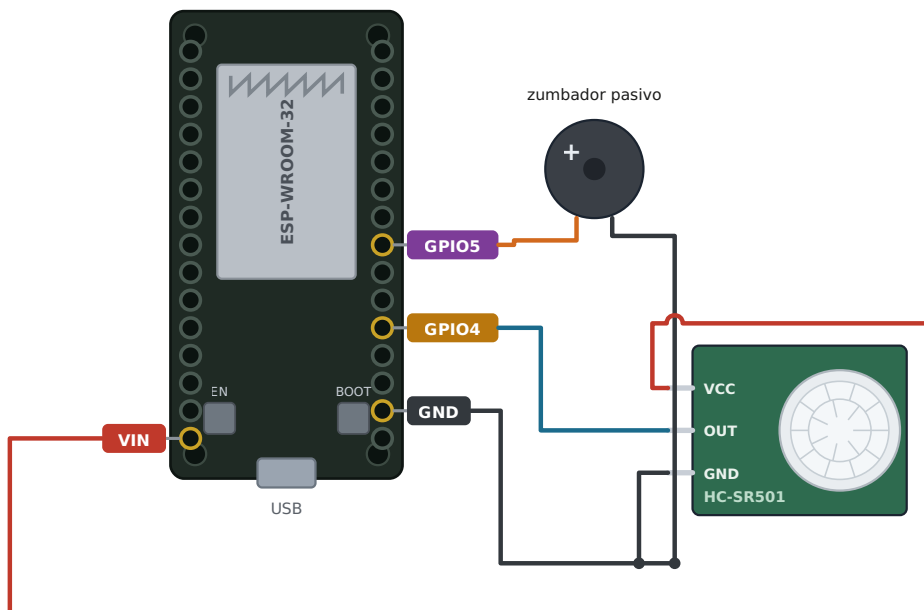


Figura 2.4 — Circuito de la alarma. El PIR se alimenta con los 5 V de VIN, y el buzzer debe ser pasivo para poder formar la sirena.

Buzzer pasivo y buzzer activo

Un buzzer **activo** trae su propio oscilador: se lo alimenta y suena, siempre en la misma nota. Un buzzer **pasivo** no tiene oscilador y hay que entregarle la frecuencia desde el pin. Este laboratorio necesita el pasivo, porque la sirena se construye alternando dos tonos, y eso sólo puede hacerse si el programa controla la frecuencia. Con un buzzer activo el resultado es un pitido constante.

El PIR necesita un minuto

Al alimentarlo, el HC-SR501 tarda alrededor de un minuto en estabilizar su referencia térmica. Durante ese lapso dispara avisos falsos. No es una falla del montaje ni del programa: hay que alimentarlo, esperar, y recién entonces evaluar el comportamiento. El sensor trae además dos potenciómetros, de sensibilidad y de tiempo de retención, que conviene dejar en su posición media al comenzar.

RAZONAMIENTO

La salida del PIR es digital: vale 1 mientras detecta movimiento y 0 el resto del tiempo. Leerla no tiene ninguna dificultad. El problema de este laboratorio es otro, y es el primero de una clase que reaparecerá en todo el libro: **hacer dos cosas a la vez**.

La sirena exige alternar dos tonos cada doscientos milisegundos. Escrito de la manera directa, eso son dos esperas dentro de la repetición; y durante esas dos esperas el programa no está leyendo el sensor. Si el intruso pasa y sale en ese intervalo, la alarma no se entera. Peor todavía: cuando deja de haber movimiento, la sirena sigue sonando hasta terminar su espera.

La solución no es acortar las esperas sino sacarlas del programa. En lugar de «esperar doscientos milisegundos», el programa pregunta en cada vuelta si ya **pasaron** doscientos milisegundos desde el último cambio de tono, comparando el reloj. Así la repetición gira de manera continua, lee el sensor cada vez, y el tono cambia cuando corresponde.

Es la misma idea que sostiene el filtro de rebote del laboratorio anterior, y la que permitirá, en el capítulo 7, que un servidor web atienda visitas mientras sigue midiendo. Conviene retenerla: **un programa que espera es un programa que no vigila**.

PROGRAMA

Archivo `lab_2_4_pir_alarma.py`

```
from machine import Pin, PWM
import time
```

```

buzzer = PWM(Pin(5))
buzzer.duty_u16(0)          # arranca en silencio
pir = Pin(4, Pin.IN)

TONOS = (400, 800)          # Hz, se alternan para formar la sirena
TRAMO_MS = 200
MITAD = 32767               # media escala de duty_u16

print("Alarma activa. Espere un minuto a que el PIR se estabilice.")

tono = 0
cambio = time.ticks_ms()
sonando = False

while True:
    hay_movimiento = pir.value() == 1
    ahora = time.ticks_ms()

    if hay_movimiento:
        if not sonando:
            print("Movimiento detectado")
            sonando = True
            buzzer.duty_u16(MITAD)

            # se cambia de tono cada tramo, sin detener el programa
            if time.ticks_diff(ahora, cambio) >= TRAMO_MS:
                tono = 1 - tono
                buzzer.freq(TONOS[tono])
                cambio = ahora
        else:
            if sonando:
                print("Sin movimiento")
                sonando = False
                buzzer.duty_u16(0)

    time.sleep_ms(20)

```

Las dos banderas

sonando recuerda si la sirena ya está activa, y sirve para que los mensajes se escriban **una sola vez** en cada cambio y no en cada vuelta de la repetición, que son cincuenta por segundo. Sin esa bandera, la consola queda inutilizable. tono guarda cuál de los dos suena, y 1 - tono lo alterna entre 0 y 1 sin necesidad de un if.

QUÉ DEBE OBSERVARSE

Pasada la estabilización, al mover la mano frente al sensor la sirena arranca de inmediato y alterna dos tonos. Al retirarse, calla. La consola escribe una sola línea por evento. La prueba que vale la pena hacer es pasar rápido frente al sensor y comprobar que igual detecta: el programa nunca deja de mirar.

SI ALGO NO SALE

Síntoma	Causa habitual
Dispara sin que haya nadie	No pasó el minuto de estabilización, o el sensor apunta a una fuente de calor: una ventana con sol, una lámpara incandescente, una salida de aire.
El buzzer suena con una sola nota	Es un buzzer activo. Con éste no se puede construir la sirena.
La sirena sigue un rato después de que el movimiento cesó	No es el programa: es el potenciómetro de tiempo de retención del propio sensor, que mantiene su salida alta. Girarlo hacia el mínimo.
No detecta nada	Verificar que el PIR esté alimentado con 5 V (VIN) y no con 3V3. Con 3,3 V el módulo no trabaja de manera confiable.

El mismo algoritmo en...

En el área eléctrica y electrónica, el PIR se reemplaza por una barrera fotoeléctrica o un detector de presencia en zona peligrosa, y el buzzer por una baliza sonora o el corte de mando de la máquina. El programa no cambia: lo que cambia es que la salida, en lugar de hacer ruido, abre un contacto de seguridad.

En el área mecánica, es la alarma perimetral del vehículo. La misma estructura —sensor de presencia, sirena de dos tonos, vigilancia ininterrumpida— se usa con el sensor de ocupación del asiento o con el de apertura de puertas.

2.5 Una carga que no puede colgar del pin: el relé

OBJETIVO

Encender y apagar un foco de 220 V desde la placa, con un pulsador, sin que la red toque en ningún punto al microcontrolador.

MATERIALES

Cantidad	Componente	Observación
1	Módulo de relé de 1 canal	con optoacoplador y puente H/L
1	Fuente de continua	de la tensión de la bobina: 5 V o 12 V
1	Pulsador	
1	Portalámparas con foco	de 220 V, el de menor potencia que haya
—	Cables de conexión	y cable de red del lado de la carga

Antes de tocar nada

Este es el único laboratorio del libro donde hay tensión de red, y la tensión de red mata. Cuatro reglas, y ninguna es negociable:

Nada de la red va a la protoboard. La protoboard no aísla, sus contactos están a dos milímetros y se sueltan solos. El lado de 220 V se cablea con conductor de instalación y se ajusta a los tornillos de la bornera del módulo, que para eso están.

Se arma desenchufado y se prueba enchufado una sola vez. Primero se monta todo el lado de mando y se comprueba que el relé chasquea; recién con el chasquido andando se cablea el foco y se enchufa. Cualquier cambio posterior, desenchufando.

El relé corta la fase, no el neutro. Cortando el neutro la lámpara también se apaga, pero el portalámparas queda con tensión y quien cambie el foco se entera a la mala.

Los dos lados no se tocan. Ninguna masa del mando va a la red, ningún conductor de la red se apoya sobre la placa, y las manos no van a la bornera mientras esté enchufado. Si en el laboratorio hay un interruptor diferencial, trabaje aguas abajo de él.

RAZONAMIENTO

El capítulo viene repitiendo que una carga mayor que un indicador no se cuelga del pin. Ahora toca decir con qué se la reemplaza, y la respuesta es un **relé**: un interruptor que en lugar de un dedo tiene un electroimán.

Lo que lo vuelve interesante no es que amplifique —de hecho no amplifica nada— sino que **separa**. Adentro hay dos circuitos que no se tocan: una bobina, que se alimenta con unos pocos miliamperios de continua, y un juego de contactos capaz de cortar diez amperios de alterna. Entre los dos no hay más que aire y un resorte. Por eso el pin del ESP32 puede gobernar un motor de 220 V sin enterarse nunca de que existe.

Los módulos que se consiguen armados agregan una segunda separación, esta vez electrónica: el **optoacoplador**, un LED y un fototransistor encerrados en el mismo encapsulado, que se comunican con luz y no con

cobre. Es el rectángulo negro que se ve junto a la entrada. Con él, entre el pin y la red hay dos barreras en serie.

El módulo trae además dos cosas que hay que mirar antes de conectar nada, y que son la causa de la mitad de los desconciertos de este laboratorio: la tensión de la bobina y el puente de disparo.

La bobina no se alimenta de la placa

En el encapsulado azul del relé está impresa la tensión de su bobina: 5 VDC o 12 VDC . Es lo primero que hay que leer.

Con una bobina de **12 V** no hay discusión: hace falta una fuente aparte. El pin VCC del módulo va al positivo de esa fuente y el GND del módulo va al negativo **y también al GND de la placa**, porque el optoacoplador necesita que las dos referencias sean la misma para reconocer la orden. Es la misma masa común de siempre, y olvidarla es la falla número uno: el módulo parece muerto.

Con una bobina de **5 V** se lo puede alimentar del pin VIN , pero sólo si el módulo es de un canal. Cada bobina consume entre 70 y 90 mA, y el regulador de la placa no sostiene dos relés y el WiFi a la vez. Un módulo de dos o cuatro canales lleva fuente propia siempre.

El puente H/L, o por qué hay programas que funcionan al revés

Al costado de la entrada hay un puente de tres patas rotulado H y L . Decide con qué nivel dispara el relé: en H se activa con un **1** en la entrada, y en L con un **0**.

De ahí salen los programas de relé que circulan y que parecen contradecirse: uno escribe `relay.value(1)` para encender y otro escribe `relay.value(0)` . Los dos están bien; lo que cambia es el puente.

Los módulos chinos más comunes vienen de fábrica en **L**, activo en bajo. Este libro los usa así, y el programa lo dice en una constante en lugar de repartir unos y ceros por el código. Si su módulo no tiene puente, mire el LED del canal: si enciende con la entrada al aire, es activo en bajo.

EL CIRCUITO

la bobina se alimenta de su propia fuente; lo único que sale de la placa es la orden

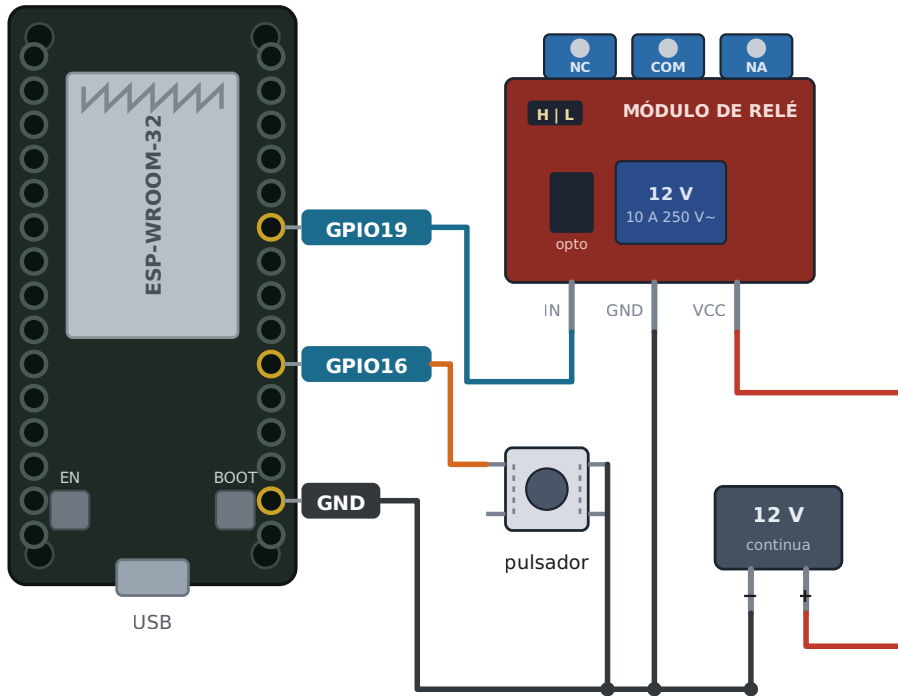


Figura 2.5 — El lado de mando. Lo único que sale de la placa es la orden: la bobina se alimenta de su propia fuente, y las tres masas —la de la placa, la del módulo y la de la fuente— tienen que ser la misma.

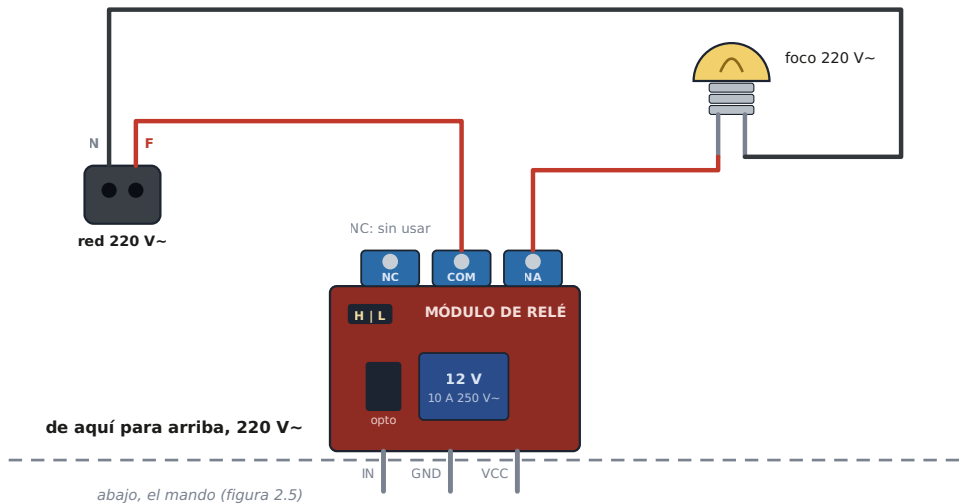


Figura 2.6 — El lado de la carga, que es un circuito aparte. La fase entra por COM y sale por NA, que sólo conduce con el relé activado; el neutro va directo al foco. NC queda sin usar: es el contacto que conduce con el relé en reposo.

NA y NC

La bornera tiene tres tornillos. **COM** es el común, el que siempre está conectado al circuito. **NA** —normalmente abierto— conduce sólo cuando el relé está activado, y es el que se usa para encender algo. **NC** —normalmente cerrado— hace lo contrario: conduce en reposo y corta al activarse.

La elección no es un detalle de cableado sino de seguridad. Si lo que se gobierna tiene que **quedar apagado** cuando el sistema falla o se queda sin alimentación, va en NA. Si tiene que quedar encendido —una luz de emergencia, un freno que se suelta— va en NC.

PROGRAMA

El programa es el del pulsador del laboratorio 2.3 con tres cambios. El primero es la constante que absorbe el puente H/L, para no tener que recordar en cada línea si el módulo es activo en alto o en bajo. El segundo es el orden de los estados: **el relé arranca apagado y se enciende al**

presionar, nunca al revés. El tercero es que aquí el relé sigue al pulsador mientras está presionado, en lugar de alternar en cada toque.

El filtro de rebote del 2.3 queda igual, y con un relé importa más que con un LED: cada rebote que se cuele es una maniobra mecánica de más, y las maniobras de un relé están contadas.

Archivo `lab_2_5_rele.py`

```
from machine import Pin
import time

# El puente del modulo decide con que nivel dispara. En L -lo mas
# comun- el rele se activa con un 0. Cambiando esta linea el programa
# sirve para los dos, y no hay que tocar nada mas.
ACTIVO_EN_BAJO = True

REBOTE_MS = 50

rele = Pin(19, Pin.OUT)
boton = Pin(16, Pin.IN, Pin.PULL_UP)

estado = False
ultimo_cambio = time.ticks_ms()

def mandar(encendido):
    """Traduce 'encendido' al nivel que pide este modulo."""
    if ACTIVO_EN_BAJO:
        rele.value(0 if encendido else 1)
    else:
        rele.value(1 if encendido else 0)

mandar(False)          # apagado antes que nada, pase lo que pase

print("Mantenga presionado para encender. Ctrl+C para salir.")

while True:
    # Con PULL_UP el pulsador en reposo lee 1 y presionado lee 0.
    presionado = not boton.value()
    ahora = time.ticks_ms()

    hubo_cambio = presionado != estado
    paso_el_rebote = time.ticks_diff(ahora, ultimo_cambio) > REBOTE_MS

    if hubo_cambio and paso_el_rebote:
```

```
estado = presionado
ultimo_cambio = ahora
mandar(estado)
print("foco encendido" if estado else "foco apagado")

time.sleep_ms(10)
```

Por qué el relé no parpadea

La versión que aparece primero cuando uno arma este laboratorio es otra: el pulsador hace parpadear el foco, con dos `sleep(0.5)`. Funciona, y en clase se ve bien. Pero hay dos razones para no dejarla.

La primera es mecánica. Medio segundo encendido y medio apagado son dos maniobras por segundo: **7200 por hora**. Un relé de estos aguanta del orden de cien mil maniobras con carga, de modo que el parpadeo se lo come en unas dos semanas de clase. Y cada cierre sobre un foco incandescente frío es una corriente de irrupción de unas diez veces la nominal, justo sobre los contactos.

La segunda es de criterio. Escrito con el pulsador en la rama que apaga, el foco **parpadea cuando el pulsador no está** —o cuando se corta su cable, que con `PULL_UP` es exactamente lo mismo que soltarlo. La falla segura está al revés: si algo se desconecta, la carga tiene que quedar apagada. Por eso el programa de arriba enciende al presionar y no al soltar.

QUÉ DEBE OBSERVARSE

Con el foco todavía sin cablear, al presionar el pulsador se oye el chasquido del relé y se enciende el LED del canal en el módulo. Ese chasquido es toda la comprobación del lado de mando: si está, el resto es cableado.

Con el foco conectado, enciende al presionar y apaga al soltar, sin retardo apreciable. Y la prueba que vale la pena hacer es desconectar el cable del pulsador con el montaje andando: el foco tiene que quedar **apagado**.

SI ALGO NO SALE

Síntoma	Causa habitual
El relé no chasquea y su LED no enciende	Falta la masa común. El GND del módulo tiene que ir al GND de la placa y al negativo de la fuente de la bobina.
El LED del canal enciende pero el relé no chasquea	La bobina no tiene su tensión. Leer qué dice el encapsulado: un relé de 12 V alimentado con 5 V hace exactamente esto.
Funciona al revés: apaga al presionar	El puente H/L está en la otra posición. Cambiar ACTIVO_EN_BAJO en lugar de recablear.
El relé chasquea pero el foco no enciende	El foco está en NC en lugar de NA, o la fase no llega a COM. Desenchufar antes de revisar.
La placa se reinicia cada vez que el relé conmuta	La bobina se está alimentando de la placa y el pico de corriente hunde la tensión. Fuente aparte.

El mismo algoritmo en...

En el área eléctrica y electrónica, es el mando de cualquier arranque: el relé excita la bobina de un contactor y el contactor mueve el motor. La cadena pin → relé → contactor → motor es la que separa la lógica de la potencia en todo tablero, y el programa es el mismo con el pulsador reemplazado por un sensor de nivel o un térmico.

En el área mecánica, es el relé del vehículo, que está en todos lados: luces, bomba de combustible, electroventilador, bocina. La llave del tablero no lleva los treinta amperios del motor del ventilador, lleva la corriente de una bobina; el relé hace el resto. Cambiar el pulsador por la señal de un sensor de temperatura es el electroventilador completo.

EJERCICIOS PROPUESTOS

1. En el laboratorio 2.2, agregue una cuarta fase de intermitente nocturno en la que el amarillo destelle cinco veces antes de reanudar el ciclo. ¿Cuántas líneas hubo que tocar?
2. Modifique el semáforo para que un pulsador de peatón, leído como en el laboratorio 2.3, adelante la fase roja. Tenga en cuenta que el programa no debe quedarse esperando el pulsador.
3. En el laboratorio 2.3, haga que una pulsación breve encienda el LED y una sostenida de más de un segundo lo apague. Ayuda: hace falta recordar el instante en que se presionó.
4. En el laboratorio 2.4, cuente cuántas detecciones ocurrieron desde que arrancó el programa y muéstrelas en la consola en cada evento.
5. Reescriba el laboratorio 2.4 usando `time.sleep(0.2)` para alternar los tonos. Mida cuánto tarda en reaccionar a un movimiento breve y compare con la versión del libro. Explique la diferencia.
6. Los tres LED del semáforo consumen unos 4 mA cada uno. Si se quisiera reemplazar cada uno por una lámpara de 220 V, ¿qué habría que agregar entre el pin y la lámpara? ¿Cambiaría el programa?
7. En el laboratorio 2.5, haga que el pulsador funcione como interruptor: una pulsación enciende y la siguiente apaga, en lugar de encender mientras se mantiene. Ayuda: hay que detectar el *flanco*, no el nivel.
8. Un módulo de relé de cuatro canales con bobinas de 5 V consume unos 80 mA por canal. ¿Puede alimentarse de VIN? Justifique con el dato del regulador de la placa, y diga qué pasaría con los cuatro activados.
9. Explique por qué el laboratorio 2.5 pone el foco en NA y no en NC. ¿En qué caso elegiría NC, y qué tendría que cambiar en el programa?

Señales analógicas

Una señal digital sólo dice sí o no. La mayoría de las magnitudes físicas no se dejan describir así: una temperatura no está «presente o ausente», vale 23,4 °C. Lo mismo la presión, el caudal, la posición de un pedal, el nivel de un tanque. Esas magnitudes llegan al microcontrolador como una **tensión continua** que varía con la medición, y traducirla a un número es el trabajo de este capítulo.

El camino tiene tres tramos y conviene no confundirlos. Primero el sensor convierte la magnitud en tensión; después hay que acondicionar esa tensión para que entre sin riesgo en el pin; y recién entonces el conversor la vuelve un número, que todavía no está en las unidades que interesan. Los cinco laboratorios recorren esos tres tramos, y el último —la calibración— es el que separa una lectura que parece correcta de una que lo es.

3.1 Leer una tensión: el potenciómetro

OBJETIVO

Convertir la posición de una perilla en un número, y entender qué significan los dos ajustes del conversor.

MATERIALES

Cantidad	Componente
1	Potenciómetro de 10 kΩ
—	Cables de conexión

EL CIRCUITO

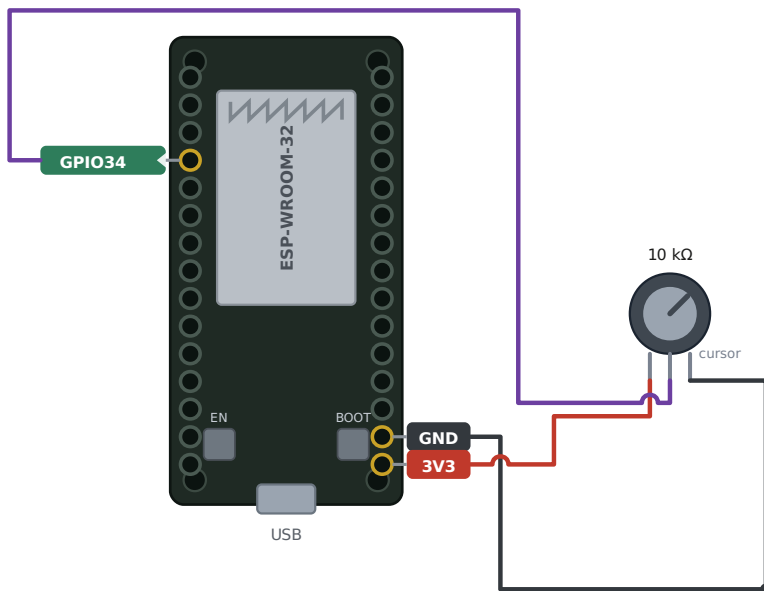


Figura 3.1 — El potenciómetro con sus dos extremos entre 3V3 y masa. El cursor —la pata del medio— entrega la tensión, y va a GPIO34, que pertenece al ADC1 y está en la columna izquierda de la placa.

Por qué todos los sensores de este libro van al ADC1

El ESP32 tiene dos conversores. El **ADC1** ocupa los pines GPIO32 a GPIO39; el **ADC2**, los GPIO 0, 2, 4, 12 a 15 y 25 a 27. La diferencia es que **el ADC2 deja de funcionar cuando el WiFi está encendido**, porque la radio lo usa internamente.

En este capítulo todavía no hay WiFi y el ADC2 andaría. Pero en el capítulo 7 estos mismos montajes van a publicar su lectura en internet, y entonces dejarían de medir. Cablearlos al ADC1 desde ahora ahorra rehacer el montaje después, y es el motivo de que en los cuatro diagramas de este capítulo el cable de señal cruce al otro lado de la placa.

RAZONAMIENTO

El conversor tiene dos ajustes y los dos hacen falta.

La **resolución** dice en cuántos escalones se parte el rango. Con `WIDTH_12BIT` son 4096 escalones, de 0 a 4095, que es lo máximo que da el ESP32.

La **atenuación** dice cuál es el rango. Ésta es la que sorprende: el conversor no mide de 0 a 3,3 V por defecto, sino hasta poco más de 1 V. Para abarcar los 3,3 V hay que pedir `ATTN_11DB` explícitamente. Sin esa línea, la lectura satura en 4095 apenas pasa de 1,1 V y parece que el sensor estuviera roto.

Atenuación	Abarca hasta	Cuándo conviene
<code>ATTN_0DB</code>	$\approx 1,1$ V	Señales muy chicas. Máxima resolución sobre un tramo angosto.
<code>ATTN_2_5DB</code>	$\approx 1,5$ V	Poco usada.
<code>ATTN_6DB</code>	$\approx 2,0$ V	El LM35 del laboratorio 3.3, que no pasa de 1 V.
<code>ATTN_11DB</code>	$\approx 3,3$ V	El caso general, y el que usa este libro salvo aviso.

La conversión a voltios es una regla de tres: si 4095 cuentas corresponden a 3,3 V, entonces n cuentas corresponden a $n \times 3,3 / 4095$. Nada más.

PROGRAMA

Archivo `lab_3_1_potenciometro.py`

```
from machine import Pin, ADC
import time

# El ADC1 corresponde a los pines GPIO32 a GPIO39. Ver el recuadro.
PIN_ENTRADA = 34

TENSION_MAXIMA = 3.3      # con atenuacion de 11 dB
CUNTAS_MAXIMAS = 4095    # con resolucio de 12 bits
```

```

entrada = ADC(Pin(PIN_ENTRADA))
entrada.atten(ADC.ATTN_11DB)      # rango de entrada de 0 a 3,3 V
entrada.width(ADC.WIDTH_12BIT)    # resolucion de 12 bits

def leer_tension():
    """Devuelve las cuentas crudas y su equivalente en voltios."""
    cuentas = entrada.read()
    return cuentas, cuentas * TENSION_MAXIMA / CUENTAS_MAXIMAS

def main():
    print("Gire la perilla del potenciómetro.")
    print("")
    print("  cuentas      tension      barra")

    while True:
        cuentas, tension = leer_tension()

        # una barra de veinticuatro posiciones para ver la variacion
        largo = int(cuentas * 24 / CUENTAS_MAXIMAS)
        barra = "#" * largo + "." * (24 - largo)

        print("    {:5d}      {:5.3f} V      {}".format(
            cuentas, tension, barra))
        time.sleep(0.5)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

cuentas	tension	barra
0	0.000 V	
1024	0.825 V	#####.....
2048	1.650 V	#####.....
4095	3.300 V	#####

Girando la perilla de un extremo al otro las cuentas recorren todo el rango y la barra se llena. Con la perilla quieta, la lectura no se queda del todo fija: baila unas pocas cuentas. Eso es ruido del propio conversor y no un defecto del montaje; el laboratorio 3.5 se ocupa de él.

SI ALGO NO SALE

Síntoma	Causa habitual
Marca 4095 casi siempre	Falta <code>atten(ADC.ATTN_11DB)</code> : el conversor satura por encima de 1,1 V.
No baja de unas 100 cuentas ni sube de 4000	Uno de los dos extremos del potenciómetro no está conectado.
La lectura no cambia al girar	El cursor no es la pata que se cableó. Es la del medio.
Salta sola entre valores muy distintos	Mal contacto en la protoboard, o se usó un pin del ADC2 con el WiFi encendido.

El mismo algoritmo en...

En el área eléctrica y electrónica, el potenciómetro es el mando manual de consigna de un variador de velocidad, y la misma lectura sirve para cualquier sensor con salida de 0 a 10 V previo acondicionamiento — que es el laboratorio siguiente.

En el área mecánica, es exactamente el sensor de posición del acelerador: un potenciómetro solidario al pedal. El programa que lee la perilla y el que lee el pedal son el mismo.

3.2 Señales de 5 V: el divisor de tensión

OBJETIVO

Leer una señal que llega con más tensión de la que el pin admite, sin dañar la placa.

MATERIALES

Cantidad	Componente
1	Potenciómetro de 10 k Ω , como fuente de señal variable
1	Resistencia de 10 k Ω (R1)
1	Resistencia de 15 k Ω (R2)

EL CIRCUITO

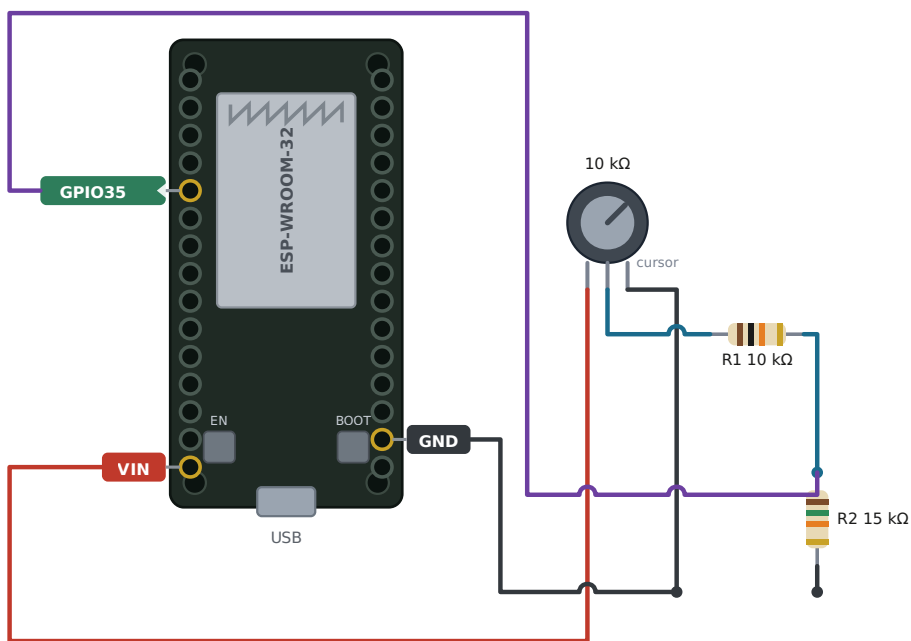


Figura 3.2 — El potenciómetro alimentado con los 5 V de VIN hace de sensor de 5 V. R1 y R2 forman el divisor, y el punto entre las dos es lo único que entra al conversor.

El pin no tolera 5 V

Los pines del ESP32 trabajan con 3,3 V. Aplicarles 5 V los daña, a veces de inmediato y a veces despacio, que es peor porque el equipo empieza a fallar semanas después.

Muchos sensores industriales entregan de 0 a 10 V y muchos sensores de vehículo trabajan con 5 V. Conectarlos directamente no es una opción: hay que bajar la tensión primero, y eso es lo que hace el divisor.

RAZONAMIENTO

Dos resistencias en serie entre la señal y masa reparten la tensión en proporción a sus valores. Midiendo en el punto del medio se obtiene:

$$V_{\text{salida}} = V_{\text{entrada}} * R2 / (R1 + R2)$$

Con R1 de 10 kΩ y R2 de 15 kΩ, el factor es $15/25 = 0,6$. Una entrada de 5 V se convierte en 3,0 V, que entra con margen en el pin. Para recuperar la tensión original en el programa hay que multiplicar por el inverso:

$$\text{FACTOR} = (R1 + R2) / R2 \quad \# \ 25/15 = 1,667$$

Entrada real	En el pin	Lo que reconstruye el programa
0 V	0,00 V	0,00 V
2,5 V	1,50 V	2,50 V
5,0 V	3,00 V	5,00 V
10 V	6,00 V	daña el pin

La última fila importa: este divisor está calculado para 5 V. Para una señal de 0 a 10 V hay que recalcularlo —con R1 de 22 kΩ y R2 de 10 kΩ el factor deja 3,1 V a fondo de escala—, y el ejercicio 2 lo propone.

Por qué no se eligen resistencias mucho más grandes

Con valores altos el divisor consume menos, y eso suena bien. Pero el conversor del ESP32 necesita que la fuente que lo alimenta tenga una resistencia interna baja, por debajo de unos 10 kΩ, o la lectura se vuelve inestable y baja. Con 10 k y 15 k se está en el límite cómodo. El laboratorio 3.4, que usa 100 kΩ por obligación, muestra cómo se resuelve cuando no queda otra.

ANTES DE CONECTAR EL PIN

Mida primero, conecte después

Arme el divisor, alimente el potenciómetro desde VIN y **mida con el multímetro** la tensión en el punto medio, con la perilla al máximo. Debe leer alrededor de **3,0 V**. Si lee más de 3,3 V, no conecte el pin: las dos resistencias están intercambiadas.

Es un minuto de trabajo y evita una placa quemada. En el taller esta costumbre se llama «comprobar antes de energizar» y vale exactamente igual aquí.

PROGRAMA

Archivo `lab_3_2_divisor.py`

```
from machine import Pin, ADC
import time

PIN_ENTRADA = 35

R1 = 10000.0          # ohmios, del cursor al punto medio
R2 = 15000.0          # ohmios, del punto medio a masa
FACTOR = (R1 + R2) / R2  # cuanto hay que multiplicar para
                        # recuperar la tension original

TENSION_MAXIMA = 3.3
CUENTAS_MAXIMAS = 4095

entrada = ADC(Pin(PIN_ENTRADA))
entrada.atten(ADC.ATTN_11DB)
entrada.width(ADC.WIDTH_12BIT)
```

```

def leer_entrada():
    """Devuelve la tension en el pin y la tension real de la senal."""
    cuentas = entrada.read()
    en_el_pin = cuentas * TENSION_MAXIMA / CUENTAS_MAXIMAS
    return en_el_pin, en_el_pin * FACTOR

def main():
    print("Divisor de {:.0f}k / {:.0f}k, factor {:.3f}".format(
        R1 / 1000, R2 / 1000, FACTOR))
    print("")
    print("   en el pin       senal real")

    while True:
        en_el_pin, real = leer_entrada()

        aviso = ""
        if en_el_pin > 3.1:
            aviso = "   <-- revise el divisor"

        print("   {:.5.3f} V       {:.5.3f} V{}".format(
            en_el_pin, real, aviso))
        time.sleep(0.5)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Con el cursor al mínimo las dos columnas marcan cerca de cero; al máximo, 3,0 V en el pin y 5,0 V reconstruidos. Comparando la segunda columna con el multímetro puesto en el cursor del potenciómetro, las dos cifras deben coincidir dentro de unas décimas. Lo que sobre de esa diferencia es lo que corrige el laboratorio 3.5.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el acondicionamiento de cualquier transmisor de 0 a 10 V: nivel, presión, caudal. Junto con la calibración del laboratorio 3.5 es todo lo que hace falta para leer instrumentación de planta con un ESP32.

En el área mecánica, casi todos los sensores del vehículo trabajan sobre 5 V: posición de mariposa, caudalímetro, sensor de presión del múltiple. Ninguno se conecta directamente al pin; todos pasan por un divisor como éste.

3.3 Un sensor lineal: el LM35

OBJETIVO

Medir temperatura con un sensor cuya salida es proporcional, y ver por qué conviene elegir la atenuación en vez de dejar la de siempre.

MATERIALES

Cantidad	Componente	Observación
1	Sensor LM35	encapsulado TO-92, como un transistor

EL CIRCUITO

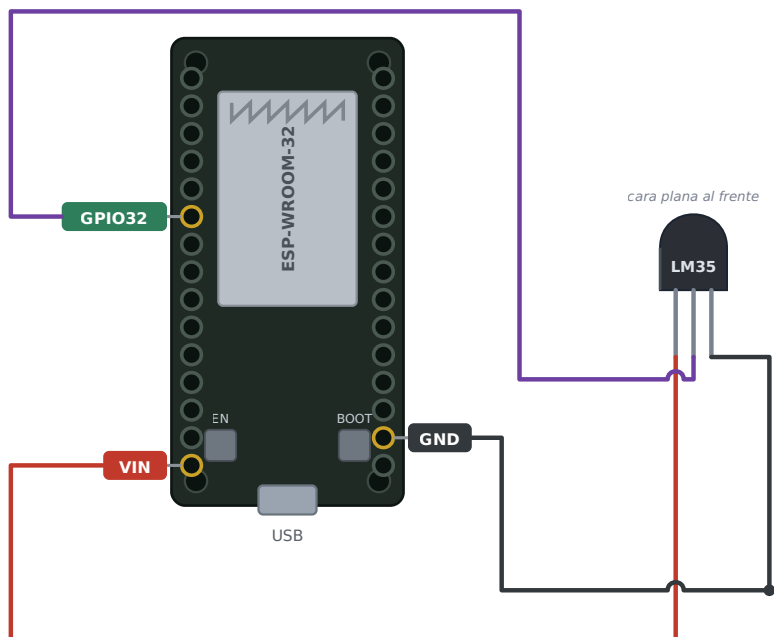


Figura 3.3 — El LM35 mirado de frente, con la cara plana hacia quien lo conecta. Se alimenta de VIN, no de 3V3.

El LM35 necesita al menos 4 V

La hoja de datos pide de 4 a 30 V de alimentación. Con los 3,3 V de la placa el sensor **funciona igual** —enciende, entrega una tensión que sube con la temperatura, no da ningún error— pero queda fuera de especificación y **mide de menos**. Es la peor clase de falla: no hay síntoma, sólo un número equivocado.

Por eso va a VIN, que entrega los 5 V del USB. Y conectarlo al revés lo calienta en segundos: si quema al tocarlo, desconecte de inmediato y revise la orientación.

RAZONAMIENTO

El LM35 entrega **10 mV por cada grado centígrado**. A 25 °C su salida vale 250 mV; a 100 °C, un volt. La conversión es una división y no hay más.

Lo interesante es la atenuación. Dentro del rango útil el sensor nunca pasa de 1 V, de modo que pedir `ATTN_11DB` —que abarca hasta 3,3 V— desperdicia dos tercios del conversor: la medición ocupa apenas un tercio de las 4096 cuentas. Con `ATTN_6DB`, que llega a unos 2,0 V, la misma medición ocupa la mitad y **se duplica la resolución**.

Ese es el criterio general: elegir la atenuación más chica en la que la señal todavía entre entera, con algo de margen. No es un ajuste fino, son grados de resolución regalados.

Atenuación	Rango	Cuentas por grado	Resolución
<code>ATTN_11DB</code>	0 - 3,3 V	12,4	≈ 0,08 °C
<code>ATTN_6DB</code>	0 - 2,0 V	20,5	≈ 0,05 °C

La diferencia es menor que el error del propio sensor, así que en este caso no cambia el resultado. Importa el criterio, que sí cambia el resultado cuando la señal es pequeña de verdad.

PROGRAMA

Archivo `lab_3_3_lm35.py`

```
from machine import Pin, ADC
import time

PIN_ENTRADA = 32
MILIVOLTIOS_POR_GRADO = 10.0
MUESTRAS = 16          # promedio para atenuar el ruido del conversor

# Con atenuacion de 6 dB el conversor llega a unos 2,0 V, que serian
# 200 C: de sobra, y con el doble de resolucion que usando los 3,3 V.
TENSION_MAXIMA = 2.0
CUENTAS_MAXIMAS = 4095

entrada = ADC(Pin(PIN_ENTRADA))
entrada.atten(ADC.ATTN_6DB)
entrada.width(ADC.WIDTH_12BIT)

def leer_milivoltios():
    """Promedia varias lecturas y devuelve la salida en milivoltios."""
    suma = 0
```

```

    for _ in range(MUESTRAS):
        suma = suma + entrada.read()
        time.sleep_ms(2)
    cuentas = suma / MUESTRAS
    return cuentas * TENSION_MAXIMA * 1000 / CUENTAS_MAXIMAS

def leer_temperatura():
    return leer_milivoltios() / MILIVOLTIOS_POR_GRADO

def main():
    print("Sensor LM35 en GPIO{}".format(PIN_ENTRADA))
    print("")
    print("  salida      temperatura")

    while True:
        mv = leer_milivoltios()
        print("    {:6.1f} mV    {:5.1f} C".format(
            mv, mv / MILIVOLTIOS_POR_GRADO))
        time.sleep(1)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

A temperatura ambiente marca entre 18 y 25 °C. Tomando el sensor con los dedos sube varios grados en pocos segundos y baja al soltarlo: la respuesta es rápida porque el encapsulado es pequeño.

Conviene comparar con un termómetro. La diferencia que aparezca —y va a aparecer— es lo que corrige el laboratorio 3.5.

SI ALGO NO SALE

Síntoma	Causa habitual
Marca unos grados de menos, siempre	Está alimentado con 3V3. Es el caso del recuadro.
Se calienta al tocarlo	Está al revés. Desconecte de inmediato.
Marca cerca de cero y no cambia	La pata del medio no llegó al pin, o se cableó la de al lado.
Marca valores altísimos	La atenuación quedó en 11 dB pero TENSION_MAXIMA dice 2,0: las dos constantes tienen que corresponderse.

3.4 Un sensor que no es lineal: el termistor NTC

OBJETIVO

Medir temperatura con un sensor cuya resistencia varía de manera exponencial, y resolver el problema que eso trae al conversor.

MATERIALES

Cantidad	Componente	Observación
1	Termistor NTC de 100 k Ω	Beta 3950, el de sonda con cable
1	Resistencia de 100 k Ω	la resistencia fija del divisor
1	Condensador de 100 nF	no es opcional , ver el recuadro

EL CIRCUITO

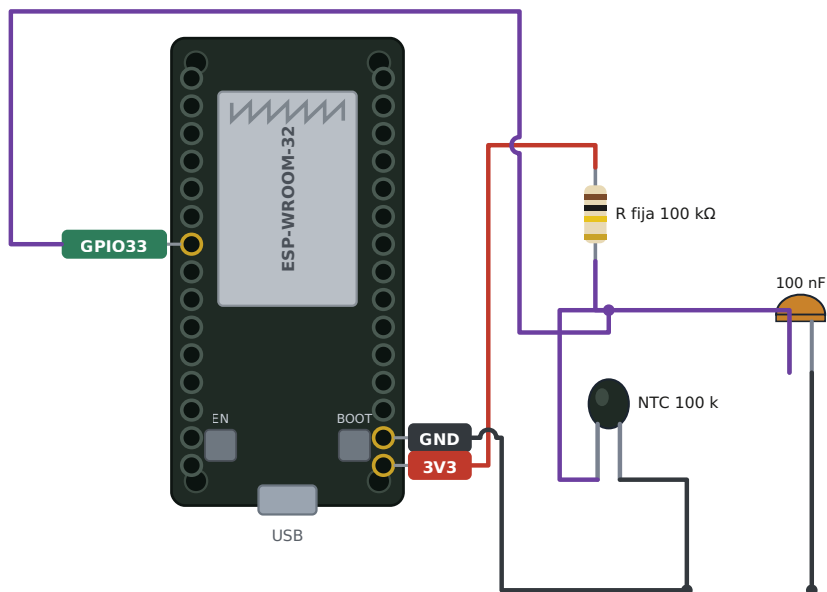


Figura 3.4 — El termistor y la resistencia fija forman un divisor; el punto medio es la señal. El condensador va en paralelo con el termistor, entre el pin y masa.

Cien kilohmios son demasiados para el conversor

El conversor del ESP32 no mide una tensión de manera pasiva: internamente carga un pequeño condensador con la señal, y para hacerlo necesita que la fuente tenga una resistencia baja, **por debajo de unos 10 kΩ**.

Un divisor de 100 k Ω presenta unos 50 k Ω , cinco veces más de lo admisible. La consecuencia es que la lectura sale baja e inestable, y parece ruido cuando en realidad es que el convertor no alcanza a cargarse.

El condensador de **100 nF** entre el pin y masa resuelve el problema: es el que entrega la carga instantánea que el conversor pide, y el divisor sólo tiene que reponerla despacio. Sin él este laboratorio no funciona, y el síntoma no dice nada de la causa.

RAZONAMIENTO

Un termistor no entrega tensión: cambia de resistencia. Para leerlo se lo pone en serie con una resistencia fija y se mide el punto medio, igual que en el laboratorio 3.2. De la tensión medida se despeja la resistencia del termistor:

$$R_{ntc} = R_{fija} * V_{medida} / (V_{alimentacion} - V_{medida})$$

Y ahí empieza lo distinto. La resistencia no varía en proporción a la temperatura sino de manera exponencial, y la relación se describe con la **ecuación de Beta**, donde las temperaturas van en kelvin:

$$1/T = 1/T_0 + \ln(R / R_0) / BETA$$

Constante	Qué es	Este termistor
R0	Resistencia a la temperatura nominal	100 kΩ
T0	Temperatura nominal	25 °C = 298,15 K
BETA	Constante del material, la da el fabricante	3950 K

Las tres salen de la hoja de datos y están en constantes al principio del programa. Si el termistor es de otro valor —los de 10 kΩ son los más comunes— se cambian esas tres líneas y **la resistencia fija**, que debe ser del mismo valor que el nominal del termistor.

PROGRAMA

Archivo `lab_3_4_termistor.py`

```
from machine import Pin, ADC
import math
import time

PIN_ENTRADA = 33

R_FIJA = 100000.0      # ohmios, la resistencia de arriba
R_NOMINAL = 100000.0  # ohmios del termistor a T_NOMINAL
T_NOMINAL = 25.0       # grados centigrados
BETA = 3950.0          # kelvin, constante del material
```

```

ALIMENTACION = 3.3
CERO_ABSOLUTO = 273.15
CUENTAS_MAXIMAS = 4095
MUESTRAS = 16

entrada = ADC(Pin(PIN_ENTRADA))
entrada.atten(ADC.ATTN_11DB)
entrada.width(ADC.WIDTH_12BIT)

def leer_tension():
    suma = 0
    for _ in range(MUESTRAS):
        suma = suma + entrada.read()
        time.sleep_ms(2)
    return (suma / MUESTRAS) * ALIMENTACION / CUENTAS_MAXIMAS

def resistencia_del_ntc(tension):
    """Despeja la resistencia. None si el sensor esta desconectado."""
    if tension <= 0 or tension >= ALIMENTACION:
        return None
    return R_FIJA * tension / (ALIMENTACION - tension)

def temperatura(resistencia):
    """Aplica la ecuacion de Beta. Devuelve grados centigrados."""
    t0 = T_NOMINAL + CERO_ABSOLUTO
    inversa = 1.0 / t0 + math.log(resistencia / R_NOMINAL) / BETA
    return 1.0 / inversa - CERO_ABSOLUTO

def main():
    print("Termistor NTC de {:.0f}k, Beta {:.0f}".format(
        R_NOMINAL / 1000, BETA))
    print("")
    print(" tension      resistencia      temperatura")

    while True:
        tension = leer_tension()
        r = resistencia_del_ntc(tension)

        if r is None:
            print(" {:.5.3f} V      sensor desconectado".format(tension))
        else:
            print(" {:.5.3f} V      {:.8.0f} ohm      {:.5.1f} C".format(
                tension, r, temperatura(r)))

```

```
time.sleep(1)

if __name__ == "__main__":
    main()
```

Por qué se comprueba el rango antes de dividir

Si el termistor se desconecta, la tensión sube hasta la alimentación y la fórmula de la resistencia divide por cero. Si se pone en corto, baja a cero y el logaritmo recibe un cero. Los dos casos detienen el programa con una excepción poco clara.

Comprobar el rango antes convierte una falla del cableado en un mensaje que dice qué pasa. En un equipo que queda instalado, ésta es la diferencia entre «sensor desconectado» y un programa muerto.

QUÉ DEBE OBSERVARSE

A temperatura ambiente la resistencia ronda los 100 kΩ y la temperatura cae cerca de 25 °C, que es justamente la condición nominal del sensor. A 0 °C la resistencia sube a unos 336 kΩ y a 80 °C baja a 12,7 kΩ. Tomando la sonda con la mano la resistencia **baja** —es lo que significa NTC, coeficiente negativo— y la temperatura sube.

Vale la pena mirar las dos primeras columnas juntas. Entre 20 y 30 °C la resistencia cae unos **45 000 Ω**; entre 75 y 85 °C, el mismo salto de diez grados mueve apenas **4 000 Ω**, once veces menos. Eso es la no linealidad vista de frente, y es la razón por la que un termistor mide muy bien cerca de su temperatura nominal y cada vez peor al alejarse.

SI ALGO NO SALE

Síntoma	Causa habitual
La lectura baila mucho y marca de menos	Falta el condensador de 100 nF. Es el caso del recuadro.
«sensor desconectado»	Una de las dos patas no hace contacto, o la resistencia fija no está.
Marca una temperatura constante y absurda	La resistencia fija no es de 100 kΩ, o el termistor no es el de 100 kΩ.
Sube cuando debería bajar	El termistor y la resistencia fija están intercambiados de lugar.

El mismo algoritmo en...

En el área eléctrica y electrónica, el termistor es el sensor de temperatura de bobinado de un motor y el de los equipos de refrigeración. La protección térmica de un motor es, en el fondo, este laboratorio con un umbral y un relé.

En el área mecánica, es el sensor de temperatura de refrigerante y el de aire de admisión: los dos son termistores NTC leídos exactamente así. La diferencia es que en el vehículo la resistencia fija está dentro de la computadora y el termistor cuelga del motor.

3.5 Calibración de dos puntos

OBJETIVO

Corregir la diferencia entre lo que el sensor dice y lo que realmente hay, sin cambiar de sensor ni de fórmula.

RAZONAMIENTO

Los cuatro laboratorios anteriores usan las constantes del fabricante, y esas constantes casi nunca coinciden con el componente que uno tiene en la mano. Las resistencias tienen tolerancia del 5 %, los cables tienen

caída, el propio sensor tiene desvío y el conversor del ESP32 tiene un error conocido de varias decenas de milivoltios.

La calibración de dos puntos resuelve todo eso junto y sin teoría: se mide en dos condiciones conocidas, se anota qué dijo el instrumento en cada una, y se traza la recta que las une. Esa recta corrige de una vez todos los desvíos anteriores, sin importar de dónde vengan.

La recta es $y = m \cdot x + b$, y de dos puntos salen los dos coeficientes:

```
pendiente = (real2 - real1) / (crudo2 - crudo1)
ordenada = real1 - pendiente * crudo1
```

Los dos puntos deben estar separados

Cuanto más juntos estén, más se amplifica cualquier error al calcular la pendiente. Calibrar con dos puntos que difieren en dos grados es peor que no calibrar.

Para temperatura, los dos puntos clásicos son agua con hielo (0 °C) y agua hirviendo (100 °C, algo menos en altura). En La Paz el agua hierve cerca de 88 °C, de modo que ese valor hay que corregirlo o usar otro punto conocido.

FILTRAR ANTES DE CALIBRAR

Calibrar sobre una lectura que baila no sirve. Hay dos maneras de estabilizarla y no dan lo mismo.

Filtro	Qué hace	Cuándo conviene
Promedio	Suma varias lecturas y divide.	Ruido parejo, el caso normal. Es el que usan los laboratorios anteriores.
Mediana	Ordena las lecturas y toma la del medio.	Cuando hay picos aislados: motores arrancando, contactores, soldadura cerca. Un solo valor disparatado no la mueve; al promedio sí.

PROGRAMA

Archivo `lab_3_5_calibracion.py`

```
from machine import Pin, ADC
import time

PIN_ENTRADA = 34

entrada = ADC(Pin(PIN_ENTRADA))
entrada.atten(ADC.ATTN_11DB)
entrada.width(ADC.WIDTH_12BIT)

# ----- filtrado
def leer_promedio(muestras=32, espera_ms=2):
    """Promedia varias lecturas. Atenua el ruido de fondo."""
    suma = 0
    for _ in range(muestras):
        suma = suma + entrada.read()
        time.sleep_ms(espera_ms)
    return suma / muestras

def leer_mediana(muestras=15, espera_ms=2):
    """Valor central de varias lecturas ordenadas.

    A diferencia del promedio, no se deja arrastrar por una lectura
    disparatada. Conviene cuando hay motores o contactores cerca.
    """
    valores = []
    for _ in range(muestras):
        valores.append(entrada.read())
        time.sleep_ms(espera_ms)
    valores.sort()
    return valores[len(valores) // 2]

# ----- calibracion
def recta_de_calibracion(crudo1, real1, crudo2, real2):
    """Pendiente y ordenada de la recta que une los dos puntos."""
    if crudo2 == crudo1:
        raise ValueError("los dos puntos dan la misma lectura")
    pendiente = (real2 - real1) / (crudo2 - crudo1)
    ordenada = real1 - pendiente * crudo1
    return pendiente, ordenada
```

```

def aplicar(crudo, pendiente, ordenada):
    return pendiente * crudo + ordenada

def calibrar(unidad="C"):
    """Guía la toma de los dos puntos y devuelve la recta."""
    print("Punto 1: lleve el sensor a la primera condicion conocida.")
    input("    Pulse Enter cuando este estable... ")
    crudo1 = leer_promedio()
    real1 = float(input("    Valor real, en {}: ".format(unidad)))

    print("Punto 2: lleve el sensor a la segunda condicion.")
    input("    Pulse Enter cuando este estable... ")
    crudo2 = leer_promedio()
    real2 = float(input("    Valor real, en {}: ".format(unidad)))

    pendiente, ordenada = recta_de_calibracion(
        crudo1, real1, crudo2, real2)
    print("")
    print("    pendiente = {:.6f}".format(pendiente))
    print("    ordenada   = {:.4f}".format(ordenada))
    print("    Copie estas dos cifras al programa definitivo.")
    return pendiente, ordenada

def main():
    pendiente, ordenada = calibrar()
    print("")
    print(" crudo      calibrado")

    while True:
        crudo = leer_promedio()
        print("    {:6.0f}    {:7.2f}".format(
            crudo, aplicar(crudo, pendiente, ordenada)))
        time.sleep(1)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

El programa pide las dos condiciones, calcula la recta y muestra las dos columnas en paralelo. Las dos cifras que imprime —pendiente y ordenada— son el resultado del laboratorio: se copian como constantes al

programa definitivo y no hace falta volver a calibrar hasta que se cambie el sensor o el cableado.

Una advertencia sobre la que conviene insistir: la calibración vale para *ese* sensor con *ese* cableado. Cambiar el cable por uno más largo, o el sensor por otro igual, obliga a repetirla.

El mismo algoritmo en...

En el área eléctrica y electrónica, esto es literalmente lo que se hace al poner en marcha un lazo de instrumentación: se ajusta el cero y el span del transmisor contra un patrón. La pendiente es el span y la ordenada es el cero, y el procedimiento de la norma es este mismo.

En el área mecánica, es el aprendizaje de posición que hace la computadora con el sensor de mariposa: se le enseña qué lectura corresponde a cerrado y cuál a pleno, y de ahí en más interpola.

EJERCICIOS PROPUESTOS

1. En el laboratorio 3.1, calcule cuántos milivoltios representa una sola cuenta con cada una de las cuatro atenuaciones.
2. Calcule R_1 y R_2 para leer una señal de 0 a 10 V dejando como máximo 3,1 V en el pin, con resistencias de valores comerciales y sin pasarse de 25 k Ω en total.
3. El LM35 del laboratorio 3.3 y el termistor del 3.4, puestos juntos, ¿marcan lo mismo? Anote la diferencia y explique de dónde puede venir.
4. Modifique el laboratorio 3.4 para un termistor de 10 k Ω . ¿Qué cuatro cosas hay que cambiar? ¿Sigue haciendo falta el condensador?
5. Aplique la calibración del 3.5 al LM35 usando agua con hielo y agua caliente medida con termómetro. ¿Cuánto vale la ordenada? ¿Qué significaría que valiera exactamente cero?
6. Con el potenciómetro quieto, tome cien lecturas y anote la mayor y la menor. ¿Cuántas cuentas de ruido hay? ¿Cuántos grados serían si fuera el termistor del 3.4 a temperatura ambiente?
7. Explique por qué la mediana de quince lecturas resiste un pico y el promedio de treinta y dos no. ¿En qué caso preferiría igual el promedio?

Señales de pulso

Hay una familia de sensores que no entrega ni un sí o un no ni una tensión proporcional: entrega **pulsos**, y la información está en cuántos llegan o cada cuánto llegan. Un caudalímetro da un pulso por cada tanto volumen que pasa; un encoder, uno por cada fracción de vuelta; un sensor de rueda fónica, uno por cada diente.

La señal es digital —cada pulso es un cambio de nivel como los del capítulo 2— pero el problema es nuevo, y es de tiempo. Un pulso que se pierde no se recupera, y el programa tiene que estar mirando en el instante justo. Los cuatro laboratorios de este capítulo tratan ese problema, y los tres primeros no necesitan ningún sensor: la placa se genera los pulsos a sí misma, de modo que se sabe de antemano cuántos debería haber contado y se puede medir exactamente cuántos se perdió.

4.1 Contar sin perder: consulta contra interrupción

OBJETIVO

Medir, con números, la diferencia entre revisar un pin cada tanto y hacer que el pin avise.

MATERIALES

Cantidad	Componente
1	Un cable de conexión. Nada más.

EL CIRCUITO

no hace falta ningún sensor: la placa genera los pulsos y los cuenta

GPIO25 los produce, GPIO26 los recibe

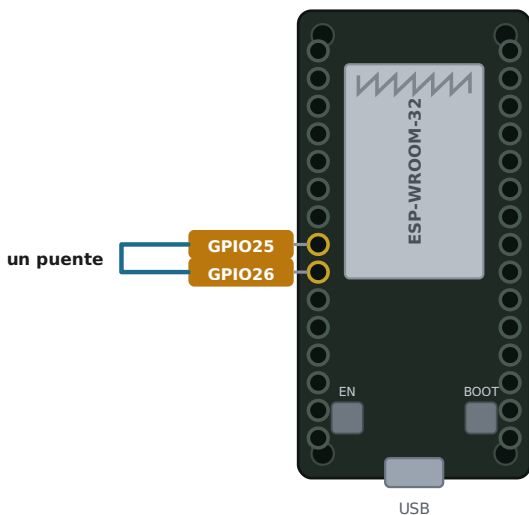


Figura 4.1 — El circuito más corto del libro. GPIO25 genera una señal cuadrada y GPIO26 la recibe; como la frecuencia es conocida, también se conoce cuántos pulsos deberían contarse.

RAZONAMIENTO

Hay dos maneras de enterarse de que llegó un pulso.

La **consulta** es la evidente: en cada vuelta del programa se lee el pin y se compara con la lectura anterior. Funciona, y es lo que se hizo en el laboratorio 2.3 con el pulsador. Pero entre dos lecturas el programa está haciendo otra cosa —calcular, escribir en la consola, refrescar una pantalla— y si en ese rato la señal sube y vuelve a bajar, ese pulso **no lo contó nadie**. No hay error, no hay aviso: simplemente falta.

La **interrupción** invierte la relación. Se le dice al microcontrolador que cuando ese pin baje, ejecute una función determinada, y él la ejecuta *interrumpiendo* lo que estuviera haciendo. No hay manera de perderse un pulso, porque el programa ya no tiene que estar mirando.

Para un pulsador que se aprieta con el dedo la diferencia no se nota. Para un caudalímetro a doscientos pulsos por segundo, la consulta pierde la mayoría.

```
entrada.irq(trigger=Pin.IRQ_FALLING, handler=al_llegar_un_pulso)
```

Disparador	Cuándo se ejecuta
<code>Pin.IRQ_FALLING</code>	Cuando la señal pasa de alto a bajo.
<code>Pin.IRQ_RISING</code>	Cuando pasa de bajo a alto.
<code>Pin.IRQ_RISING</code> <code>Pin.IRQ_FALLING</code>	En los dos flancos. Cuenta el doble de veces.

Lo que se hace dentro de un manejador, y lo que no

Mientras el manejador corre, el programa principal está detenido y ninguna otra interrupción se atiende. Por eso debe ser **lo más corto posible**: sumar uno, anotar un instante, cambiar una bandera. Nada más.

Dentro de un manejador **no** van: `print`, esperas, cálculos largos, ni nada que reserve memoria —una cadena con `format`, por ejemplo—. Reservar memoria dentro de una interrupción provoca un error que aparece minutos u horas después y es muy difícil de rastrear. Todo eso se hace en el programa principal, leyendo lo que el manejador dejó anotado.

LEER EL CONTADOR SIN PERDER CUENTAS

Hay un detalle fino. Si el programa principal hace `pulsos = 0` justo después de haber leído el valor, y entre esas dos instrucciones llega un pulso, ese pulso se pierde: el manejador lo sumó al valor viejo y el programa lo acaba de borrar.

La solución es apagar las interrupciones durante las dos instrucciones, que juntas tardan microsegundos:

```
def tomar_pulsos():
    """Lee el contador y lo pone en cero sin perder nada.

    Entre leer y poner en cero podría llegar un pulso, y se
```

```

perderia. Apagar las interrupciones durante esas dos
instrucciones lo impide; duran microsegundos.
"""

global pulsos
estado = machine.disable_irq()
cuenta = pulsos
pulsos = 0
machine.enable_irq(estado)
return cuenta

```

Por qué se guarda el estado y no se enciende y ya

`disable_irq()` devuelve cómo estaban las interrupciones antes, y `enable_irq()` las deja como estaban. Si en lugar de eso se encendieran sin más, una función llamada desde dentro de otra que las había apagado las encendería a destiempo. Es una costumbre que cuesta una variable y evita un problema difícil.

PROGRAMA

Archivo `lab_4_1_consulta_interrupcion.py`

```

from machine import Pin, PWM
import machine
import time

PIN_GENERADOR = 25
PIN_CONTADOR = 26

FRECUENCIA = 200          # pulsos por segundo que se generan
DURACION = 3              # segundos que dura cada medicion
TRABAJO_MS = 5            # lo que el programa tarda en cada vuelta

generador = PWM(Pin(PIN_GENERADOR), freq=FRECUENCIA)
generador.duty(512)        # 50 %: mitad alto, mitad bajo

entrada = Pin(PIN_CONTADOR, Pin.IN)

pulsos = 0

def al_llegar_un_pulso(pin):
    """Se ejecuta sola, apenas baja la senal. Solo suma uno."""
    global pulsos
    pulsos = pulsos + 1

```

```

def tomar_pulsos():
    global pulsos
    estado = machine.disable_irq()
    cuenta = pulsos
    pulsos = 0
    machine.enable_irq(estado)
    return cuenta

def contar_por_consulta(segundos):
    """Revisa el pin una y otra vez, haciendo otra cosa entre medio.

    Ese "otra cosa" es lo que ocurre en cualquier programa real. Si
    en ese rato la señal sube y vuelve a bajar, ese pulso no lo
    cuenta nadie.
    """
    cuenta = 0
    anterior = entrada.value()
    limite = time.ticks_add(time.ticks_ms(), segundos * 1000)

    while time.ticks_diff(limite, time.ticks_ms()) > 0:
        actual = entrada.value()
        if anterior == 1 and actual == 0:
            cuenta = cuenta + 1
        anterior = actual
        time.sleep_ms(TRABAJO_MS)      # el programa hace otra cosa

    return cuenta

def contar_por_interrupcion(segundos):
    """Deja que el pin avise, perdiendo el mismo tiempo que antes."""
    entrada.irq(trigger=Pin.IRQ_FALLING, handler=al_llegar_un_pulso)
    tomar_pulsos()                    # descarta lo acumulado

    limite = time.ticks_add(time.ticks_ms(), segundos * 1000)
    while time.ticks_diff(limite, time.ticks_ms()) > 0:
        time.sleep_ms(TRABAJO_MS)      # exactamente el mismo trabajo

    entrada.irq(handler=None)
    return tomar_pulsos()

def main():
    esperados = FRECUENCIA * DURACION
    print("Se generan {} pulsos por segundo durante {} s.".format(

```

```

        FRECUENCIA, DURACION))
    print("Deberian contarse {} pulsos.".format(esperados))
    print("")

    por_consulta = contar_por_consulta(DURACION)
    por_interrupcion = contar_por_interrupcion(DURACION)

    print(" metodo          contados   perdidos")
    for nombre, cuenta in (("consulta", por_consulta),
                           ("interrupcion", por_interrupcion)):
        perdidos = esperados - cuenta
        print("  {:<14}  {:6d}   {:5d}   ({:4.1f} %)".format(
            nombre, cuenta, perdidos, perdidos * 100.0 / esperados))

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Se generan 200 pulsos por segundo durante 3 s.
Deberian contarse 600 pulsos.

metodo	contados	perdidos	
consulta	120	480	(80.0 %)
interrupcion	600	0	(0.0 %)

Las cifras exactas dependen de la placa, pero el orden de magnitud es ése: con el programa ocupado cinco milisegundos por vuelta y pulsos cada cinco milisegundos, la consulta pierde la mayoría y la interrupción no pierde ninguno.

Conviene repetir la prueba bajando **FRECUENCIA** a 10 y subiéndola a 1000. A diez pulsos por segundo los dos métodos aciertan; a mil, la consulta es inservible. **Ese es el criterio para elegir**, y el ejercicio 1 pide encontrar el punto donde deja de servir.

El mismo algoritmo en...

En el área eléctrica y electrónica, esto decide si un contador de producción es confiable. Una fotocélula que cuenta piezas en una cinta rápida, leída por consulta, cuenta de menos y nadie se entera hasta que los números no cierran con el inventario.

En el área mecánica, la señal del sensor de cigüeñal a 6000 rpm con una rueda de 60 dientes da 6000 pulsos por segundo. Ahí la consulta no es una alternativa peor: es imposible.

4.2 Frecuencímetro por conteo

OBJETIVO

Medir la frecuencia contando cuántos pulsos llegan en un tiempo fijo, y encontrar el límite del método.

EL CIRCUITO

El mismo del laboratorio 4.1: un puente entre GPIO25 y GPIO26.

RAZONAMIENTO

Si se cuentan los pulsos que llegan en exactamente un segundo, ese número **es** la frecuencia en hercios. No hay cálculo. Es el método que usan casi todos los tacómetros y caudalímetros, y su virtud es que no se puede equivocar en la aritmética.

Su defecto aparece abajo. La cuenta es siempre un número entero: o llegaron nueve pulsos o llegaron diez. A 1000 Hz, equivocarse en uno es un error del 0,1 %; a 10 Hz, el mismo pulso de diferencia es un error del **10 %**. El método es exacto arriba y grosero abajo, y el laboratorio siguiente resuelve justamente eso.

Frecuencia	Pulsos en 1 s	Error de ± 1 pulso
10 Hz	10	$\pm 10 \%$
50 Hz	50	$\pm 2 \%$
200 Hz	200	$\pm 0,5 \%$
1000 Hz	1000	$\pm 0,1 \%$
5000 Hz	5000	$\pm 0,02 \%$

La ventana y la resolución van de la mano

Con una ventana de un segundo, la resolución es de 1 Hz. Con media ventana, la medición llega el doble de rápido pero la resolución cae a 2 Hz. Con dos segundos, mejora a 0,5 Hz pero hay que esperar el doble.

No hay manera de tener las dos cosas con este método: es la limitación de fondo, y por eso existe el otro.

PROGRAMA

Archivo `lab_4_2_frecuencimetro.py`

```
from machine import Pin, PWM
import machine
import time

PIN_GENERADOR = 25
PIN_CONTADOR = 26

VENTANA_MS = 1000          # duracion de cada medicion
FRECUENCIAS = (10, 50, 200, 1000, 5000)

generador = PWM(Pin(PIN_GENERADOR), freq=FRECUENCIAS[0])
generador.duty(512)

entrada = Pin(PIN_CONTADOR, Pin.IN)

pulsos = 0

def contar(pin):
    global pulsos
```

```

    pulsos = pulsos + 1

def tomar_pulsos():
    global pulsos
    estado = machine.disable_irq()
    cuenta = pulsos
    pulsos = 0
    machine.enable_irq(estado)
    return cuenta

def medir_frecuencia(ventana_ms=VENTANA_MS):
    """Cuenta los pulsos de una ventana y los pasa a hercios."""
    tomar_pulsos()          # arranca de cero
    time.sleep_ms(ventana_ms)
    return tomar_pulsos() * 1000.0 / ventana_ms

def main():
    entrada.irq(trigger=Pin.IRQ_FALLING, handler=contar)
    print("Frecuencimetro por conteo, ventana de {} ms".format(
        VENTANA_MS))
    print("")
    print(" generada   medida   error")

    for f in FRECUENCIAS:
        generador.freq(f)
        time.sleep_ms(200)          # deja que se asiente
        medida = medir_frecuencia()
        error = (medida - f) * 100.0 / f
        print("  {:6d} Hz  {:7.1f} Hz  {:+5.1f} %".format(
            f, medida, error))

    resolucion = 1000.0 / VENTANA_MS
    print("")
    print("Con {} ms de ventana no se baja de {:.1f} Hz.".format(
        VENTANA_MS, resolucion))
    print("Por eso el error relativo es grande abajo y chico arriba.")

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Las cinco frecuencias se miden con error decreciente. La columna de error es la lección del laboratorio: en 10 Hz ronda el 10 % y en 5000 Hz es despreciable. Vale la pena repetirlo con `VENTANA_MS = 100` y ver que todos los errores se multiplican por diez.

4.3 Medición por período

OBJETIVO

Medir bien las frecuencias bajas, que es donde el conteo falla.

EL CIRCUITO

El mismo puente entre GPIO25 y GPIO26.

RAZONAMIENTO

En vez de contar cuántos pulsos entran en un tiempo fijo, se mide **cuánto tarda un pulso** y se invierte: si entre dos flancos pasaron 100 000 microsegundos, la frecuencia es 10 Hz.

La ventaja es inmediata. El reloj de microsegundos del ESP32 tiene mucha más resolución que el conteo de pulsos enteros, de modo que a frecuencias bajas la medición es muy precisa. Y no hay que esperar una ventana completa: con un solo período ya hay resultado.

La desventaja es la simétrica. A frecuencias altas los períodos son tan cortos que el propio tiempo de atención de la interrupción —unos pocos microsegundos— pesa sobre la medida, y el resultado empieza a bailar. Por eso se promedian varios períodos.

Método	Va bien en	Va mal en
Conteo (4.2)	frecuencias altas	frecuencias bajas: ± 1 pulso pesa mucho
Período (4.3)	frecuencias bajas	frecuencias altas: el período se acerca al tiempo de atención

Un instrumento serio usa los dos y cambia de método según el rango. El ejercicio 4 propone escribirlo.

ticks_diff, siempre

El contador de microsegundos no crece indefinidamente: llega a un máximo y vuelve a cero. Restando dos lecturas con el signo menos, si el vuelco cae entre ambas, el resultado es un número negativo enorme y la frecuencia sale disparatada.

Con `ticks_us()` el vuelco ocurre aproximadamente cada 18 minutos. Un instrumento que anda bien y de golpe marca un valor absurdo, más o menos cada cuarto de hora, casi siempre es esto.

PROGRAMA

Archivo `lab_4_3_periodo.py`

```
from machine import Pin, PWM
import time

PIN_GENERADOR = 25
PIN_CONTADOR = 26

MUESTRAS = 16          # periodos que se promedian
FRECUENCIAS = (2, 10, 50, 200, 1000)

generador = PWM(Pin(PIN_GENERADOR), freq=FRECUENCIAS[0])
generador.duty(512)

entrada = Pin(PIN_CONTADOR, Pin.IN)

ultimo_us = 0
periodo_us = 0

def al_llegar_un_pulso(pin):
    """Anota cuanto paso desde el pulso anterior.

    Se usa ticks_diff y no una resta comun porque el contador de
    microsegundos se da vuelta: una resta directa daría, en ese
    momento, un valor negativo enorme.
    """
    global ultimo_us, periodo_us
    ahora = time.ticks_us()
```

```

    if ultimo_us != 0:
        periodo_us = time.time_diff(ahora, ultimo_us)
        ultimo_us = ahora

def medir_frecuencia(muestras=MUESTRAS, espera_ms=600):
    """Promedia varios periodos. None si no llego ningun pulso."""
    global periodo_us
    suma = 0
    tomadas = 0
    limite = time.time_add(time.time_ms(), espera_ms)

    while tomadas < muestras and \
        time.time_diff(limite, time.time_ms()) > 0:
        if periodo_us > 0:
            suma = suma + periodo_us
            periodo_us = 0
            tomadas = tomadas + 1
            time.sleep_ms(1)

    if tomadas == 0:
        return None
    return 1000000.0 / (suma / tomadas)

def main():
    entrada.irq(trigger=Pin.IRQ_FALLING, handler=al_llegar_un_pulso)
    print("Medicion por periodo, promediando {} pulsos".format(
        MUESTRAS))
    print("")
    print(" generada   medida   error")

    for f in FRECUENCIAS:
        generador.freq(f)
        time.sleep_ms(300)
        medida = medir_frecuencia()

        if medida is None:
            print("  {:6d} Hz   sin pulsos".format(f))
        else:
            error = (medida - f) * 100.0 / f
            print("  {:6d} Hz   {:7.1f} Hz   {:+5.1f} %".format(
                f, medida, error))

    print("")
    print("Este metodo mide bien abajo, donde el conteo se equivocaba.")

```

```
if __name__ == "__main__":  
    main()
```

QUÉ DEBE OBSERVARSE

A 2 Hz y a 10 Hz el error es de décimas de por ciento, contra el 10 % que daba el conteo. A 1000 Hz empieza a notarse ruido. Comparar las dos tablas —la del 4.2 y la de éste— es el ejercicio más útil del capítulo: cada método es bueno exactamente donde el otro falla.

4.4 Tacómetro: del pulso a las revoluciones

OBJETIVO

Medir las revoluciones de un motor real, con todo lo que eso trae que no aparecía con la señal limpia del generador.

MATERIALES

Cantidad	Componente	Observación
1	Sensor óptico de ranura	con su comparador, el del disco
1	Disco con marcas	el de este laboratorio tiene 4
1	Módulo L298N	punto H, para gobernar el motor
1	Motor de corriente continua	con su fuente

EL CIRCUITO

la masa del puente H y la del ESP32 deben estar unidas

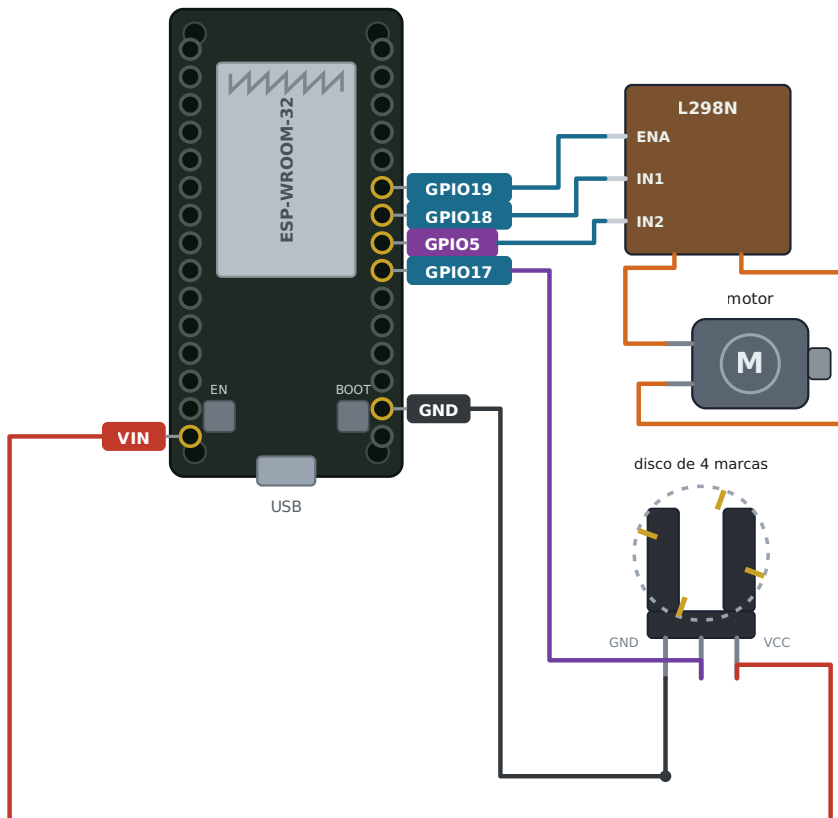


Figura 4.2 — El sensor óptico ve pasar las marcas del disco montado en el eje. El L298N gobierna el motor y el ESP32 le manda el PWM por ENA.

Las dos masas tienen que estar unidas

El motor se alimenta de su propia fuente por los bornes del L298N, y el ESP32 del USB. Son dos fuentes distintas, y si sus masas no se unen, las señales de mando no tienen referencia común: el puente H hace cualquier cosa o no hace nada.

Unir **sólo las masas**. No unir los positivos.

RAZONAMIENTO

La cuenta es directa. Si el disco tiene m marcas, cada vuelta produce m pulsos, de modo que:

$$\text{RPM} = (\text{pulsos} / \text{marcas_por_vuelta}) * 60000 / \text{ventana_ms}$$

Con cuatro marcas y una ventana de un segundo, 40 pulsos son 600 RPM. Si el disco tuviera una sola marca serían 2400. **Equivocarse en el número de marcas multiplica o divide el resultado**, y es el error más común de este laboratorio: el programa no tiene manera de darse cuenta.

Lo que aparece aquí y no aparecía antes es que la señal ya no es limpia. El comparador del módulo óptico no conmuta de una vez: en cada flanco puede dar varios cambios en pocos microsegundos. Sin filtro, las revoluciones salen infladas, a veces al doble.

El filtro es el mismo del pulsador del laboratorio 2.3, con dos diferencias. Se mide en **microsegundos**, porque aquí los pulsos buenos están mucho más cerca; y el umbral hay que elegirlo con cuidado, porque si se pasa empieza a descartar pulsos legítimos a alta velocidad.

Cómo elegir el tiempo de rebote

Tiene que ser mayor que el rebote y menor que el intervalo entre dos pulsos buenos a la máxima velocidad prevista. Con 4 marcas a 3000 RPM hay 200 pulsos por segundo, o sea uno cada 5000 μs ; un filtro de 1500 μs deja margen de sobra. Si el motor llegara a 12 000 RPM el intervalo bajaría a 1250 μs y ese filtro empezaría a comerse pulsos: el tacómetro marcaría de menos justamente a fondo.

PROGRAMA

Archivo `lab_4_4_tacometro.py`

```
from machine import Pin, PWM
import machine
import time

PIN_SENSOR = 17
MARCAS_POR_VUELTA = 4      # las que tenga SU disco. Ver el razonamiento.
```

```

ENA = PWM(Pin(19), freq=1000)
IN1 = Pin(18, Pin.OUT)
IN2 = Pin(5, Pin.OUT)
IN1.on()
IN2.off()                # sentido de giro fijo
ENA.duty(0)

sensor = Pin(PIN_SENSOR, Pin.IN)

VENTANA_MS = 1000
REBOTE_US = 1500         # descarta los rebotes del comparador

pulsos = 0
_ultimo_us = 0

def contar(pin):
    """Suma un pulso, descartando los rebotes del comparador."""
    global pulsos, _ultimo_us
    ahora = time.ticks_us()
    if time.ticks_diff(ahora, _ultimo_us) > REBOTE_US:
        pulsos = pulsos + 1
        _ultimo_us = ahora

def tomar_pulsos():
    global pulsos
    estado = machine.disable_irq()
    cuenta = pulsos
    pulsos = 0
    machine.enable_irq(estado)
    return cuenta

def rpm_desde_pulsos(cuenta, ventana_ms):
    """Pasa una cuenta de pulsos a revoluciones por minuto."""
    vueltas = cuenta / MARCAS_POR_VUELTA
    return vueltas * 60000.0 / ventana_ms

def main():
    sensor.irq(trigger=Pin.IRQ_FALLING, handler=contar)
    print("Tacometro. Disco de {} marcas.".format(MARCAS_POR_VUELTA))
    print("")
    print("  velocidad    pulsos      RPM")

    for porcentaje in (40, 60, 80, 100, 0):

```

```

    ENA.duty(int(porcentaje * 1023 / 100))
    time.sleep_ms(600)           # deja que el motor se acomode
    tomar_pulsos()               # descarta lo del transitorio

    time.sleep_ms(VENTANA_MS)
    cuenta = tomar_pulsos()

    print("    {:3d} %        {:4d}    {:7.0f}".format(
        porcentaje, cuenta, rpm_desde_pulsos(cuenta, VENTANA_MS)))

    ENA.duty(0)
    sensor.irq(handler=None)
    print("")
    print("Motor detenido.")

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Las revoluciones crecen con el porcentaje, pero **no en proporción**: por debajo de cierto valor el motor ni arranca, y arriba la curva se aplana. Un motor de corriente continua no es lineal contra el ciclo de trabajo, y conviene verlo antes de suponer lo contrario.

Conviene comprobar la cifra con un tacómetro óptico de mano, o marcando el eje y contando a ojo a baja velocidad. Si el resultado sale justo al doble o a la mitad, el número de marcas está mal.

SI ALGO NO SALE

Síntoma	Causa habitual
Las RPM salen al doble	El disco tiene el doble de marcas de las declaradas, o el disparador está en los dos flancos.
Salen infladas y erráticas	Falta el filtro de rebote, o REBOTE_US es demasiado corto.
Marca de menos a alta velocidad	REBOTE_US es demasiado largo y descarta pulsos buenos.
Cuenta con el motor detenido	Ruido eléctrico del motor acoplado al cable del sensor. Separarlo del cableado de potencia, y unir las masas en un solo punto.
El motor no arranca	Las masas no están unidas, o el ciclo de trabajo es menor que el mínimo que ese motor necesita para vencer su propio rozamiento.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el control de velocidad de una bomba o una cinta con realimentación: se mide la velocidad real y se corrige el mando hasta alcanzar la consigna. Con el encoder de un motor cambian las marcas por vuelta —suelen ser cientos— y nada más.

En el área mecánica, es el sensor de régimen del motor. La rueda fónica del cigüeñal tiene típicamente 60 dientes menos 2, y esos dos que faltan son los que marcan la posición. El algoritmo de conteo es éste; lo que se agrega es detectar el hueco.

EJERCICIOS PROPUESTOS

1. En el laboratorio 4.1, encuentre bajando la frecuencia el punto en que la consulta deja de perder pulsos. Relacione ese valor con `TRABAJO_MS`.
2. Cambie el disparador del 4.1 a los dos flancos. ¿Cuántos pulsos cuenta ahora? ¿Qué habría que cambiar para que la frecuencia siga saliendo bien?
3. Con la tabla del 4.2 y la del 4.3, dibuje el error de los dos métodos contra la frecuencia. ¿En qué valor se cruzan las dos curvas?
4. Escriba una función que elija sola el método: período por debajo de la frecuencia de cruce y conteo por encima.
5. Un caudalímetro da 450 pulsos por litro. Escriba el programa que muestre el caudal en litros por minuto y el total acumulado.
6. En el 4.4, calcule qué valor de `REBOTE_US` empezaría a comerse pulsos si el motor llegara a 9000 RPM con el disco de 4 marcas.
7. El tacómetro con ventana de un segundo tarda un segundo en enterarse de un cambio. ¿Cómo lo haría más rápido sin perder resolución? Ayuda: mire el laboratorio 4.3.

Salidas moduladas

Un pin digital sólo sabe hacer dos cosas: poner 3,3 V o poner 0 V. No hay manera de pedirle 1,7 V. Y sin embargo, con ese único recurso se regula el brillo de una lámpara, la velocidad de un motor, la posición de un servomotor y la nota de un zumbador.

El truco consiste en conmutar muy rápido y repartir el tiempo. Si el pin está en alto la cuarta parte de cada ciclo, el valor medio es la cuarta parte de 3,3 V, y una lámpara —que no alcanza a apagarse entre ciclo y ciclo— se comporta como si recibiera esa fracción. A eso se le llama **modulación por ancho de pulso**, y en los programas aparece abreviada como PWM, de *pulse width modulation*.

Este capítulo es el reverso del capítulo 3. Allí el problema era traer hacia adentro una magnitud continua; aquí es sacarla hacia afuera. Y el laboratorio que los une —un potenciómetro que gobierna el brillo de un LED— es el primero de los cinco.

5.1 Qué se controla y qué no

Una salida modulada tiene **dos** parámetros, y conviene no confundirlos porque cada aplicación gobierna uno distinto.

Parámetro	Qué es	Se fija con
Frecuencia	Cuántos ciclos por segundo. No cambia la intensidad: cambia sólo cuán rápido se conmuta.	<code>PWM(pin, freq=1000)</code> o <code>.freq(1000)</code>
Ciclo de trabajo	Qué proporción de cada ciclo pasa en alto. Esto sí es la intensidad.	<code>.duty(0...1023)</code> o <code>.duty_u16(0...65535)</code>

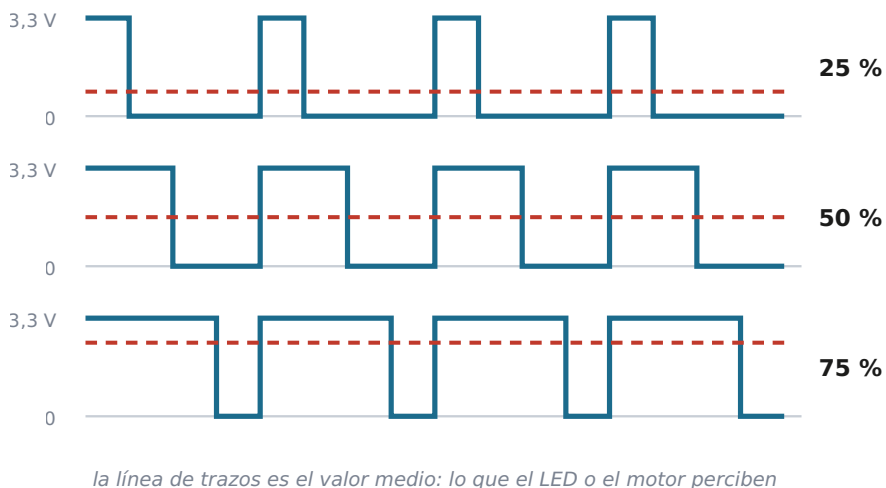


Figura 5.1 — Tres ciclos de trabajo sobre la misma frecuencia. La onda no cambia de altura ni de ritmo: lo único que cambia es cuánto dura la parte alta, y con ella el valor medio.

Dos escalas para lo mismo

MicroPython ofrece dos maneras de fijar el ciclo de trabajo y no son intercambiables. `duty()` trabaja sobre **1023** y `duty_u16()` sobre **65535**. Escribir `duty(32767)` pensando en la mitad no da la mitad: da el máximo, porque cualquier valor por encima de 1023 se satura.

Este libro usa `duty()` donde el valor viene del conversor analógico —que también entrega una escala corta— y `duty_u16()` donde el valor se escribe a mano. Cada laboratorio dice cuál, y mezclarlas es la causa más común de que un programa de PWM «no haga nada» o «esté siempre al máximo».

QUÉ FRECUENCIA CONVIENE

Aplicación	Frecuencia	Por qué
LED, iluminación	500 – 5000 Hz	Por debajo de unos 100 Hz el parpadeo se nota, sobre todo al mover la vista.
Motor de corriente continua	1 – 20 kHz	Por debajo de 1 kHz el motor chilla: la propia bobina reproduce la frecuencia de conmutación.
Servomotor	50 Hz, fija	No es una elección: el servo espera un pulso cada 20 ms y mide su ancho. Ver el laboratorio 5.3.
Zumbador pasivo	la nota que se quiera	Aquí la frecuencia es la información: 440 Hz es un la.

El caso del servomotor es distinto

En los tres primeros casos lo que importa es el ciclo de trabajo y la frecuencia es un detalle de implementación. En el servomotor ocurre al revés: la frecuencia está fijada por la norma y lo que el servo mide es el **ancho del pulso en milisegundos**. El ciclo de trabajo es apenas la manera que tiene MicroPython de expresar ese ancho. Tenerlo claro evita el error del laboratorio 5.4.

5.2 Brillo de un LED gobernado por un potenciómetro

OBJETIVO

Unir la entrada analógica del capítulo 3 con la salida modulada de éste: leer la posición de una perilla y convertirla en brillo.

MATERIALES

Cantidad	Componente
1	Potenciómetro de 10 kΩ
1	LED de 5 mm y resistencia de 330 Ω
—	Cables de conexión

EL CIRCUITO

GPIO34 pertenece al ADC1 y está en la otra columna: el cable rodea la placa

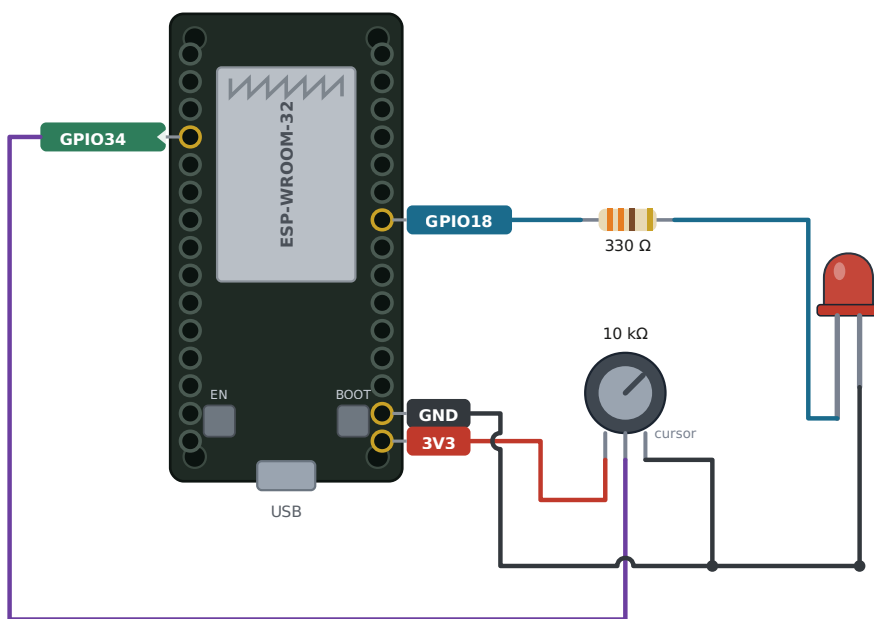


Figura 5.2 — El potenciómetro va a GPIO34, que pertenece al ADC1 y está en la columna opuesta: su cable rodea la placa. El LED cuelga de GPIO18, que sí admite PWM.

Por qué GPIO34 y no cualquier otro

El cursor del potenciómetro tiene que ir a una pata del **ADC1**. En este laboratorio todavía no hay WiFi y el ADC2 funcionaría, pero en el capítulo 7 este mismo montaje va a publicar su lectura en internet, y entonces el ADC2 dejaría de medir. Cablearlo bien desde ahora ahorra rehacerlo después.

Obsérvese además que GPIO34 es uno de los cuatro pines que **sólo pueden ser entrada**. Para un potenciómetro es perfecto; para el LED no serviría.

RAZONAMIENTO

El conversor entrega un número de 0 a 4095, porque tiene 12 bits. El PWM acepta de 0 a 1023, porque `duty()` trabaja con 10 bits. Hay que llevar una escala a la otra, y como 4096 es exactamente cuatro veces 1024, basta con dividir entre cuatro.

Dividir entre cuatro es desplazar dos lugares en binario, y eso es lo que hace `cuentas >> 2`. Podría escribirse `cuentas // 4` y el resultado sería idéntico; se usa el desplazamiento porque es la forma habitual en programación de microcontroladores y conviene reconocerla al leer código ajeno.

No hay que redondear ni ajustar nada más: el extremo bajo del potenciómetro da 0, que apaga el LED, y el alto da 4095, que al desplazarse queda en 1023, el máximo.

PROGRAMA

Archivo `lab_5_2_brillo.py`

```
from machine import Pin, PWM, ADC
import time

led = PWM(Pin(18), freq=1000)
led.duty(0)

pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB)      # rango de entrada de 0 a 3,3 V
```

```

DUTY_MAXIMO = 1023          # duty() trabaja con 10 bits

while True:
    cuentas = pot.read()     # 0 a 4095, porque el ADC tiene 12 bits

    # Cuatro mil noventa y seis niveles no entran en mil veinticuatro.
    # Desplazar dos lugares a la derecha divide entre cuatro y ajusta
    # una escala a la otra sin perder el extremo: 4095 >> 2 da 1023.
    brillo = cuentas >> 2

    led.duty(brillo)
    print("Perilla: {:4d}    Brillo: {:4d}  ( {:.30f} %)".format(
        cuentas, brillo, brillo * 100 / DUTY_MAXIMO))
    time.sleep(0.1)

```

QUÉ DEBE OBSERVARSE

Al girar la perilla el LED pasa de apagado a pleno brillo de manera continua, y la consola muestra las dos escalas en paralelo. Vale la pena mirar los números: la columna de la izquierda avanza de cuatro en cuatro por cada unidad de la derecha.

El brillo no se percibe proporcional a la cifra. Entre el 0 y el 20 % el cambio se nota mucho; entre el 80 y el 100 %, casi nada. No es un defecto del montaje: el ojo responde de manera logarítmica, y es el motivo por el cual los reguladores de iluminación comerciales no usan una recta. El ejercicio 3 propone corregirlo.

SI ALGO NO SALE

Síntoma	Causa habitual
El LED está siempre encendido al máximo	Se usó <code>duty_u16()</code> con un valor pensado para <code>duty()</code> , o al revés.
El brillo no llega nunca al máximo	Falta <code>atten(ADC.ATTN_11DB)</code> : sin esa línea el conversor satura cerca de 1,1 V y nunca alcanza las 4095 cuentas.
El LED parpadea de manera visible	La frecuencia quedó por debajo de unos 100 Hz.
La lectura salta sola con la perilla quieta	El cursor no hace buen contacto, o el potenciómetro no tiene sus dos extremos conectados.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el regulador de intensidad de un circuito de iluminación, o la consigna de velocidad de un variador. El potenciómetro se reemplaza por la señal de 0 a 10 V de un mando remoto —acondicionada con el divisor del laboratorio 3.2— y el LED por la etapa de potencia.

En el área mecánica, es la atenuación de la iluminación del tablero, que en el vehículo se manda con una rueda dentada idéntica a este potenciómetro. El mismo programa, con otra etapa de salida.

5.3 El servomotor: la posición va en el ancho del pulso

OBJETIVO

Llevar un servomotor a un ángulo determinado y entender por qué aquí el ciclo de trabajo no significa «intensidad».

MATERIALES

Cantidad	Componente	Observación
1	Servomotor SG90	el de 9 g, con sus aspas
—	Cables de conexión	

EL CIRCUITO

el servo se alimenta de VIN: con 3V3 no tiene par y reinicia la placa

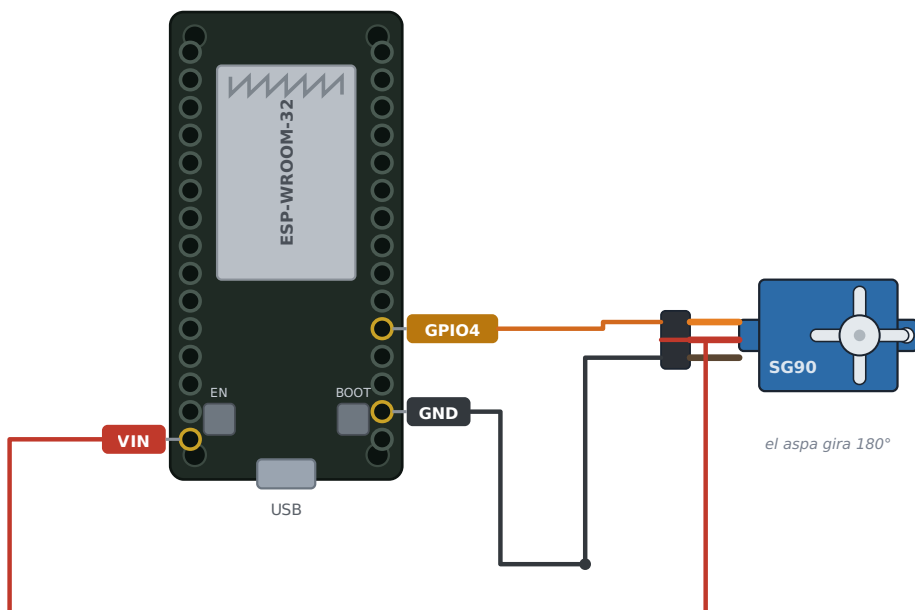


Figura 5.3 — Los tres cables del SG90 tienen colores normalizados: marrón a masa, rojo a la alimentación y naranja a la señal.

Alimentación del servo

El servo va a **VIN**, no a **3V3**. Con 3,3 V puede llegar a moverse en vacío, pero apenas encuentra resistencia consume un pico de corriente que el regulador de 3,3 V no sostiene, y la placa se reinicia sola. El síntoma es un servo que da un tirón y un ESP32 que vuelve a arrancar.

Si el servo mueve carga, conviene alimentarlo de una fuente aparte de 5 V y unir **solamente las masas** de las dos fuentes. Alimentarlo desde el USB del computador a través de la placa está bien para el laboratorio y no para nada más.

RAZONAMIENTO

El servomotor no interpreta el ciclo de trabajo como una cantidad. Espera un pulso cada 20 milisegundos —de ahí los 50 Hz, que no son negociables— y **mide cuánto dura ese pulso**. Un pulso de 1 ms lo manda a un extremo del recorrido; uno de 2 ms, al otro; uno de 1,5 ms, al centro. Entre medio interpola.

MicroPython no permite pedir «un pulso de 1,5 ms»: pide un ciclo de trabajo. Hay entonces que traducir, y la cuenta es directa. Con `duty()`, que va de 0 a 1023, y un período de 20 ms:

```
ancho_del_pulso = duty / 1023 * 20 ms
```

De donde salen los dos extremos que usa este libro:

Ciclo de trabajo	Ancho del pulso	Posición
40	0,78 ms	un extremo, cerca de 0°
77	1,51 ms	el centro, 90°
115	2,25 ms	el otro extremo, cerca de 180°
150	2,93 ms	fuera de rango: el tope mecánico

Los valores exactos varían de un servo a otro, y por eso están en constantes al principio del programa. La manera de ajustarlos es

empírica: bajar `DUTY_MINIMO` hasta que el servo deje de moverse, y subir `DUTY_MAXIMO` hasta lo mismo, retrocediendo después un par de unidades.

PROGRAMA

Archivo `lab_5_3_servo.py`

```
from machine import Pin, PWM
import time

servo = PWM(Pin(4), freq=50)      # 50 Hz: un ciclo cada 20 ms

DUTY_MINIMO = 40                  # unos 0,78 ms, extremo de 0 grados
DUTY_MAXIMO = 115                 # unos 2,25 ms, extremo de 180 grados
RECORRIDO = DUTY_MAXIMO - DUTY_MINIMO

def angulo_a_duty(angulo):
    """Convierte un angulo de 0 a 180 grados en ciclo de trabajo.

    Los dos extremos se recortan a proposito. Pedir 200 grados no
    consigue mas recorrido: manda al servo contra su tope, donde
    zumba, se calienta y termina rompiendo el engranaje de salida.
    """
    if angulo < 0:
        angulo = 0
    if angulo > 180:
        angulo = 180
    return int(angulo * RECORRIDO / 180 + DUTY_MINIMO)

for angulo in (0, 45, 90, 135, 180, 90):
    servo.duty(angulo_a_duty(angulo))
    print("{:3d} grados -> duty {:3d}".format(
        angulo, angulo_a_duty(angulo)))
    time.sleep(1)

servo.deinit()                    # suelta el pin y deja de mandar pulsos
```

Por qué termina con deinit()

Mientras el ESP32 siga mandando pulsos, el servo sigue sosteniendo la posición, y para sostenerla consume corriente y se calienta. `deinit()` apaga la salida y lo deja libre. En un montaje que sólo posiciona de vez en cuando —una compuerta, una válvula— esto es lo correcto; en uno que debe resistir una fuerza, no.

QUÉ DEBE OBSERVARSE

El aspa se detiene en cinco posiciones bien diferenciadas y vuelve al centro. Conviene marcar con lápiz la posición del aspa a 0° y a 180° y comprobar que el recorrido se acerca a media vuelta. Si queda notablemente corto, los valores de las dos constantes son conservadores para ese servo y pueden ampliarse.

SI ALGO NO SALE

Síntoma	Causa habitual
El servo zumba y se calienta sin moverse	Está contra el tope: el ciclo de trabajo se fue del rango. Es el error que corrige el laboratorio siguiente.
La placa se reinicia al mover el servo	Alimentación insuficiente. Es el caso del recuadro anterior.
No se mueve en absoluto	La frecuencia no es 50 Hz, o el cable naranja no llegó al pin.
Tiembla en reposo	Normal en los servos económicos. Se reduce con <code>deinit()</code> una vez alcanzada la posición.

5.4 Barrido continuo

OBJETIVO

Recorrer todo el rango de manera suave, y entender por qué el límite superior no puede elegirse al azar.

EL CIRCUITO

El mismo del laboratorio 5.3, sin cambios.

RAZONAMIENTO

Un barrido es la repetición del posicionamiento anterior sobre una sucesión de ángulos. Lo único que hay que decidir es el paso y la espera entre pasos: juntos determinan la velocidad aparente. Dos grados cada 30 milisegundos dan un recorrido completo en unos 2,7 segundos, que se ve fluido sin ser brusco.

Este laboratorio se incluye sobre todo por el error que contenía. La versión de la que parte llevaba el ciclo de trabajo hasta **150**, convencida de que así se aprovechaba todo el recorrido. Con la fórmula del laboratorio anterior, 150 equivale a un pulso de **2,93 ms**, muy por encima de los 2,4 ms que el SG90 admite. El servo no gira más: llega a su tope y empuja contra él en cada barrido.

El síntoma es engañoso, porque el servo *parece* funcionar. Zumba un poco en los extremos y se calienta, y a las pocas horas el engranaje de salida — que es de plástico— pierde dientes. Por eso el recorrido se recorta en el propio conversor de ángulos y no en el lazo: así ningún programa que use esa función puede pasarse, aunque se equivoque al llamarla.

PROGRAMA

Archivo `lab_5_4_barrido.py`

```
from machine import Pin, PWM
import time

servo = PWM(Pin(4), freq=50)

DUTY_MINIMO = 40
DUTY_MAXIMO = 115          # 2,25 ms. Subirlo a 150 da 2,93 ms:
RECORRIDO = DUTY_MAXIMO - DUTY_MINIMO    # el tope mecánico del servo

PASO = 2                    # grados por escalon
ESPERA_MS = 30              # cuanto se detiene en cada escalon

def angulo_a_duty(angulo):
    """Recorta el angulo ANTES de convertirlo, de modo que ningun
```

```

    programa que use esta funcion pueda mandar al servo al tope."""
    if angulo < 0:
        angulo = 0
    if angulo > 180:
        angulo = 180
    return int(angulo * RECORRIDO / 180 + DUTY_MINIMO)

def barrer(desde, hasta):
    paso = PASO if hasta > desde else -PASO
    for angulo in range(desde, hasta + paso, paso):
        servo.duty(angulo_a_duty(angulo))
        time.sleep_ms(ESPERA_MS)

while True:
    barrer(0, 180)
    barrer(180, 0)
    print("Barrido completo")

```

QUÉ DEBE OBSERVARSE

El aspa recorre media vuelta de ida y de vuelta sin tirones, y la consola escribe una línea por ciclo completo. En los extremos el servo debe **callarse**: si zumba, todavía está llegando al tope y hay que bajar `DUTY_MAXIMO` de a cinco unidades hasta que deje de hacerlo.

El mismo algoritmo en...

En el área eléctrica y electrónica, un servomotor de este tipo mueve compuertas de ventilación, válvulas de tres vías y desviadores en cintas transportadoras. El recorte del recorrido no es una precaución teórica: es el equivalente al final de carrera de una máquina, y en un actuador grande la diferencia entre recortarlo y no hacerlo es una reparación.

En el área mecánica, el mismo esquema gobierna las compuertas del sistema de climatización y el actuador de ralentí. Los servos del vehículo trabajan con realimentación de posición, pero el pulso de mando responde a la misma norma de 20 ms que se usa aquí.

5.5 Tonos y sirena con zumbador pasivo

OBJETIVO

Usar la frecuencia como información, no como detalle: aquí lo que se controla es la nota.

MATERIALES

Cantidad	Componente	Observación
1	Zumbador pasivo	el activo no sirve
—	Cables de conexión	

EL CIRCUITO

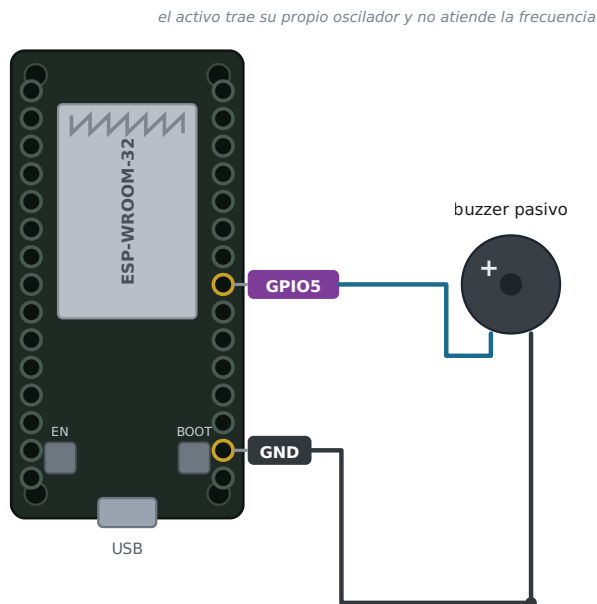


Figura 5.4 — El zumbador pasivo no tiene oscilador propio: la frecuencia se la entrega el pin, y por eso se lo puede hacer sonar en cualquier nota.

Cómo distinguir un zumbador pasivo de uno activo

Mirados por debajo, el activo suele venir sellado con resina y el pasivo deja ver la bobina. La prueba segura es conectarlo un instante a 3,3 V continuos: el **activo** suena con una nota fija; el **pasivo** hace un chasquido y se calla. Sólo este último sirve para el laboratorio.

RAZONAMIENTO

Aquí se invierte todo lo dicho hasta ahora. La frecuencia deja de ser un detalle de implementación y pasa a ser el dato: 440 Hz es un la, 880 Hz es el la de la octava siguiente. El ciclo de trabajo, en cambio, casi no importa, y se deja en la mitad porque ahí la onda cuadrada es simétrica y el tono sale más limpio.

Un detalle de programación que vale para todo el libro: el zumbador debe quedar callado cuando el programa termina. Si se interrumpe con `Ctrl+C` a mitad de un tono, el PWM sigue funcionando por su cuenta y el zumbador sigue sonando hasta que se reinicie la placa. El programa atrapa esa interrupción y lo apaga.

PROGRAMA

Archivo `lab_5_5_buzzer.py`

```
from machine import Pin, PWM
import time

buzzer = PWM(Pin(5))
buzzer.duty_u16(0)          # arranca en silencio

MITAD = 32767                # 50 % de 65535: onda cuadrada simetrica
NOTAS = (("do", 262), ("re", 294), ("mi", 330), ("fa", 349),
         ("sol", 392), ("la", 440), ("si", 494))

try:
    print("Escala")
    buzzer.duty_u16(MITAD)
    for nombre, hz in NOTAS:
        buzzer.freq(hz)
        print("  {:4s} {:3d} Hz".format(nombre, hz))
        time.sleep_ms(400)
```

```

print("Sirena. Interrumpa con Ctrl+C.")
while True:
    for hz in (400, 800):
        buzzer.freq(hz)
        time.sleep_ms(200)

except KeyboardInterrupt:
    pass

finally:
    # Pase lo que pase, el zumbador queda callado. Sin esto, una
    # interrupcion a mitad de un tono lo deja sonando hasta que
    # alguien reinicie la placa.
    buzzer.duty_u16(0)
    buzzer.deinit()
    print("Silencio.")

```

QUÉ DEBE OBSERVARSE

Primero suena una escala de siete notas, cada una un poco más aguda, y después arranca la sirena de dos tonos. Al interrumpir con **Ctrl+C** el zumbador se calla de inmediato y la consola escribe la última línea.

SI ALGO NO SALE

Síntoma	Causa habitual
Suena siempre la misma nota	Es un zumbador activo. No hay programa que lo arregle.
No suena nada	Falta <code>duty_u16(MITAD)</code> : fijar la frecuencia no basta, hay que abrir el ciclo de trabajo.
Sigue sonando después de detener el programa	Se ejecutó una versión sin el bloque <code>finally</code> . El botón EN lo calla.
El sonido es muy débil	Normal: el zumbador pasivo suena poco alimentado desde un pin. No conviene subir la corriente; si hace falta volumen, va un transistor.

El mismo algoritmo en...

En el área eléctrica y electrónica, la sirena de dos tonos es la señal acústica de alarma de una máquina, y la escala sirve para distinguir avisos: un tono grave para el arranque inminente, uno agudo para la parada de emergencia. Que el programa pueda elegir la nota es lo que permite codificar el tipo de aviso con un solo zumbador.

En el área mecánica, es el avisador acústico del tablero. Un solo elemento distingue el cinturón sin abrochar, la puerta abierta y la marcha atrás, cambiando frecuencia y cadencia.

EJERCICIOS PROPUESTOS

1. En el laboratorio 5.2, invierta el sentido: que el LED esté al máximo con la perilla al mínimo. ¿Hace falta tocar el cableado?
2. Modifique el 5.2 para que el brillo cambie por escalones de 10 %, en lugar de hacerlo de manera continua. ¿Cuántos niveles distintos quedan?
3. El ojo percibe el brillo de manera aproximadamente logarítmica. Sustituya `brillo = cuentas >> 2` por una relación cuadrática — `brillo = cuentas * cuentas // 16392` — y compare. ¿Cuál de las dos se siente más uniforme al girar la perilla?
4. Con el potenciómetro del 5.2 y el servo del 5.3, escriba un programa que posicione el servo según la perilla. Recorte el recorrido como corresponde.
5. Calcule qué ciclo de trabajo produce un pulso de 1,5 ms a 50 Hz usando `duty_u16()` en lugar de `duty()`. Verifique que el servo va al centro.
6. En el laboratorio 5.5, escriba una función `tono(hz, ms)` que suene una nota y se calle, y úsela para tocar una melodía corta a partir de una lista de pares.
7. Un LED regulado al 50 % de ciclo de trabajo, ¿consume la mitad de corriente media que al 100 %? ¿Y disipa la mitad de potencia? Justifique.

Sensores con bus y protocolo

Los capítulos anteriores trataron señales: un nivel, una tensión, un tren de pulsos. En todos los casos el sensor entregaba algo físico y el programa lo interpretaba.

Hay otra familia de sensores que no entrega una señal sino **datos**. Adentro tienen su propio microcontrolador, que ya hizo la medición y la conversión, y lo que sale por el cable son números codificados según un acuerdo. Ahí el trabajo del programa deja de ser interpretar una señal y pasa a ser **conversar**.

Ese cambio trae dos ventajas y un costo. Las ventajas son que la medición viene calibrada de fábrica y que varios sensores pueden compartir los mismos dos cables. El costo es que hay que respetar el acuerdo al pie de la letra, y cuando algo no funciona el sensor no dice por qué: simplemente no contesta. Los seis laboratorios de este capítulo recorren cuatro acuerdos distintos, de más a menos normalizado.

Acuerdo	Cables	Sensor de este capítulo	Laboratorio
I ² C, bus normalizado	2 compartidos	Pantalla OLED	6.1 y 6.2
Un hilo, propio del fabricante	1	DHT11	6.3
Con reloj, propio del fabricante	2	HX710B	6.4
Puerto serie con Modbus	2	PZEM-004T	6.5

6.1 Explorar el bus I²C

OBJETIVO

Averiguar qué hay conectado al bus antes de escribir una sola línea de programa para usarlo.

MATERIALES

Cantidad	Componente	Observación
1	Pantalla OLED de 128×64	con controlador SSD1306, I ² C

EL CIRCUITO

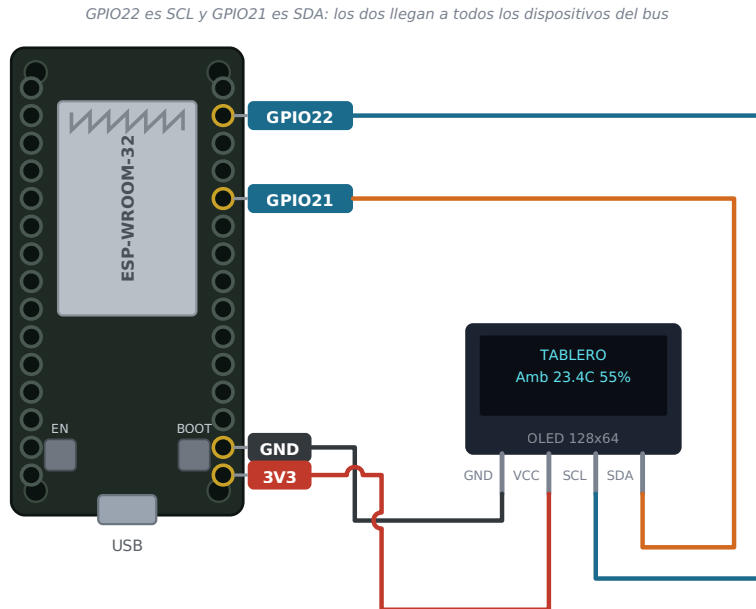


Figura 6.1 — Los cuatro cables de la pantalla. SCL y SDA son el bus, y los mismos dos hilos servirán para cualquier otro dispositivo I²C que se agregue después.

RAZONAMIENTO

Un bus es una línea compartida. En I²C hay sólo dos hilos —**SCL**, el reloj, y **SDA**, los datos— y todos los dispositivos cuelgan de ellos en paralelo. Para que no se pisen, cada uno tiene una **dirección** de siete bits, y el ESP32 empieza cada conversación nombrando a quién le habla.

La consecuencia práctica es que agregar un segundo sensor no cuesta cables: se conecta a los mismos dos. El límite está en las direcciones, y ahí aparece el problema típico: dos dispositivos iguales tienen la misma dirección de fábrica y no pueden convivir sin cambiarla —muchos módulos traen un puente para eso.

Antes de programar nada conviene preguntarle al bus quién está. MicroPython lo resuelve en una línea.

PROGRAMA

Archivo `lab_6_1_explorar_i2c.py`

```
from machine import Pin, SoftI2C

PIN_SCL = 22
PIN_SDA = 21

CONOCIDOS = {
    0x3C: "pantalla OLED SSD1306",
    0x3D: "pantalla OLED SSD1306 (direccion alternativa)",
    0x27: "pantalla LCD con adaptador PCF8574",
    0x68: "reloj DS1307 o acelerometro MPU6050",
    0x76: "sensor de presion BMP280 o BME280",
    0x77: "sensor de presion BMP280 (direccion alternativa)",
}

i2c = SoftI2C(scl=Pin(PIN_SCL), sda=Pin(PIN_SDA), freq=400000)

print("Explorando el bus I2C en SCL=GPIO{}, SDA=GPIO{}".format(
    PIN_SCL, PIN_SDA))
print("")

encontrados = i2c.scan()

if not encontrados:
    print("No contesta nadie.")
    print(" - Revise VCC y GND del modulo.")
```

```

    print(" - Revise que SCL y SDA no esten intercambiados.")
    print(" - Algunos modulos necesitan resistencias de elevacion.")
else:
    print(" direccion          que suele ser")
    for direccion in encontrados:
        nombre = CONOCIDOS.get(direccion, "desconocido")
        print(" 0x{:02X} ({:3d}) {}".format(
            direccion, direccion, nombre))
    print("")
    print("{} dispositivo(s) en el bus.".format(len(encontrados)))

```

QUÉ DEBE OBSERVARSE

Explorando el bus I2C en SCL=GPI022, SDA=GPI021

```

direccion          que suele ser
0x3C ( 60)        pantalla OLED SSD1306

```

1 dispositivo(s) en el bus.

Este programa vale para todo el capítulo

Cada vez que un sensor I²C no responda, lo primero es correr este explorador. Si el dispositivo no aparece, el problema es de cableado o de alimentación y no tiene sentido revisar el programa. Si aparece, el cableado está bien y el problema es otro.

Es el equivalente a comprobar la continuidad antes de buscar la falla en el circuito de mando.

SI ALGO NO SALE

Síntoma	Causa habitual
No contesta nadie	SCL y SDA intercambiados —es lo más frecuente—, o el módulo sin alimentar.
Aparece una dirección inesperada	El módulo es de otro fabricante. La dirección que importa es la que aparece, no la de la hoja de datos.
Aparecen muchísimas direcciones	SDA está suelto y el bus lee ruido.
Aparece a veces sí y a veces no	Faltan las resistencias de elevación, o el cable es demasiado largo para 400 kHz. Baje <code>freq</code> a 100000.

6.2 La pantalla OLED

OBJETIVO

Mostrar texto y gráficos en la pantalla, y entender por qué nada aparece hasta que se lo pide expresamente.

EL CIRCUITO

El mismo del laboratorio 6.1 (figura 6.1).

Hace falta el módulo `ssd1306.py`

MicroPython no trae el controlador de la pantalla. Hay que copiar el archivo `ssd1306.py` **dentro de la placa**, igual que `config.py` o `ble_uart.py`. Sin él, `import ssd1306` falla y no hay programa que lo arregle.

RAZONAMIENTO

La pantalla no se dibuja sola. Todas las instrucciones de dibujo —`text`, `rect`, `fill_rect`, `hline`— trabajan sobre una copia en la memoria del ESP32, y no se ve nada hasta que se llama a `show()`, que manda esa copia entera a la pantalla.

Es la causa del problema más común con estas pantallas: el programa dibuja bien y la pantalla queda negra. Falta `show()`.

Tiene una consecuencia útil: como el dibujo se arma completo antes de mostrarse, no hay parpadeo. Lo habitual es `fill(0)` para borrar, después todo el dibujo, y `show()` al final.

Instrucción	Qué hace
<code>oled.fill(0)</code>	Borra la copia en memoria.
<code>oled.text(s, x, y, 1)</code>	Escribe. Cada carácter mide 8×8 píxeles.
<code>oled.rect(x, y, w, h, 1)</code>	Dibuja el contorno.
<code>oled.fill_rect(x, y, w, h, 1)</code>	Lo dibuja relleno.
<code>oled.hline(x, y, largo, 1)</code>	Una línea horizontal.
<code>oled.show()</code>	Manda todo a la pantalla.

128 por 64, y ni un píxel más

Con caracteres de 8×8 píxeles entran **16 columnas y 8 filas** de texto. Dibujar fuera de esos límites no da error: el controlador simplemente descarta lo que sobra, de modo que una barra mal calculada aparece cortada sin ninguna pista de por qué. Conviene calcular las posiciones con la altura de celda como unidad y no con números sueltos.

PROGRAMA

Archivo `lab_6_2_oled.py`

```
from machine import Pin, SoftI2C
import ssd1306
import time

ANCHO, ALTO, CELDA = 128, 64, 8

i2c = SoftI2C(scl=Pin(22), sda=Pin(21))
oled = ssd1306.SSD1306_I2C(ANCHO, ALTO, i2c)

def centrar(texto, fila):
```

```

"""Escribe centrado en una fila, contando en celdas de 8x8."""
x = (ANCHO - len(texto) * CELDA) // 2
oled.text(texto, max(0, x), fila * CELDA, 1)

def barra(valor, minimo, maximo, x, y, ancho, alto):
    """Barra de progreso. Recorta el valor para no salirse."""
    oled.rect(x, y, ancho, alto, 1)
    recorrido = maximo - minimo
    if recorrido <= 0:
        return
    proporcion = (valor - minimo) / recorrido
    if proporcion < 0:
        proporcion = 0
    if proporcion > 1:
        proporcion = 1
    lleno = int((ancho - 2) * proporcion)
    if lleno > 0:
        oled.fill_rect(x + 1, y + 1, lleno, alto - 2, 1)

def main():
    # --- texto centrado
    oled.fill(0)
    centrar("SENSORICA", 2)
    centrar("ESP32", 4)
    oled.show() # sin esta linea la pantalla queda negra
    time.sleep(2)

    # --- una barra que se llena
    for valor in range(0, 101, 5):
        oled.fill(0)
        centrar("CARGA", 0)
        oled.text("{:3d} %".format(valor), 48, 2 * CELDA, 1)
        barra(valor, 0, 100, 8, 4 * CELDA, 112, 14)
        oled.show()
        time.sleep_ms(80)

    # --- lo que no entra, se recorta solo
    oled.fill(0)
    centrar("RECORTE", 0)
    barra(150, 0, 100, 8, 3 * CELDA, 112, 14) # 150 sobre 100
    barra(-20, 0, 100, 8, 5 * CELDA, 112, 14) # -20 sobre 100
    oled.show()

```

```
if __name__ == "__main__":  
    main()
```

QUÉ DEBE OBSERVARSE

Primero dos líneas centradas, después una barra que se llena sin parpadear, y al final dos barras con valores imposibles: una llena del todo y otra vacía. Que un valor fuera de rango no rompa el dibujo es deliberado, y es la misma idea que el indicador del laboratorio 7.3.

6.3 DHT11: un hilo y un acuerdo propio

OBJETIVO

Leer un sensor que habla por un solo cable con un protocolo que no es de nadie más, y convivir con que a veces se equivoque.

EL CIRCUITO

VCC a 3V3, GND a masa y DATA a GPIO14. Es el montaje que reaparece en los laboratorios 7.2, 7.4 y 7.6 (figura 7.2).

RAZONAMIENTO

El DHT11 usa un solo hilo y un protocolo inventado por su fabricante: el ESP32 lo baja un rato para pedir la medición, y el sensor contesta con 40 bits cuya duración de pulso codifica cada uno. Todo eso lo hace el módulo `dht`, que viene con MicroPython: el programa sólo llama a `measure()` y después lee.

Lo que sí es asunto del programa es que **esa lectura falla de vez en cuando**. Los últimos 8 de los 40 bits son una suma de verificación, y cuando no cuadra MicroPython levanta un `OSError`. No significa que el sensor esté malo: significa que esa trama llegó mal y hay que pedir otra.

Un programa que no atrape ese error se detiene, y lo hará en algún momento: es cuestión de minutos. Todo programa de este libro que use un DHT11 tiene el `try`, y no es una precaución teórica.

Dos segundos entre lecturas, como mínimo

El DHT11 no admite que se le pida una medición más seguido que cada dos segundos. Pidiéndosela antes, devuelve la anterior o directamente falla. Es la causa de la mitad de los `OSError` que aparecen en clase: no es el cableado, es el ritmo.

PROGRAMA

Archivo `lab_6_3_dht11.py`

```
from machine import Pin
import time
import dht

PIN_DATOS = 14
INTERVALO = 2          # segundos. El DHT11 no admite menos.
AVISAR_TRAS = 5         # fallas seguidas antes de sospechar del cable

sensor = dht.DHT11(Pin(PIN_DATOS))

def leer():
    """Temperatura y humedad, o None si la trama llego mal.

    La suma de verificacion del DHT11 falla cada tanto y no
    significa que el sensor este malo: hay que pedir otra trama.
    Sin este try, el programa se detiene.
    """
    try:
        sensor.measure()
        return sensor.temperature(), sensor.humidity()
    except OSError:
        return None

def main():
    print("DHT11 en GPIO{}".format(PIN_DATOS))
    print("")
    fallas = 0

    while True:
        lectura = leer()

        if lectura is None:
            fallas = fallas + 1
```

```

        print("Lectura fallida ({}).".format(fallas))
        if fallas == AVISAR TRAS:
            print("Varias seguidas: revise el cable y la")
            print("resistencia de elevacion en la linea.")
        else:
            temperatura, humedad = lectura
            fallas = 0
            print("Temperatura: {} C    Humedad: {} %".format(
                temperatura, humedad))

        time.sleep(INTERVALO)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Las lecturas salen cada dos segundos. De tanto en tanto aparece una línea de falla y enseguida vuelve a andar: eso es normal. Lo que no es normal es que fallen muchas seguidas, y por eso el programa avisa.

Soplando sobre el sensor la humedad sube enseguida y baja despacio. La temperatura se mueve poco: el DHT11 da grados enteros y no tiene resolución para cambios finos.

6.4 HX710B: un protocolo con reloj propio

OBJETIVO

Leer un conversor de 24 bits marcándole el compás, y resolver el problema de signo que eso trae.

MATERIALES

Cantidad	Componente	Observación
1	Transmisor de presión con HX710B	de 0 a 40 kPa

EL CIRCUITO

OUT lleva el dato y SCK el reloj; la placa marca el ritmo

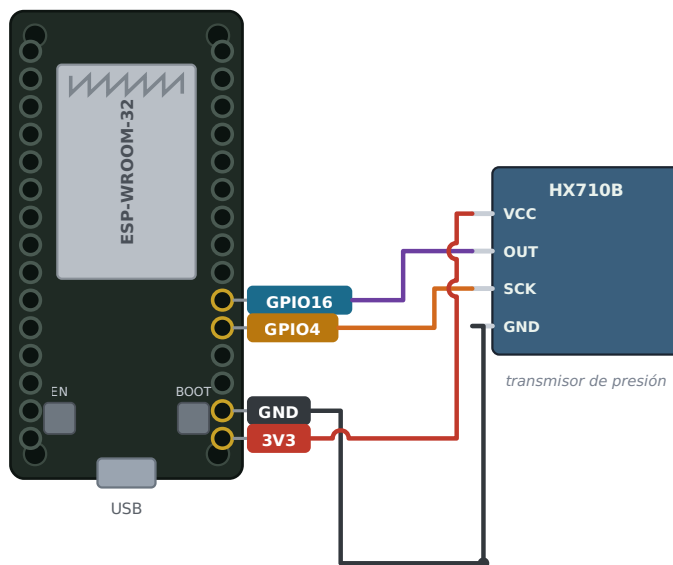


Figura 6.2 — Dos hilos de datos: OUT trae el dato y SCK lleva el reloj. La placa marca el ritmo y el integrado contesta bit a bit.

RAZONAMIENTO

Aquí no hay bus ni norma. El HX710B avisa que tiene un dato listo poniendo su salida en cero, y a partir de ahí el ESP32 le da **veinticuatro pulsos de reloj**; en cada uno el integrado pone un bit en la salida. Veinticuatro pulsos, veinticuatro bits. Después hay un pulso más que le dice con qué ganancia debe hacer la medición siguiente.

Todo eso hay que escribirlo a mano, y es lo que hace este laboratorio distinto de los anteriores: no hay módulo que lo resuelva.

El error de signo que sólo ocurre en Python

El HX710B entrega un número con signo en complemento a dos de 24 bits. Los ejemplos en C lo convierten así:

```
if (dato & 0x800000) dato |= 0xFF000000;
```

Esa línea traducida a Python **no funciona**. En C las variables tienen 32 bits y poner los ocho de arriba produce el número negativo correcto. En Python los enteros no tienen tamaño: la misma operación da **4 294 967 196** en lugar de **-100**.

La conversión correcta en Python es restar el rango completo: `dato = dato - 0x1000000`. Es un error que no da ningún síntoma hasta que el sensor mide por debajo del cero, y entonces aparecen valores astronómicos.

PROGRAMA

Archivo `lab_6_4_presion.py`

```
from machine import Pin
import time

PIN_DATOS = 16
PIN_RELOJ = 4
INTERVALO = 2

# Calibracion. No son numeros magicos: son los dos puntos de la
# recta del laboratorio 3.5. CERO es lo que entrega el sensor sin
# presion y FONDO lo que entrega a fondo de escala.
CERO = 1037416
FONDO = 8388607
KPA_FONDO = 40.0
KPA_A_PSI = 0.145038

salida = Pin(PIN_DATOS, Pin.IN)
reloj = Pin(PIN_RELOJ, Pin.OUT)
reloj.value(0)

def leer_crudo(espera_ms=200):
    """Los 24 bits del HX710B, o None si el sensor no responde.

    El integrado avisa que tiene un dato listo poniendo su salida
```

```

en cero. Si eso no ocurre dentro del plazo se abandona, en
lugar de quedar esperando para siempre.
"""
limite = time.time() + espera_ms
while salida.value() == 1:
    if time.time() - limite >= 0:
        return None
    time.sleep(1)

dato = 0
for _ in range(24):
    reloj.value(1)
    dato = (dato << 1) | salida.value()
    reloj.value(0)

reloj.value(1)          # pulso 25: fija la ganancia de la proxima
reloj.value(0)

# Complemento a dos de 24 bits. En Python NO se hace poniendo
# los bits de arriba: los enteros no tienen tamaño y saldría un
# número enorme. Se resta el rango completo.
if dato & 0x800000:
    dato = dato - 0x1000000
return dato

def presion(crudo):
    """Pasa las cuentas a kPa y a psi, con la recta de calibración."""
    kpa = (crudo - CERO) * KPA_FONDO / (FONDO - CERO)
    return kpa, kpa * KPA_A_PSI

def main():
    print("HX710B en OUT=GPIO{} SCK=GPIO{}".format(
        PIN_DATOS, PIN_RELOJ))
    print("Calibración: {} = 0 kPa, {} = {:.0f} kPa".format(
        CERO, FONDO, KPA_FONDO))
    print("")

    while True:
        crudo = leer_crudo()

        if crudo is None:
            print("El sensor no responde. Revise OUT, SCK y VCC.")
        else:
            kpa, psi = presion(crudo)
            print("{:9d} cuentas   {:.2f} kPa   {:.2f} psi".format(
                crudo, kpa, psi))

```

```

        time.sleep(INTERVAL0)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

Sin presión la lectura ronda el valor de **CERO** y la columna de kPa queda cerca de cero. Soplando por el tubo la presión sube y vuelve al soltar. Las dos constantes de calibración son de *ese* sensor: para otro hay que rehacerlas con el procedimiento del laboratorio 3.5.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el transmisor de presión de una línea neumática o de un filtro. La lectura de presión diferencial entre la entrada y la salida de un filtro es lo que dice cuándo hay que cambiarlo.

En el área mecánica, el mismo integrado se usa en los sensores de presión de múltiple y en los medidores de presión de aceite de tablero digital.

6.5 PZEM-004T: conversar con un instrumento

OBJETIVO

Leer cuatro magnitudes eléctricas de un instrumento que habla Modbus por puerto serie.

MATERIALES

Cantidad	Componente	Observación
1	Medidor PZEM-004T	con su bobina toroidal
1	Carga para ensayar	una lámpara sirve

Este laboratorio trabaja sobre 220 V

El PZEM se conecta a la red de corriente alterna. El montaje se hace con la línea **desconectada**, se revisa completo y recién entonces se energiza. La parte de baja tensión —los cuatro hilos al ESP32— está aislada de la de red dentro del módulo, pero eso no autoriza a manipular el conjunto con tensión presente.

Si el curso no tiene un puesto preparado para trabajar con red, este laboratorio se demuestra y no se arma.

EL CIRCUITO

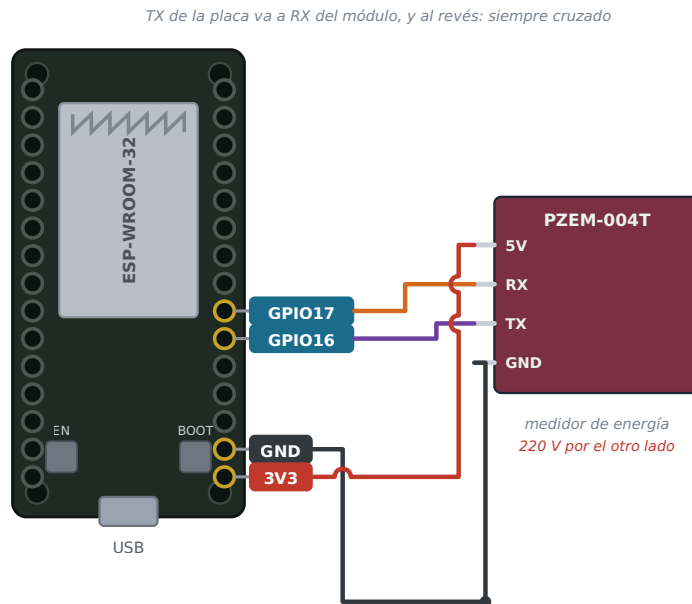


Figura 6.3 — Los cuatro hilos de baja tensión. TX de la placa va a RX del módulo y al revés: en un enlace serie los nombres se cruzan siempre.

La bobina abraza UN solo conductor

El toroide mide el campo magnético que produce la corriente. Si se lo cierra alrededor de los **dos** conductores de la línea, los dos campos son iguales y opuestos, se cancelan, y la corriente medida da **cero** con todo perfectamente conectado.

Es el error más frecuente de este montaje y el más desconcertante, porque la tensión se mide bien y sólo la corriente sale mal.

RAZONAMIENTO

Un puerto serie tiene dos hilos, uno para cada sentido, y los nombres están puestos desde el punto de vista de cada extremo: lo que uno transmite el otro lo recibe. Por eso **TX va siempre a RX**. Conectarlos derecho —TX con TX— es el otro error clásico, y el síntoma es silencio absoluto.

Sobre esos dos hilos el PZEM habla **Modbus**, que es el protocolo industrial más extendido: el maestro pide un grupo de registros, el esclavo contesta, y cada mensaje lleva una verificación al final. El módulo `pzem.py` se ocupa de armar y comprobar esas tramas; hay que copiarlo a la placa.

Como cualquier conversación, ésta puede fallar: el instrumento puede no contestar a tiempo o contestar mal. El programa lo comprueba antes de usar los valores, en lugar de dar por bueno lo que haya.

PROGRAMA

Archivo `lab_6_5_pzem.py`

```
from machine import UART
from pzem import PZEM
import time

INTERVALO = 3

uart = UART(1, baudrate=9600, tx=17, rx=16)
medidor = PZEM(uart)

MAGNITUDES = (("Tension", "getVoltage", "{:7.2f} V"),
```

```

        ("Corriente", "getCurrent", "{:7.3f} A"),
        ("Potencia activa", "getActivePower", "{:7.2f} W"),
        ("Factor de potencia", "getPowerFactor", "{:7.3f}")

def potencia_aparente(tension, corriente):
    """Los voltiamperios: lo que la instalacion tiene que soportar."""
    return tension * corriente

def main():
    print("Medidor PZEM-004T")
    print("")

    while True:
        if not medidor.read():
            print("Sin lectura valida. Revise el cableado serie.")
            time.sleep(INTERVALO)
            continue

        valores = []
        for nombre, metodo, formato in MAGNITUDES:
            valor = getattr(medidor, metodo)()
            valores.append(valor)
            print("  {:<20}{:7.2f}".format(nombre, formato.format(valor)))

        # Lo que el instrumento no da y conviene calcular
        aparente = potencia_aparente(valores[0], valores[1])
        reactiva = (aparente ** 2 - valores[2] ** 2) ** 0.5
        print("  {:<20}{:7.2f} VA".format(
            "Potencia aparente", aparente))
        print("  {:<20}{:7.2f} var".format(
            "Potencia reactiva", reactiva))
        print("")

        time.sleep(INTERVALO)

if __name__ == "__main__":
    main()

```

Por qué se calculan la aparente y la reactiva

El instrumento entrega la potencia **activa**, que es la que hace trabajo. Pero el conductor y la protección se dimensionan por la **aparente**, que es tensión por corriente sin importar el desfase. Con un factor de potencia de 0,7, una carga de 700 W exige una instalación de 1000 VA.

Que el libro las calcule y no sólo las muestre es deliberado: es la diferencia entre leer un instrumento y entender lo que dice.

QUÉ DEBE OBSERVARSE

Con una lámpara incandescente el factor de potencia da cerca de 1 y la aparente coincide con la activa. Con una carga inductiva —un motor pequeño, un balasto— el factor baja y la diferencia entre las dos potencias se hace visible. Ése es el experimento que vale la pena hacer.

6.6 Tablero local: leer, decidir y mostrar

OBJETIVO

Juntar lo del capítulo en un solo programa, repartido de manera que el capítulo 7 pueda agregarle el envío sin tocar nada.

EL CIRCUITO

La pantalla del laboratorio 6.1, el DHT11 del 6.3 en GPIO14 y el termistor del 3.4 en GPIO33, todos a la vez.

RAZONAMIENTO

Este laboratorio no trae ninguna técnica nueva. Lo que trae es una manera de repartir el programa, y conviene detenerse en ella porque es lo que hace posible el capítulo siguiente.

El programa se parte en tres responsabilidades que no se mezclan:

Responsabilidad	Qué hace	Qué NO hace
Leer	Habla con los sensores y devuelve números, o None si algo falló.	No decide nada ni muestra nada.
Decidir	Compara con los umbrales y saca conclusiones.	No sabe de dónde salieron los números ni adónde van.
Mostrar	Dibuja en la pantalla.	No lee sensores ni decide.

Escrito así, agregar la publicación en internet del capítulo 7 es agregar una **cuarta** responsabilidad al lado de las otras tres, sin tocar ninguna. Si en cambio la lectura del sensor estuviera metida dentro del dibujo —que es lo que sale naturalmente— habría que desarmarlo todo.

El ejercicio 6 del capítulo 7 pide justamente eso, y sirve para comprobar si la separación era real o sólo aparente.

PROGRAMA

Archivo `lab_6_6_tablero.py`

```
from machine import Pin, SoftI2C, ADC
import math
import time
import dht
import ssd1306

ANCHO, ALTO, CELDA = 128, 64, 8
INTERVALO = 3
TEMPERATURA_ALTA = 30.0          # umbral del aviso en pantalla

# ----- leer
i2c = SoftI2C(scl=Pin(22), sda=Pin(21))
oled = ssd1306.SSD1306_I2C(ANCHO, ALTO, i2c)

sensor = dht.DHT11(Pin(14))

ntc = ADC(Pin(33))
ntc.atten(ADC.ATTN_11DB)
ntc.width(ADC.WIDTH_12BIT)
```

```

R_FIJA = 100000.0
R_NOMINAL = 100000.0
T_NOMINAL = 25.0
BETA = 3950.0
ALIMENTACION = 3.3
CERO_ABSOLUTO = 273.15

def leer_dht():
    try:
        sensor.measure()
        return sensor.temperature(), sensor.humidity()
    except OSError:
        return None

def leer_ntc(muestras=16):
    suma = 0
    for _ in range(muestras):
        suma = suma + ntc.read()
        time.sleep_ms(2)
    tension = (suma / muestras) * ALIMENTACION / 4095

    if tension <= 0 or tension >= ALIMENTACION:
        return None

    resistencia = R_FIJA * tension / (ALIMENTACION - tension)
    t0 = T_NOMINAL + CERO_ABSOLUTO
    inversa = 1.0 / t0 + math.log(resistencia / R_NOMINAL) / BETA
    return 1.0 / inversa - CERO_ABSOLUTO

# ----- mostrar
def centrar(texto, fila):
    x = (ANCHOR - len(texto) * CELDA) // 2
    oled.text(texto, max(0, x), fila * CELDA, 1)

def barra(valor, minimo, maximo, x, y, ancho, alto):
    oled.rect(x, y, ancho, alto, 1)
    recorrido = maximo - minimo
    if recorrido <= 0:
        return
    proporcion = (valor - minimo) / recorrido
    proporcion = max(0, min(1, proporcion))
    lleno = int((ancho - 2) * proporcion)
    if lleno > 0:
        oled.fill_rect(x + 1, y + 1, lleno, alto - 2, 1)

```

```

def dibujar(temperatura, humedad, temperatura_ntc, aviso):
    oled.fill(0)
    centrar("TABLERO", 0)
    oled.hline(0, 10, ANCHO, 1)

    if temperatura is None:
        oled.text("DHT11 sin datos", 0, 2 * CELDA, 1)
    else:
        oled.text("Amb {:.4lf}C {:.3d}%".format(temperatura, humedad),
                  0, 2 * CELDA, 1)

    if temperatura_ntc is None:
        oled.text("NTC desconectado", 0, 3 * CELDA, 1)
    else:
        oled.text("NTC {:.5lf} C".format(temperatura_ntc),
                  0, 3 * CELDA, 1)
        barra(temperatura_ntc, 0, 60, 4, 4 * CELDA + 4, 120, 10)

    if aviso:
        centrar("TEMP ALTA", 7)

    oled.show()

def main():
    print("Tablero local en marcha.")

    while True:
        # leer
        lectura = leer_dht()
        temperatura, humedad = lectura if lectura else (None, None)
        temperatura_ntc = leer_ntc()

        # decidir: el umbral se evalua con el sensor que haya
        referencia = temperatura_ntc
        if referencia is None:
            referencia = temperatura
        aviso = (referencia is not None
                 and referencia >= TEMPERATURA_ALTA)

        # mostrar
        dibujar(temperatura, humedad, temperatura_ntc, aviso)

        time.sleep(INTERVALO)

```

```
if __name__ == "__main__":  
    main()
```

QUÉ DEBE OBSERVARSE

La pantalla muestra las dos temperaturas y la humedad, con una barra para el termistor. Calentando la sonda con la mano hasta pasar el umbral aparece el aviso abajo.

Desconectando cualquiera de los dos sensores, la pantalla lo dice y el otro sigue funcionando. Un tablero que se apaga entero porque falló un sensor no sirve, y esa es la razón de que cada lectura devuelva `None` en vez de detener el programa.

El mismo algoritmo en...

En el área eléctrica y electrónica, este reparto es el de cualquier autómatas: la imagen de entradas, la lógica y la imagen de salidas son tres cosas separadas, y por eso un programa de autómatas se puede modificar sin desarmarlo. Lo que aquí se hace a mano, ahí lo impone la arquitectura.

En el área mecánica, es la estructura de una unidad de control: leer sensores, aplicar los mapas, mandar a los actuadores. Que el tablero siga funcionando con un sensor caído es exactamente lo que hace la computadora del vehículo cuando enciende la luz de avería.

EJERCICIOS PROPUESTOS

1. Agregue un segundo dispositivo I²C al bus del laboratorio 6.1 y compruebe que el explorador encuentra los dos. ¿Cuántos cables hubo que agregar?
2. En el laboratorio 6.2, escriba una función que dibuje un marco alrededor de toda la pantalla. ¿Qué coordenadas son las últimas válidas?
3. Quite el `try` del laboratorio 6.3 y deje el programa corriendo. Cronometre cuánto tarda en detenerse.
4. En el laboratorio 6.4, calcule qué valor en kPa daría una lectura cruda de -100 si la conversión de signo estuviera mal hecha al estilo de C.
5. Con el PZEM del 6.5 y una carga inductiva, anote el factor de potencia y calcule cuánta corriente de más circula respecto de una carga resistiva de la misma potencia activa.
6. Tome el laboratorio 6.6 y agregue un tercer sensor sin tocar `dibujar()` más que para una línea. Si tuvo que tocar más, ¿dónde estaba mal separado?

Monitoreo remoto

Hasta aquí el dato nacía y moría en la mesa de trabajo. Se medía, se decidía y se mostraba, pero para verlo había que estar sentado frente a la placa. Este capítulo lo saca de ahí.

Hay cuatro maneras de hacerlo y no compiten entre sí: resuelven problemas distintos y cada una tiene su costo. Conviene conocer las cuatro antes de elegir, porque la tentación es ir directamente a internet cuando muchas veces alcanza con algo mucho más simple.



La propia placa es el servidor

laboratorios 7.2 y 7.3

El teléfono entra a su dirección. No hace falta ninguna cuenta ni internet, pero sólo alcanza dentro de la misma red.



Al teléfono, por Bluetooth

laboratorio 7.4

El teléfono se conecta directamente a la placa. No hace falta red de ninguna clase, pero hay que estar al lado.



De placa a placa, por radio

laboratorio 7.5

ESP-NOW usa la radio del WiFi sin conectarse a ninguna red. Llega lejos y no depende de nadie, pero sólo habla con otra placa.



A un servidor de internet

laboratorios 7.6 y 7.7

MQTT publica en un servidor que guarda el histórico y se mira desde cualquier parte. Hace falta cuenta, clave y conexión.

Figura 7.1 — Las cuatro maneras de sacar el dato de la placa. Las dos primeras no necesitan más que la red del taller o ni siquiera eso; la última es la única que guarda el histórico.

El orden de los laboratorios sigue ese cuadro, de menos a más dependencias. Se empieza por lo que funciona con la placa sola y se termina en el tablero que se mira desde otra ciudad.

7.1 Conectar a la red sin publicar las claves

OBJETIVO

Conectar el ESP32 al WiFi, y establecer de una vez la disciplina que van a seguir todos los programas del capítulo: que ninguna clave quede escrita dentro del programa.

RAZONAMIENTO

Conectarse es corto: se enciende la interfaz, se le pasan el nombre de la red y la clave, y se espera. Lo que merece atención es dónde se guardan esos dos datos.

La manera evidente —escribirlos en el programa— tiene un problema que no se nota hasta que ya ocurrió. El programa se comparte con los compañeros, se sube a un repositorio, se imprime en un informe, se pega en una diapositiva. La clave del WiFi del taller viaja con él a todas partes, y lo mismo pasa con la clave del servicio de internet del laboratorio 7.6, que además da acceso a una cuenta.

La solución es separar el **programa** de la **configuración**. El programa se comparte; el archivo de configuración no sale de la placa. Es una línea más de trabajo y evita un problema que después no tiene arreglo: una clave publicada hay que cambiarla, no basta con borrar el archivo.

LOS DOS ARCHIVOS

El primero es la plantilla, y es el único de los dos que se comparte. Se copia con el nombre `config.py` y ahí se escriben los datos reales.

Archivo `config_ejemplo.py`

```
# config_ejemplo.py – plantilla. Copiela como config.py y
# complete los datos. config.py no se comparte con nadie.

WIFI_SSID = "NOMBRE_DE_SU_RED"
WIFI_CLAVE = "CLAVE_DE_SU_RED"

AIO_USUARIO = "su_usuario"
AIO_CLAVE = "aio_00000000000000000000000000000000"

# Estos cuatro recién hacen falta en el laboratorio 7.7, pero van
```

```
# aqui desde el principio: la plantilla es una sola y asi no hay dos
# versiones dando vueltas.
INFLUX_HOST = "192.168.0.10"
INFLUX_ORG = "taller"
INFLUX_BUCKET = "sensores"
INFLUX_TOKEN = "PEGUE_AQUI_SU_TOKEN"
```

El segundo es el módulo que van a importar todos los programas del capítulo. Se escribe una sola vez:

Archivo `iot.py`

```
# iot.py – conexion a la red, compartida por todo el capitulo
import time
import network
import config

def conectar_wifi(intentos=20):
    """Conecta el ESP32 a la red. Devuelve la direccion IP."""
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Conectando a la red", config.WIFI_SSID, "...")
        wlan.connect(config.WIFI_SSID, config.WIFI_CLAVE)
        for _ in range(intentos * 2):
            if wlan.isconnected():
                break
            time.sleep(0.5)

    if not wlan.isconnected():
        raise OSError("No se pudo conectar. Revise config.py")

    ip = wlan.ifconfig()[0]
    print("Red conectada. IP:", ip)
    return ip
```

Si una clave ya se publicó

Borrar el archivo no alcanza. Una clave que estuvo publicada hay que darla por perdida y **cambiarla**: la del WiFi en el router, la del servicio de internet regenerándola desde la cuenta. Es incómodo una vez y barato comparado con la alternativa.

QUÉ DEBE OBSERVARSE

```
>>> import iot
>>> iot.conectar_wifi()
Conectando a la red TALLER ...
Red conectada. IP: 192.168.0.50
'192.168.0.50'
```

Esa dirección es la que va a hacer falta en el laboratorio siguiente. Conviene anotarla: en muchas redes cambia cada vez que la placa se reconecta.

SI ALGO NO SALE

Síntoma	Causa habitual
«No se pudo conectar. Revise config.py»	Nombre o clave equivocados. Ojo con las mayúsculas: se distinguen.
Conecta en casa y no en el taller	La red del taller es de 5 GHz . El ESP32 sólo trabaja en 2,4 GHz y esa red no le aparece siquiera.
<code>ImportError: no module named 'config'</code>	Falta copiar la plantilla como <code>config.py</code> dentro de la placa , no en el computador.
Conecta pero después se cae sola	Señal débil, o la red exige portal de acceso. El ESP32 no puede pasar un portal con usuario y contraseña de navegador.

7.2 La placa como servidor web

OBJETIVO

Que cualquier teléfono de la misma red abra la dirección de la placa y vea la medición, sin ninguna cuenta ni servicio de por medio.

MATERIALES

Cantidad	Componente	Observación
1	Sensor DHT11	el mismo del laboratorio 6.3
1	Teléfono o computadora	en la misma red que la placa

EL CIRCUITO

es el mismo montaje del laboratorio 6.3: no hay que recablear

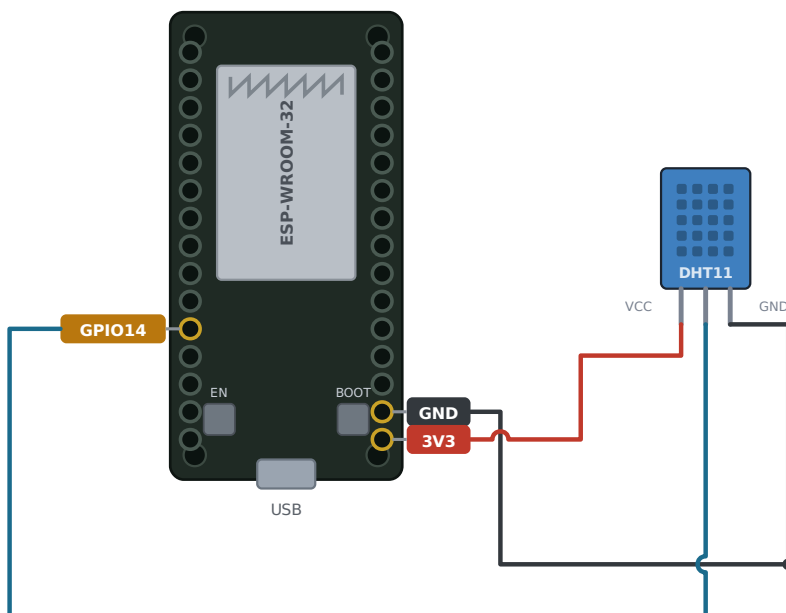


Figura 7.2 — El montaje es el del laboratorio 6.3, sin recablear. GPIO14 está en la columna izquierda, así que su cable rodea la placa.

RAZONAMIENTO

Un servidor web es un programa que espera conexiones en un puerto, y cuando llega una, contesta con un texto. Ese texto es la página. No hay más misterio: el navegador pide, la placa responde.

La parte que sí tiene miga es **dónde se mide**. La manera natural de escribirlo es: llega la visita, leo el sensor, contesto. Y esa manera está mal por dos razones.

La primera es el tiempo. El DHT11 necesita alrededor de dos segundos entre lecturas, de modo que hay que esperarlo; y durante esa espera el programa no atiende a nadie más. Con la página refrescándose sola cada cuatro segundos, la mitad del tiempo el servidor está dormido. Con un curso entero apuntando a la misma placa, las visitas se hacen cola. Medido sobre el programa: **2,00 s por visita y 20 s para diez visitas seguidas**.

La segunda es más grave. El DHT11 entrega de vez en cuando una trama con error de suma de verificación —es normal, no significa que esté malo—, y si la lectura está dentro del lazo sin protección, ese error termina el programa. La página deja de responder hasta que alguien reinicia la placa.

La corrección es una sola y arregla las dos cosas: **medir por reloj**, en el lazo principal, y que la visita reciba el último valor guardado. La misma idea del laboratorio 2.4, donde la sirena no podía impedir que el programa siguiera vigilando. Con la medición separada, la respuesta baja a **menos de 10 ms** y un sensor que falla deja el último valor bueno en pantalla en vez de matar el servidor.

PROGRAMA

Archivo `lab_7_2_servidor_web.py`

```
import socket
import time
import network
from machine import Pin
import dht
import config

PIN_DATOS = 14
PUERTO = 80
INTERVALO_MS = 3000      # cada cuanto se toma una medicion nueva

sensor = dht.DHT11(Pin(PIN_DATOS))
```

```

# Ultima medicion valida. El servidor entrega esto, no una lectura
# hecha en el momento: medir dentro de la atencion de la peticion
# haria esperar al visitante y bloquearia a los demas.
temperatura = None
humedad = None
medido_en = 0
fallas = 0

def conectar_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Conectando a la red", config.WIFI_SSID, "...")
        wlan.connect(config.WIFI_SSID, config.WIFI_CLAVE)
        for _ in range(40):
            if wlan.isconnected():
                break
            time.sleep(0.5)
    if not wlan.isconnected():
        raise OSError("No se pudo conectar. Revise config.py")
    return wlan.ifconfig()[0]

def medir_si_toca():
    """Toma una medicion nueva si paso el intervalo.

    Si el sensor falla se conserva el ultimo valor bueno en lugar
    de detener el servidor.
    """
    global temperatura, humedad, medido_en, fallas
    if temperatura is not None and \
        time.time_diff(time.time_ms(), medido_en) < INTERVALO_MS:
        return
    try:
        sensor.measure()
        temperatura = sensor.temperature()
        humedad = sensor.humidity()
        fallas = 0
    except OSError:
        fallas = fallas + 1
    medido_en = time.time_ms()

def pagina():
    if temperatura is None:
        valores = "<p>Esperando la primera medicion...</p>"
    else:

```

```

        valores = ("<p>Temperatura: {} C</p>\n"
                    "<p>Humedad: {} %</p>").format(temperatura, humedad)
    if fallas > 0:
        valores += ("\n<p class='aviso'>{} lecturas "
                    "fallidas</p>").format(fallas)

    return """<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta http-equiv="refresh" content="4">
<title>Monitoreo DHT11</title>
<style>
body {{ text-align:center; font-family:Arial, sans-serif; }}
h1 {{ color:navy; }}
p {{ font-size:30px; }}
p.aviso {{ font-size:16px; color:#A81C22; }}
</style>
</head>
<body>
<h1>MONITOREO SENSOR DHT11</h1>
{}
</body>
</html>
""".format(valores)

```

```

def camino_pedido(peticion):
    """Extrae el camino de la primera linea: GET /algo HTTP/1.1"""
    try:
        return peticion.split(" ")[1]
    except IndexError:
        return "/"

```

```

def main():
    ip = conectar_wifi()

    servidor = socket.socket()
    # Sin esto, al reiniciar el programa el puerto queda ocupado un
    # rato y la placa responde "address in use".
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    servidor.bind(socket.getaddrinfo("0.0.0.0", PUERTO)[0][-1])
    servidor.listen(2)
    servidor.settimeout(1)      # para poder medir entre visita y visita

    print("Servidor listo. Abra http://{}/ en el navegador.".format(ip))

```

```

while True:
    medir_si_toca()

    try:
        cliente, direccion = servidor.accept()
    except OSError:
        continue          # no vino nadie: se vuelve a medir

    try:
        cliente.settimeout(3)
        peticion = cliente.recv(1024).decode()

        if camino_pedido(peticion) == "/favicon.ico":
            # El navegador pide siempre el icono. Se le contesta
            # que no existe en vez de mandarle la pagina entera.
            cliente.sendall(b"HTTP/1.0 404 Not Found\r\n\r\n")
        else:
            cliente.sendall(b"HTTP/1.0 200 OK\r\n"
                           b"Content-Type: text/html\r\n\r\n")
            cliente.sendall(pagina().encode())
    except OSError as e:
        print("Visita interrumpida:", e)
    finally:
        cliente.close()

if __name__ == "__main__":
    main()

```

Tres detalles pequeños que evitan tres molestias

SO_REUSEADDR evita el Errno 98, address in use al detener y volver a ejecutar. settimeout(3) en el cliente impide que un navegador que se conecta y no pide nada deje al servidor esperando para siempre. Y sendall —en lugar de send— garantiza que la página salga entera: send puede mandar sólo una parte.

QUÉ DEBE OBSERVARSE

Escribiendo la dirección en el teléfono aparece el título y las dos magnitudes. La página se refresca sola y el número cambia al soplar sobre el sensor. La respuesta es inmediata: no hay espera perceptible.

La prueba que vale la pena hacer en clase: desconecte la pata de datos del DHT11 con el servidor andando. La página **sigue respondiendo**, muestra el último valor bueno y avisa en rojo cuántas lecturas fallaron. Vuelva a conectarla y el aviso desaparece solo. Con la versión que mide dentro de la petición, ese mismo gesto deja la página muerta.

SI ALGO NO SALE

Síntoma	Causa habitual
El navegador no encuentra la dirección	El teléfono está en otra red, o en datos móviles.
Address already in use	Quedó corriendo una copia anterior. El botón EN reinicia.
Dice «Esperando la primera medición» y no cambia	El sensor no entregó ninguna trama buena todavía. Revise la resistencia de elevación en la línea de datos.
La dirección funcionaba ayer y hoy no	El router le dio otra a la placa. Vuelva a mirar lo que imprime al arrancar.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el tablero de un equipo consultable desde el celular dentro de la planta, sin pasar por la red corporativa ni pedir permisos de informática. Cambiando el DHT11 por el transmisor de presión del laboratorio 6.4 se obtiene un indicador remoto de línea.

En el área mecánica, el mismo servidor con el termistor de refrigerante —laboratorio 3.4— y la escala llevada a 60-110 °C da un tablero que se mira desde el celular con el motor en marcha, sin tender cables hasta el banco.

7.3 La misma página con indicadores de barra

OBJETIVO

Cambiar la presentación sin tocar nada de lo que consigue los datos.

RAZONAMIENTO

Es el mismo servidor. Lo único que cambia es la función `pagina()`, que en vez de escribir el número dibuja una barra proporcional. Vale la pena hacerlo notar, porque es la idea que el libro viene repitiendo desde el capítulo 6: **la forma de mostrar los datos no debería obligar a tocar la parte que los consigue.**

Archivo `lab_7_3_barras.py`

```
TEMPERATURA_MIN, TEMPERATURA_MAX = 0, 50
HUMEDAD_MIN, HUMEDAD_MAX = 20, 90

def porcentaje(valor, minimo, maximo):
    """Posicion de valor dentro de la escala, de 0 a 100.

    Recorta solo el LARGO DE LA BARRA. El numero que se escribe al
    lado sigue siendo el que entrego el sensor.
    """
    recorrido = maximo - minimo
    if recorrido <= 0:
        return 0.0
    proporcion = (valor - minimo) * 100.0 / recorrido
    if proporcion < 0:
        return 0.0
    if proporcion > 100:
        return 100.0
    return proporcion

def barra(etiqueta, valor, unidad, minimo, maximo, color):
    fuera = "" if minimo <= valor <= maximo else " (fuera de escala)"
    return """<div class="marco">
<div class="relleno"
    style="width:{:.1f}%; background-color:{};"></div>
<div class="cifra">{} {}</div>
</div>
<div class="rotulo">{} ({} a {} {}){}</div>""".format(
```

porcentaje(valor, minimo, maximo), color, valor, unidad,
etiqueta, minimo, maximo, unidad, fuera)

Un indicador no corrige la medición

La hoja de datos del DHT11 declara humedad de 20 a 90 %, y es tentador recortar la lectura a ese rango antes de mostrarla. Si se hace, una humedad real de 15 % aparece en pantalla como 20 %, y el instrumento está mintiendo.

El recorte se aplica **sólo al largo de la barra**. El número que se informa es siempre el que entregó el sensor, y si queda fuera de escala la página lo dice. En instrumentación este criterio no es negociable: un indicador que ajusta la medición para que entre en su escala es peor que no tener indicador.

QUÉ DEBE OBSERVARSE

Dos barras, la de temperatura sobre una escala de 0 a 50 °C y la de humedad de 20 a 90 %, con la cifra escrita encima. Soplando sobre el sensor la barra de humedad avanza a la vista.

7.4 El celular por Bluetooth

OBJETIVO

Mandar la medición al teléfono por Bluetooth, sin red, sin router y sin que el teléfono ni la placa estén conectados a nada.

MATERIALES

Cantidad	Componente	Observación
1	Sensor DHT11	el mismo del laboratorio 7.2
1	Teléfono	con la aplicación Serial Bluetooth Terminal , gratuita

EL CIRCUITO

El mismo del laboratorio 7.2, sin cambios (figura 7.2).

Bluetooth clásico y Bluetooth LE no son lo mismo

El ESP32 tiene los dos, pero **MicroPython sólo expone el Bluetooth de baja energía (BLE)**. El «puerto serie por Bluetooth» de toda la vida —el perfil SPP, el que se usa con los módulos HC-05— no está disponible desde MicroPython.

Esto importa porque la aplicación *Serial Bluetooth Terminal* maneja las dos clases y arranca en la de siempre. Si se busca la placa en la pestaña de **Bluetooth Classic** no aparece nunca, y no hay nada mal en el programa: hay que buscarla en la pestaña de **Bluetooth LE**. Es el tropiezo número uno de este laboratorio.

RAZONAMIENTO

BLE no es un puerto serie: es una base de datos de características que el teléfono lee, escribe o se suscribe. Para que se comporte como un cable serie existe un acuerdo de la industria —el *Nordic UART Service*— que define dos características, una para cada sentido. Cualquier aplicación de terminal lo reconoce, y ése es el motivo por el que funciona con la aplicación del teléfono sin configurar nada.

Ese acuerdo se implementa una vez, en un módulo que se copia a la placa y que los programas importan. Lo que el programa del laboratorio hace es sólo tres cosas: crear el enlace con un nombre, preguntar si hay alguien conectado y escribir.

EL MÓDULO, QUE SE COPIA A LA PLACA

Archivo `ble_uart.py`

```
# ble_uart.py – puerto serie sobre Bluetooth LE.
# Se copia a la placa una sola vez, como ssd1306.py.
import bluetooth
from micropython import const

_CONECTADO = const(1)
_DESCONECTADO = const(2)
_ESCRITURA = const(3)
```

```

# Identificadores del Nordic UART Service. Son estos y no otros:
# es lo que la aplicacion del telefono busca.
_SERV = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
_TX = (bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
       bluetooth.FLAG_NOTIFY)
_RX = (bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
       bluetooth.FLAG_WRITE)

class BLEUART:
    def __init__(self, nombre="ESP32"):
        self._ble = bluetooth.BLE()
        self._ble.active(True)
        self._ble.irq(self._evento)
        ((self._tx, self._rx),) = self._ble.gatts_register_services(
            ((_SERV, (_TX, _RX)),))
        self._centrales = set()
        self._buzon = bytearray()
        # anuncio: banderas + nombre completo
        n = nombre.encode()
        self._anuncio = bytearray((2, 0x01, 0x06, len(n) + 1, 0x09)) + n
        self._anunciar()

    def _anunciar(self):
        self._ble.gap_advertise(100000, adv_data=self._anuncio)

    def _evento(self, evento, datos):
        if evento == _CONECTADO:
            conn, _, _ = datos
            self._centrales.add(conn)
        elif evento == _DESCONECTADO:
            conn, _, _ = datos
            self._centrales.discard(conn)
            # Hay que volver a anunciarse: si no, el telefono no
            # puede reconectarse y parece que la placa se colgo.
            self._anunciar()
        elif evento == _ESCRITURA:
            conn, atributo = datos
            if atributo == self._rx:
                self._buzon += self._ble.gatts_read(self._rx)

    def conectado(self):
        return len(self._centrales) > 0

    def write(self, texto):
        """Manda texto al telefono. Si no hay nadie, no hace nada."""
        datos = texto.encode() if isinstance(texto, str) else texto

```

```

        for conn in self._centrales:
            self._ble.gatts_notify(conn, self._tx, datos)

    def read(self):
        """Devuelve lo recibido y vacia el buzón. Puede ser vacío."""
        datos = bytes(self._buzon)
        self._buzon = bytearray()
        return datos

```

Veinte caracteres por mensaje

Una notificación de BLE entra en **20 bytes** mientras el teléfono no negocie un tamaño mayor, y MicroPython no lo negocia. Lo que pase de ahí se pierde sin aviso: el mensaje llega cortado y no hay ningún error.

Por eso los mensajes de este laboratorio son cortos y van de a uno — T: 23.0C son siete caracteres— en lugar de una línea larga con todo junto. Es una restricción del medio, no del programa, y conviene tenerla presente al inventar el formato.

PROGRAMA

Archivo `lab_7_4_bluetooth.py`

```

from machine import Pin
import time
import dht
from ble_uart import BLEUART

PIN_DATOS = 14
NOMBRE = "ESP32-TALLER"      # el que va a aparecer en el telefono
INTERVALO = 2                # segundos entre mediciones

sensor = dht.DHT11(Pin(PIN_DATOS))
enlace = BLEUART(NOMBRE)

print("Bluetooth activo como", NOMBRE)
print("Busque ese nombre en la pestaña BLE de la aplicacion.")

fallas = 0

while True:
    # Lo que el telefono haya escrito. Puede venir vacío, así que
    # se comprueba ANTES de decodificar: .decode() sobre nada
    # levanta una excepción y corta el programa.

```

```

recibido = enlace.read()
if recibido:
    print("El telefono dice:", recibido.decode().strip())

try:
    sensor.measure()
    temperatura = sensor.temperature()
    humedad = sensor.humidity()
    fallas = 0
except OSError:
    fallas = fallas + 1
    print("Lectura fallida ({}).".format(fallas))
    time.sleep(INTERVALO)
    continue

print("Temperatura: {} C   Humedad: {} %".format(
    temperatura, humedad))

# Escribir sin nadie conectado no da error, pero tampoco sirve:
# preguntarlo permite avisar en la consola que el telefono se fue.
if enlace.conectado():
    enlace.write("T:{}C\n".format(temperatura))
    enlace.write("H:{}%\n".format(humedad))
else:
    print("    (nadie conectado todavia)")

time.sleep(INTERVALO)

```

CÓMO CONECTARSE DESDE EL TELÉFONO

Paso	Qué hacer
1	Instalar Serial Bluetooth Terminal y dar permiso de ubicación: Android lo exige para poder buscar dispositivos BLE.
2	Abrir el menú lateral y entrar en <i>Devices</i> .
3	Elegir la pestaña Bluetooth LE . No la de Classic.
4	Pulsar <i>Scan</i> y esperar a que aparezca ESP32-TALLER .
5	Tocar el nombre. La barra de arriba pasa a <i>Connected</i> y empiezan a llegar las líneas.

QUÉ DEBE OBSERVARSE

En el teléfono aparecen dos líneas cada dos segundos, con la temperatura y la humedad. Soplando sobre el sensor la humedad sube a la vista. En la consola de Thonny se ve lo mismo, más el aviso de cuando no hay nadie conectado.

La aplicación también manda: escribiendo cualquier cosa en el campo de abajo, el texto aparece en la consola de la placa. Eso abre la puerta al ejercicio 3, donde el teléfono pasa de mirar a mandar.

La prueba que conviene hacer: cerrar la aplicación con el programa corriendo. La consola empieza a avisar que no hay nadie. Volver a abrir y reconectar: sigue funcionando sin reiniciar la placa. Eso es lo que consigue el `_anunciar()` del evento de desconexión.

SI ALGO NO SALE

Síntoma	Causa habitual
La placa no aparece en la búsqueda	Se está buscando en la pestaña de Classic . Tiene que ser la de Bluetooth LE .
No aparece ni en la pestaña LE	Falta el permiso de ubicación de Android, o el programa se detuvo. En Thonny debe verse «Bluetooth activo».
<code>ImportError: no module named 'ble_uart'</code>	Falta copiar <code>ble_uart.py</code> dentro de la placa .
Conecta, llega una línea y se corta	El mensaje pasa de veinte caracteres, o el programa se detuvo al fallar una lectura del sensor.
Se desconecta y ya no vuelve a aparecer	El programa no se anuncia de nuevo al desconectarse. Es lo que resuelve el manejador de <code>_DESCONECTADO</code> .
<code>MemoryError</code> al arrancar	BLE y WiFi encendidos a la vez consumen mucha memoria. Este laboratorio no necesita WiFi: no lo active.

Bluetooth o WiFi, no los dos porque sí

El ESP32 puede tener las dos radios activas, pero comparten memoria y el margen es estrecho. Si un programa no necesita la red —y éste no la necesita— conviene dejar el WiFi apagado. Un `MemoryError` al arrancar suele ser eso y no un programa demasiado grande.

El mismo algoritmo en...

En el área eléctrica y electrónica, es el diagnóstico en sitio: el técnico se para frente al tablero, abre la aplicación y lee lo que el equipo está midiendo, sin acceso a la red de planta ni permisos de informática. Con el medidor del laboratorio 6.5 se leen tensión y corriente delante del equipo.

En el área mecánica, es exactamente lo que hace un lector de diagnóstico con el teléfono. La misma estructura —placa midiendo, teléfono recibiendo por BLE— con el termistor de refrigerante o el tacómetro del laboratorio 4.4 da un instrumento que se lee desde la butaca mientras otro maneja.

7.5 De placa a placa, sin red: ESP-NOW

OBJETIVO

Que una placa dispare una alarma en otra, sin router, sin claves y sin internet.

MATERIALES

Cantidad	Componente	Observación
2	Placas ESP32	una transmisora y otra receptora
1	Pulsador	en la transmisora
1	Zumbador activo	en la receptora; aquí sí sirve

EL CIRCUITO

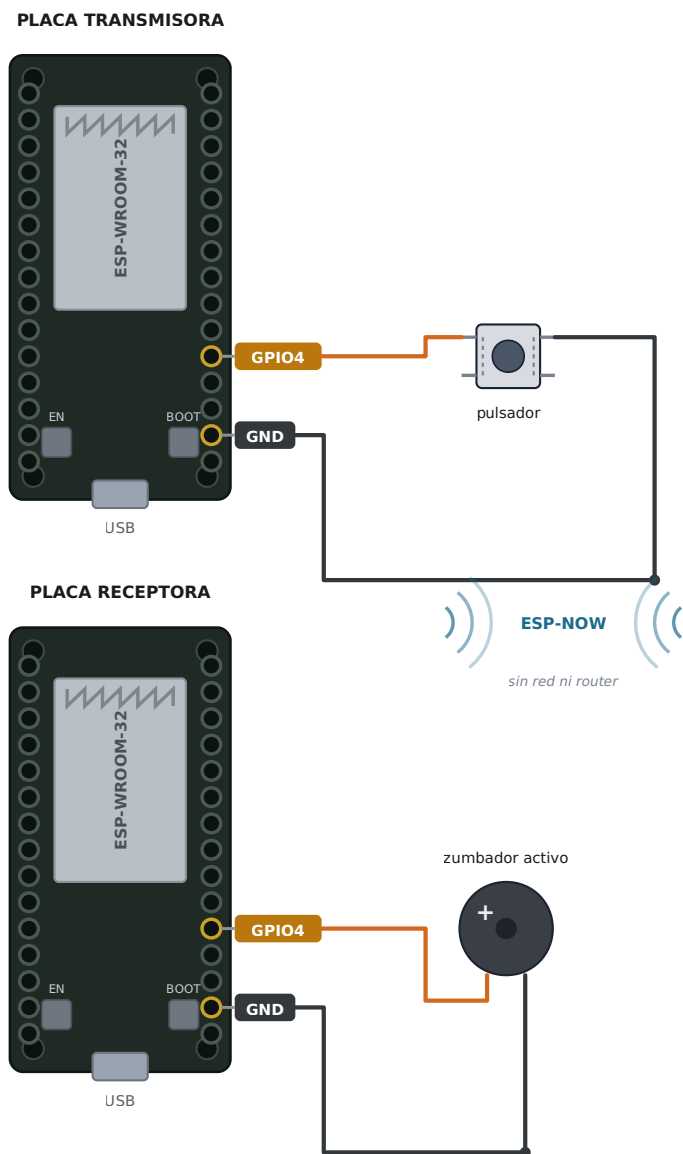


Figura 7.3 — Las dos placas, cada una con su componente. Entre ellas no hay cable ni router: sólo la radio.

ESP-NOW usa la radio del WiFi, no la red del WiFi

Es la confusión más común. ESP-NOW necesita que la interfaz inalámbrica esté **encendida**, porque usa el mismo transmisor de radio, pero **no se conecta a ninguna red**. No hay router, no hay clave, no hay dirección IP. Por eso los programas hacen `wlan.active(True)` seguido de `wlan.disconnect()`: encienden la radio y se aseguran de no estar asociados a nada.

La consecuencia práctica es que funciona donde no hay red: en una nave sin cobertura, entre dos máquinas, en un vehículo. Y que las dos placas deben compartir el mismo canal de radio, que es automático mientras ninguna esté conectada a una red.

RAZONAMIENTO

Cada placa tiene una dirección física única —su MAC— y el transmisor necesita la del receptor para dirigirle el mensaje. Ese es el único paso de configuración, y hay que hacerlo una vez: se ejecuta un programa corto en la receptora, se anota lo que imprime y se copia en la transmisora.

Del lado del transmisor, el envío se dispara en el flanco del pulsador y con el filtro de rebote del laboratorio 2.3: sin él, un solo toque manda diez mensajes. Del lado del receptor, el zumbador debe sonar dos segundos **sin** que el programa deje de escuchar, de modo que en vez de una espera se anota el instante en que debe callarse. Es, otra vez, la idea del laboratorio 2.4.

PASO 1: AVERIGUAR LA DIRECCIÓN DE LA RECEPTORA

Archivo `lab_7_5_mac.py`

```
import network

wlan = network.WLAN(network.STA_IF)
wlan.active(True)

mac = wlan.config("mac")
print("MAC de esta placa:")
print("  legible  : " + ":".join("{:02X}".format(b) for b in mac))
print("  para el código del transmisor:")
```

```
escapada = "".join("\\x{:02x}".format(b) for b in mac)
print("    PAREJA = b'" + escapada + "'")
```

```
MAC de esta placa:
legible   : 3C:61:05:0A:1B:2C
para el código del transmisor:
    PAREJA = b'\x3c\x61\x05\x0a\x1b\x2c'
```

Esa última línea se copia tal cual en el programa del transmisor.

PASO 2: LA PLACA TRANSMISORA

Archivo `lab_7_5_transmisor.py`

```
import network
import espnow
from machine import Pin
import time

PAREJA = b'\xbb\xbb\xbb\xbb\xbb\xbb' # <-- la MAC de la receptora
MENSAJE = b"alarma"
REBOTE_MS = 200

pulsador = Pin(4, Pin.IN, Pin.PULL_UP)

# La radio debe estar encendida, pero sin conectarse a ninguna red.
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.disconnect()

enlace = espnow.ESPNow()
enlace.active(True)
enlace.add_peer(PAREJA)

print("Transmisor listo. Pulse el boton.")

estaba_pulsado = False
ultimo_cambio = time.ticks_ms()

while True:
    ahora = time.ticks_ms()
    pulsado = not pulsador.value()

    hubo_cambio = pulsado != estaba_pulsado
    paso_el_rebote = time.ticks_diff(ahora, ultimo_cambio) > REBOTE_MS
```

```

if hubo_cambio and paso_el_rebote:
    estaba_pulsado = pulsado
    ultimo_cambio = ahora

    if pulsado:
        # Se saca el aviso fuera del try para no anidar cuatro
        # niveles: a esa profundidad la linea ya no entra impresa.
        try:
            entregado = enlace.send(PAREJA, MENSAJE)
        except OSError as e:
            entregado = None
            print("No se pudo enviar:", e)

        if entregado:
            print("Mensaje entregado.")
        elif entregado is not None:
            print("Sin respuesta. Esta encendida la otra?")

    time.sleep_ms(10)

```

PASO 3: LA PLACA RECEPTORA

Archivo `lab_7_5_receptor.py`

```

import network
import espnow
from machine import Pin
import time

MENSAJE = b"alarma"
DURACION_MS = 2000

buzzer = Pin(4, Pin.OUT)
buzzer.off()

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.disconnect()

enlace = espnow.ESPNow()
enlace.active(True)

print("Receptor listo. Esperando mensajes...")

callar_en = 0

while True:

```

```

# Espera hasta 50 ms por un mensaje. Si no llega ninguno
# devuelve None y el programa sigue: nunca queda bloqueado.
quien, mensaje = enlace.recv(50)

if mensaje == MENSAJE:
    origen = ":".join("{:02X}".format(b) for b in quien)
    print("Alarma recibida desde", origen)
    buzzer.on()
    callar_en = time.ticks_add(time.ticks_ms(), DURACION_MS)

elif mensaje is not None:
    print("Mensaje desconocido:", mensaje)

# El zumbador se calla por reloj, no por espera: mientras suena,
# el programa sigue atendiendo mensajes.
if callar_en and time.ticks_diff(time.ticks_ms(), callar_en) >= 0:
    buzzer.off()
    callar_en = 0

```

QUÉ DEBE OBSERVARSE

Al pulsar en una placa, la otra suena dos segundos. La consola del transmisor confirma la entrega y la del receptor informa desde qué dirección llegó.

Vale la pena hacer dos pruebas. La primera: apagar la receptora y pulsar. El transmisor avisa «Sin respuesta» — ESP-NOW confirma la entrega, cosa que el WiFi común no hace. La segunda: alejarse. En interiores el enlace aguanta bastante más que la red del taller, porque no hay router de por medio.

SI ALGO NO SALE

Síntoma	Causa habitual
«Sin respuesta» siempre	La MAC copiada no es la de la receptora, o se copió la del formato legible en lugar de la línea con <code>\\x</code> .
Funcionaba y dejó de funcionar al agregar WiFi	Una de las dos placas se conectó a una red y cambió de canal de radio. Las dos tienen que estar en el mismo.
Un toque dispara varias alarmas	El filtro de rebote es corto para ese pulsador. Subir <code>REBOTE_MS</code> .
<code>ImportError: no module named 'espnow'</code>	El firmware de MicroPython es anterior a la versión 1.19. Hay que actualizarlo.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el enlace entre dos tableros que no tienen red tendida entre ellos: un pulsador de paro de emergencia remoto, un aviso de nivel entre un tanque y la sala de bombas. La confirmación de entrega es lo que lo hace utilizable en seguridad: el transmisor sabe si su mensaje llegó.

En el área mecánica, permite instrumentar un vehículo sin tender cables: un sensor en la rueda o en el motor manda a una placa en el tablero. Sin router, que en un vehículo en movimiento no existe.

EJERCICIOS PROPUESTOS

1. En el laboratorio 7.2, haga que la dirección `/datos` devuelva sólo las dos cifras separadas por una coma, sin HTML. ¿Para qué serviría una dirección así?
2. El intervalo de medición está en `INTERVALO_MS` y el de refresco de la página en la etiqueta `meta`. ¿Qué pasa si el refresco es más rápido que la medición? ¿Y al revés? Justifique cuál de los dos conviene que sea mayor.
3. Dos personas abren la página al mismo tiempo. ¿Ven exactamente la misma cifra? Explique por qué, mirando dónde se guarda la medición.
4. En el laboratorio 7.5, haga que el transmisor mande la cuenta de pulsaciones en lugar de la palabra fija, y que el receptor la muestre.
5. Agregue una tercera placa al montaje de ESP-NOW, de modo que el mensaje llegue a las dos receptoras. Ayuda: mire qué acepta `add_peer` y qué significa la dirección de difusión `b'\\x\xff' * 6`.
6. Compare las cuatro rutas del cuadro de apertura para este caso: un sensor de nivel en un tanque a 80 metros del tablero, en un galpón sin cobertura de WiFi. ¿Cuál elegiría y por qué?

7.6 Publicar en internet: MQTT y Adafruit IO

OBJETIVO

Que la medición salga de la red del taller y quede guardada, de modo que se pueda mirar desde cualquier parte y consultar el histórico.

EL CIRCUITO

El mismo del laboratorio 7.2, sin cambios (figura 7.2).

RAZONAMIENTO

El servidor web del laboratorio 7.2 tiene dos límites de fondo. Sólo alcanza dentro de la misma red —para verlo desde afuera habría que abrir puertos en el router, que en una empresa nadie va a autorizar—, y

no guarda nada: muestra el valor de ahora y lo de hace una hora se perdió.

MQTT resuelve las dos cosas invirtiendo quién llama a quién. En vez de esperar visitas, la placa **publica**: se conecta a un servidor de internet —el *broker*— y le manda el dato. Como es la placa la que sale, el router de la empresa lo permite sin configurar nada. El broker lo guarda y se lo entrega a quien lo pida.

Cada magnitud va a su propio **tópico**, que es simplemente un nombre. En Adafruit IO los tópicos se llaman *feeds* y se nombran con el usuario adelante:

```
su_usuario/feeds/temperatura  
su_usuario/feeds/humedad
```

Los feeds hay que crearlos primero desde la página de la cuenta. Si se publica en un feed que no existe, el broker acepta la conexión y descarta el dato sin avisar.

El límite que explica los avisos de «throttle»

La cuenta gratuita de Adafruit IO admite **30 datos por minuto**, contando todos los feeds juntos. Pasado ese número, el servidor empieza a rechazar publicaciones y manda avisos de *throttle*; si se insiste, corta la conexión por un rato.

La cuenta es directa. Con n feeds publicando cada t segundos: $n \times 60 / t \leq 30$. Con dos feeds hacen falta al menos 4 segundos entre publicaciones; con cuatro feeds —tensión, corriente, potencia y factor de potencia— hacen falta **8**, y a 8 segundos exactos se está justo en el techo, de modo que cualquier reintento lo pasa. Por eso este libro publica cada **15 segundos**: deja margen y para una magnitud física es de sobra.

umqtt.robust, no umqtt.simple

Las dos bibliotecas hacen lo mismo, pero `umqtt.robust` reintenta sola cuando la conexión se corta, que en una red inalámbrica pasa. Con `umqtt.simple` hay que escribir esa lógica a mano. Aun así el programa atrapa el error al publicar y vuelve a conectarse: un equipo que tiene que quedar corriendo semanas no puede detenerse porque el router se reinició una noche.

EL MÓDULO COMPARTIDO, AMPLIADO

Al `iot.py` del laboratorio 7.1 se le agregan dos funciones. Van abajo del todo, y siguen sin contener ninguna clave: la leen de `config.py`. El módulo completo, que es el que hay que tener copiado en la placa de aquí en adelante, queda así:

Archivo `iot.py`

```
# iot.py – conexion a la red, compartida por todo el capitulo
import time
import network
from umqtt.robust import MQTTClient
import config

SERVIDOR = "io.adafruit.com"
PUERTO = 1883

def conectar_wifi(intentos=20):
    """Conecta el ESP32 a la red. Devuelve la direccion IP."""
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Conectando a la red", config.WIFI_SSID, "...")
        wlan.connect(config.WIFI_SSID, config.WIFI_CLAVE)
        for _ in range(intentos * 2):
            if wlan.isconnected():
                break
            time.sleep(0.5)

    if not wlan.isconnected():
        raise OSError("No se pudo conectar. Revise config.py")

    ip = wlan.ifconfig()[0]
```

```

    print("Red conectada. IP:", ip)
    return ip

def conectar_adafruit(identificador):
    """Abre la sesion MQTT con Adafruit IO. Devuelve el cliente."""
    cliente = MQTTClient(client_id=identificador,
                          server=SERVIDOR,
                          port=PUERTO,
                          user=config.AIO_USUARIO,
                          password=config.AIO_CLAVE,
                          keepalive=60)

    cliente.connect()
    print("Conectado a Adafruit IO como", config.AIO_USUARIO)
    return cliente

def feed(nombre):
    """Devuelve el topico MQTT que corresponde a un feed."""
    return "{}/feeds/{}".format(config.AIO_USUARIO, nombre)

```

PROGRAMA

Archivo `lab_7_6_mqtt.py`

```

from machine import Pin
import time
import dht
import iot

PIN_DATOS = 14
INTERVALO = 15          # segundos. Ver el recuadro del limite.

sensor = dht.DHT11(Pin(PIN_DATOS))
TEMA_TEMP = iot.feed("temperatura")
TEMA_HUM = iot.feed("humedad")

def leer_sensor():
    """Temperatura y humedad, o None si la trama falla."""
    try:
        sensor.measure()
        return sensor.temperature(), sensor.humidity()
    except OSError:
        return None

```

```

def main():
    iot.conectar_wifi()
    cliente = iot.conectar_adafruit("esp32_dht11")
    fallas = 0

    while True:
        lectura = leer_sensor()

        if lectura is None:
            fallas = fallas + 1
            print("Lectura fallida ({}). Se reintenta.".format(fallas))
        else:
            temperatura, humedad = lectura
            fallas = 0
            print("Temperatura: {} C   Humedad: {} %".format(
                temperatura, humedad))
            try:
                cliente.publish(TEMA_TEMP, str(temperatura))
                cliente.publish(TEMA_HUM, str(humedad))
            except Exception as e:
                # La conexion se cayo. Se vuelve a abrir en vez de
                # detener el programa: un equipo que queda corriendo
                # semanas no puede pararse porque se reinicio el router.
                print("No se pudo publicar:", e)
                cliente = iot.conectar_adafruit("esp32_dht11")

        time.sleep(INTERVAL0)

if __name__ == "__main__":
    main()

```

QUÉ DEBE OBSERVARSE

En el panel de Adafruit IO los dos feeds se van llenando cada quince segundos, y el gráfico muestra la evolución. Cerrando el navegador y volviendo al día siguiente, el histórico sigue ahí: eso es lo que el servidor web no podía hacer.

Vale la pena una prueba: apagar el WiFi del router un minuto y volver a encenderlo. El programa avisa que no pudo publicar, se reconecta solo y sigue. En el gráfico queda un hueco, no una interrupción del equipo.

SI ALGO NO SALE

Síntoma	Causa habitual
Conecta pero el feed queda vacío	El feed no existe en la cuenta, o el nombre está escrito distinto. Se descarta el dato sin avisar.
Avisos de <i>throttle</i> y después se corta	Se pasó de 30 datos por minuto. Subir <code>INTERVALO</code> .
<code>MQTTException: 5</code>	Usuario o clave equivocados. La clave es la del servicio, no la de la cuenta de correo.
Publica un rato y se detiene	Se está usando <code>umqtt.simple</code> , o falta el <code>try</code> alrededor de <code>publish</code> .

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el monitoreo de consumo de un tablero, con el medidor del laboratorio 6.5 en lugar del DHT11: tensión, corriente, potencia y factor de potencia publicados a un panel que gerencia mira desde su oficina. Ahí el límite de 30 datos por minuto deja de ser un detalle: son cuatro feeds.

En el área mecánica, es la telemetría de un vehículo de flota. La temperatura del motor y las revoluciones —laboratorio 4.4— publicadas mientras el vehículo circula, usando el teléfono como punto de acceso.

7.7 Un tablero propio: InfluxDB y Grafana

OBJETIVO

Guardar el histórico en un servidor del propio taller, sin límite de datos por minuto y sin depender de ninguna cuenta de internet.

RAZONAMIENTO

Adafruit IO resuelve el laboratorio anterior con muy poco trabajo, pero impone tres condiciones: un tope de datos por minuto, un histórico corto en la cuenta gratuita, y que los datos del taller vivan en un servidor ajeno.

Para un ensayo de aula está bien; para instrumentar una máquina durante un semestre, no.

La alternativa es montar el servidor en la propia red. Son dos programas que corren en cualquier computadora del taller: **InfluxDB**, que es una base de datos pensada para valores con fecha y hora, y **Grafana**, que dibuja los tableros. Los dos son gratuitos y no necesitan internet: basta con que la placa y la computadora estén en la misma red.

Del lado del ESP32 el cambio es menor de lo que parece. En vez de publicar por MQTT, se manda una petición HTTP con el dato. InfluxDB acepta un formato de texto muy simple, de una sola línea:

```
ambiente temperatura=23.4,humedad=55.0
|           |           |
|           |           +-- otro campo, separado por coma
|           +--- nombre del campo y su valor
+--- nombre de la medicion
```

PREPARAR EL SERVIDOR

Paso	Qué hacer
1	Instalar InfluxDB y Grafana en una computadora del taller. Los dos traen instalador para Windows y para Linux.
2	En InfluxDB, crear una organización (por ejemplo <code>taller</code>), un <i>bucket</i> (<code>sensores</code>) y un token de escritura. El token va a <code>config.py</code> , no al programa.
3	Anotar la dirección de esa computadora en la red. Es la que el ESP32 va a usar, y conviene fijarla en el router para que no cambie.
4	En Grafana, agregar InfluxDB como origen de datos y crear un panel sobre el bucket <code>sensores</code> .

La dirección no es «localhost»

Desde la computadora donde corre InfluxDB, el servidor se ve en `localhost`. Desde el ESP32 **no**: hay que poner la dirección de esa computadora en la red, la que empieza con `192.168.`. Es el tropiezo más frecuente de este laboratorio, y el síntoma es un error de conexión que no dice nada útil.

El otro tropiezo es el cortafuegos de Windows, que bloquea el puerto `8086` hasta que se le autoriza.

PROGRAMA

Archivo `lab_7_7_influx.py`

```
import time
from machine import Pin
import urequests
import dht
import iot
import config

PIN_DATOS = 14
INTERVALO = 10          # aqui no hay limite de datos por minuto

sensor = dht.DHT11(Pin(PIN_DATOS))

URL = "http://{}:8086/api/v2/write?org={}&bucket={}&precision=s".format(
    config.INFLUX_HOST, config.INFLUX_ORG, config.INFLUX_BUCKET)

CABECERAS = {"Authorization": "Token " + config.INFLUX_TOKEN,
             "Content-Type": "text/plain"}

def escribir(temperatura, humedad):
    """Manda una linea a InfluxDB. Devuelve True si la acepto.

    La respuesta se cierra SIEMPRE. En MicroPython una respuesta sin
    cerrar deja el socket tomado, y despues de unas cuantas el
    programa se queda sin ninguno y deja de poder publicar.
    """
    linea = "ambiente temperatura={},humedad={}".format(
        temperatura, humedad)
    respuesta = None
    try:
        respuesta = urequests.post(URL, data=linea, headers=CABECERAS)
```

```

        return respuesta.status_code in (200, 204)
    except OSError as e:
        print("No se pudo escribir:", e)
        return False
    finally:
        if respuesta is not None:
            respuesta.close()

def main():
    iot.conectar_wifi()
    print("Escribiendo en", URL)

    while True:
        try:
            sensor.measure()
            temperatura = sensor.temperature()
            humedad = sensor.humidity()
        except OSError:
            print("Lectura fallida. Se reintenta.")
            time.sleep(INTERVALO)
            continue

        if escribir(temperatura, humedad):
            print("{} C {} % anotado".format(temperatura, humedad))
        else:
            print("{} C {} % NO se pudo anotar".format(
                temperatura, humedad))

        time.sleep(INTERVALO)

if __name__ == "__main__":
    main()

```

El finally no es adorno

Es la diferencia entre un programa que corre un mes y uno que se detiene a la media hora. Cada `urequests.post` abre un socket, y MicroPython tiene muy pocos: si la respuesta no se cierra —y con un `return` de por medio es facilísimo olvidarlo— se agotan. El `finally` garantiza el cierre haya salido bien, mal o por excepción.

QUÉ DEBE OBSERVARSE

La consola escribe «anotado» cada diez segundos, y el panel de Grafana dibuja la curva en tiempo real. Dejándolo toda la noche, a la mañana siguiente está el registro completo: es lo que no daban ni el servidor web ni la cuenta gratuita.

SI ALGO NO SALE

Síntoma	Causa habitual
Error de conexión inmediato	La dirección es <code>localhost</code> en lugar de la de la red, o el cortafuegos bloquea el puerto 8086.
Contesta 401	El token es incorrecto, o falta la palabra <code>Token</code> delante en la cabecera.
Contesta 400	La línea está mal formada. Los espacios importan: uno solo entre el nombre de la medición y los campos, y ninguno alrededor del igual.
Anota un rato y deja de anotar	Falta cerrar la respuesta. Es el caso del recuadro anterior.
Grafana no muestra nada y no hay error	El rango de tiempo del panel es anterior a los datos. Ponerlo en los últimos quince minutos.

El mismo algoritmo en...

En el área eléctrica y electrónica, éste es el registrador de una planta: consumo, temperaturas y presiones guardados en un servidor propio, con los datos dentro de la empresa y sin cuota mensual. Es la base de cualquier estudio de eficiencia energética, que necesita meses de registro y no unos minutos.

En el área mecánica, es el banco de pruebas instrumentado: se deja corriendo un ensayo de varias horas y después se comparan las curvas de temperatura y revoluciones. Con el límite de una cuenta gratuita ese ensayo no se puede registrar.

7.8 Una página que manda, no que informa

OBJETIVO

Encender y apagar desde el navegador el relé del laboratorio 2.5, sin cuenta, sin servidor y sin salir de la red del taller.

RAZONAMIENTO

Los siete laboratorios anteriores mandan datos hacia afuera. Éste va en sentido contrario, y el cambio de sentido trae problemas propios que la publicación no tiene.

Cuando un programa informa, un dato perdido cuesta poco: se manda el siguiente. Cuando un programa obedece, una orden perdida —o peor, una orden entendida al revés— deja algo encendido que tenía que quedar apagado. Por eso este laboratorio se arma sobre el relé del 2.5 y hereda sus reglas: **arranca apagado**, y la constante del puente H/L sigue siendo la única línea donde aparece un nivel.

El servidor es el del laboratorio 7.2 con dos diferencias. La página trae dos botones en lugar de dos números, y el programa mira **qué** se pidió antes de contestar. Esa segunda diferencia parece trivial y es la que se lleva la tarde entera.

Mirar la petición entera es lo que rompe el botón de apagar

La manera natural de escribirlo es preguntar si la petición contiene `/on` o `/off`, y así es como sale la primera vez. Funciona, y al rato el botón de apagar deja de apagar.

La causa no está en el programa sino en el navegador. Al pulsar ENCENDER, la página que vuelve queda en la dirección `http://ip/on`; al pulsar APAGAR desde ahí, el navegador agrega una cabecera `Referer: http://ip/on` para avisar de dónde viene. Esa cabecera viaja dentro de la misma petición, de modo que buscar `/on` en el texto completo la encuentra, se cumple la primera rama y el relé se enciende otra vez.

El síntoma es de los peores que puede tener un montaje: un foco de 220 V que no se apaga con el botón de apagar, y que sí se apaga si se abre una pestaña nueva, porque ahí todavía no hay `Referer`. La causa es una sola línea: hay que leer **la línea del pedido** y no la petición completa.

Ese es justamente el trabajo de `camino_pedido()`, que el laboratorio 7.2 ya usaba para descartar el pedido del icono. Una petición empieza siempre así:

```
GET /off HTTP/1.1
Host: 192.168.0.42
Referer: http://192.168.0.42/on
Connection: keep-alive
```

La primera línea es el pedido; el resto son cabeceras que el navegador agrega por su cuenta y sobre las que el programa no tiene ningún control. Quedarse con la primera línea y partirla por espacios da `/off` y nada más.

EL CIRCUITO

El del laboratorio 2.5, sin un solo cambio: el módulo de relé en `GPI019`, con su fuente y su masa común. El pulsador puede quedar puesto —no molesta— pero este programa no lo lee.

PROGRAMA

Archivo `lab_7_8_pagina_que_manda.py`

```

import socket
import iot
from machine import Pin

PUERTO = 80
PIN_RELE = 19
ACTIVO_EN_BAJO = True          # el puente H/L del modulo, como en el 2.5

rele = Pin(PIN_RELE, Pin.OUT)
encendido = False

def mandar(estado):
    """Traduce el estado al nivel que pide este modulo."""
    global encendido
    encendido = estado
    if ACTIVO_EN_BAJO:
        rele.value(0 if estado else 1)
    else:
        rele.value(1 if estado else 0)

def camino_pedido(peticion):
    """El camino de la PRIMERA linea: GET /algo HTTP/1.1

    Buscar "/on" en la peticion entera parece lo mismo y no lo es: el
    navegador agrega una cabecera Referer con la direccion anterior, y
    ahi aparece el "/on" de la visita previa. El boton de apagar deja
    de apagar.
    """
    try:
        return peticion.split("\r\n")[0].split(" ")[1]
    except IndexError:
        return "/"

def pagina():
    estado = "ENCENDIDO" if encendido else "APAGADO"
    color = "#1B7A3D" if encendido else "#8A8F98"
    return PLANTILLA.format(color=color, estado=estado)

PLANTILLA = """<!DOCTYPE html>
<html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Control del rele</title>
<style>

```

```

body {{ font-family: sans-serif; text-align: center; margin-top: 8vh }}
.estado {{ font-size: 2em; color: {color}; margin-bottom: 0.6em; }}
a {{ text-decoration: none; }}
button {{ font-size: 1.8em; padding: 0.6em 1.4em; margin: 0.4em;
         border: none; border-radius: 12px; color: white; }}
.on {{ background: #1B7A3D; }} .off {{ background: #9B2C2C; }}
</style></head><body>
<h2>Control del rele</h2>
<p class="estado">{estado}</p>
<a href="/on"><button class="on">ENCENDER</button></a>
<a href="/off"><button class="off">APAGAR</button></a>
</body></html>
"""

```

```

def main():
    mandar(False)          # apagado antes de abrir la red a nadie
    ip = iot.conectar_wifi()

    servidor = socket.socket()
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    servidor.bind(socket.getaddrinfo("0.0.0.0", PUERTO)[0][-1])
    servidor.listen(2)

    print("Abra http://{}/ en el telefono o el navegador.".format(ip))

    while True:
        cliente, direccion = servidor.accept()
        try:
            cliente.settimeout(3)
            peticion = cliente.recv(1024).decode()
            camino = camino_pedido(peticion)

            if camino == "/favicon.ico":
                cliente.sendall(b"HTTP/1.0 404 Not Found\r\n\r\n")
            else:
                if camino == "/on":
                    mandar(True)
                    print("orden desde", direccion[0], "-> encender")
                elif camino == "/off":
                    mandar(False)
                    print("orden desde", direccion[0], "-> apagar")

                cliente.sendall(b"HTTP/1.0 200 OK\r\n"
                               b"Content-Type: text/html\r\n\r\n")
                cliente.sendall(pagina().encode())
        except OSError as e:
            print("Visita interrumpida:", e)

```

```
        finally:
            cliente.close()

if __name__ == "__main__":
    main()
```

Por qué la página muestra el estado

Los dos botones solos no alcanzan. Quien abre la página desde otro teléfono no sabe si el foco está encendido, y si pulsa ENCENDER sobre algo que ya lo está no pasa nada visible, lo que invita a pulsar otra vez, y otra.

Mostrar el estado convierte la página en un **reflejo** de lo que pasa en la placa y no sólo en un juego de botones. Es la diferencia entre un mando y un tablero, y en el laboratorio siguiente se vuelve indispensable: cuando la orden puede venir de varios lados, cada lado tiene que poder enterarse de lo que hicieron los demás.

QUÉ DEBE OBSERVARSE

Al abrir la dirección aparece la página con el estado en APAGADO. ENCENDER hace chasquear el relé y el estado pasa a ENCENDIDO; APAGAR lo devuelve. La consola escribe una línea por orden, con la dirección de quien la mandó.

La prueba que hay que hacer, y que es el motivo del laboratorio: pulsar ENCENDER y después APAGAR **en la misma pestaña**, varias veces seguidas. Con el programa de arriba apaga siempre. Con la versión que busca en la petición entera, la segunda vez ya no.

SI ALGO NO SALE

Síntoma	Causa habitual
El botón de apagar no apaga, salvo en pestaña nueva	Se está buscando /on en toda la petición y lo encuentra en la cabecera Referer . Ver el recuadro.
El relé se enciende solo al arrancar el programa	El módulo es activo en bajo y el pin queda en 0 antes de que el programa mande nada. Por eso mandar(False) va primero, incluso antes de conectar la red.
La página abre pero el relé no responde	El problema es del laboratorio 2.5, no de éste: masa común o tensión de bobina.
Desde el teléfono no abre	El teléfono está en otra red, o en datos móviles. Tiene que estar en la misma red inalámbrica que la placa.
Anda un rato y deja de responder	La placa perdió la red. Este servidor no reconecta solo; el laboratorio siguiente sí.

El mismo algoritmo en...

En el área eléctrica y electrónica, es el mando local de una máquina desde el teléfono, sin cablear una botonera: arranque y parada de una bomba, de un extractor, del alumbrado de un galpón. Y la lección del Referer vale igual en cualquier interfaz: no decidir sobre el texto completo de un mensaje, sino sobre el campo que corresponde.

En el área mecánica, es el banco de pruebas gobernado desde el celular mientras se está debajo del vehículo: activar el electroventilador, la bomba de combustible o el precalentador sin volver a la cabina. El relé es el mismo que el vehículo trae de fábrica.

7.9 El botón del tablero: suscribirse en lugar de publicar

OBJETIVO

Gobernar el mismo relé desde el tablero de Adafruit IO, de modo que la orden llegue desde cualquier lugar con internet y no sólo desde la red del taller.

RAZONAMIENTO

El laboratorio 7.6 publica: la placa abre la conexión, manda el dato y sigue con lo suyo. La comunicación la empieza siempre ella.

Recibir una orden no puede funcionar así. La placa no sabe cuándo alguien va a pulsar el botón del tablero, y preguntar cada segundo «¿hay algo para mí?» es justamente lo que MQTT vino a evitar. Lo que se hace es al revés: la placa **se suscribe** a un feed y le deja al servidor el trabajo de avisarle. Mientras no pase nada, no viaja nada.

En el programa eso son tres piezas que no aparecían en el 7.6. Una **función de atención**, que es la que el cliente llama cuando llega un mensaje. La **suscripción**, que declara a qué feed. Y una llamada a `check_msg()` dentro de la repetición, que es la que le da al cliente la oportunidad de revisar si llegó algo y disparar la atención. Sin esa tercera línea las otras dos no sirven de nada, y es el olvido más frecuente.

El feed y el bloque del tablero

Hace falta un feed nuevo —este libro lo llama `foco`— y un bloque de tipo **interruptor** (*toggle*) conectado a él. En la configuración del bloque hay dos casillas con los valores que manda: conviene `1` y `0`, que es lo que el programa espera.

El anexo A explica cómo se crean los feeds y los bloques. Y conviene recordar el otro límite de la cuenta gratuita: son **cinco feeds** en total, y éste ocupa uno.

Un botón en internet enciende algo de verdad

Hasta aquí una orden equivocada encendía un LED. Ahora enciende una carga de 220 V, y el botón está en una página a la que se llega desde cualquier parte del mundo con el usuario y la clave de la cuenta.

De ahí dos cosas. La clave de Adafruit IO deja de ser una molestia administrativa y pasa a ser la llave de algo físico: si se publicó alguna vez, regenérala desde la llave amarilla antes de armar este laboratorio. Y la carga que se conecte tiene que ser una que no importe que quede encendida sola: un foco, sí; una estufa o un motor, no.

PROGRAMA

Al `iot.py` del laboratorio 7.6 no hay que agregarle nada: `conectar_adafruit()` y `feed()` ya alcanzan.

Archivo `lab_7_9_boton_adafruit.py`

```
import time
import iot
from machine import Pin

PIN_RELE = 19
ACTIVO_EN_BAJ0 = True          # el puente H/L del modulo, como en el 2.5
FEED = "foco"
PING_MS = 30000                # la mitad del keepalive de iot.py

rele = Pin(PIN_RELE, Pin.OUT)
encendido = False

def mandar(estado):
    global encendido
    encendido = estado
    if ACTIVO_EN_BAJ0:
        rele.value(0 if estado else 1)
    else:
        rele.value(1 if estado else 0)
    print("rele", "encendido" if estado else "apagado")

def atender(topico, mensaje):
    """La llama el cliente MQTT cuando llega algo del feed.
```

Los dos argumentos vienen en bytes y no en texto. Y el valor que manda el bloque interruptor es "1" o "0" segun como se lo haya configurado: cualquier otra cosa se ignora en lugar de darla por buena, que con una carga de 220 V no es lo mismo.

```
"""
orden = mensaje.decode().strip()
if orden == "1":
    mandar(True)
elif orden == "0":
    mandar(False)
else:
    print("orden desconocida, se ignora:", orden)
```

```
def main():
    mandar(False)          # apagado antes de escuchar a nadie
    iot.conectar_wifi()

    cliente = iot.conectar_adafruit("esp32-rele")
    cliente.set_callback(atender)
    cliente.subscribe(iot.feed(FEED))

    # Adafruit guarda el ultimo valor de cada feed. Pidiendolo al
    # arrancar, la placa se entera de como quedo el interruptor la
    # vez anterior en lugar de esperar a que alguien lo toque.
    cliente.subscribe(iot.feed(FEED) + "/get")
    cliente.publish(iot.feed(FEED) + "/get", "")

    print("Escuchando el feed", FEED)
    ultimo_ping = time.ticks_ms()

    while True:
        # Sin esta linea la suscripcion no sirve de nada: es aqui
        # donde el cliente mira si llego algo y llama a atender().
        cliente.check_msg()

        # Si no viaja nada durante el keepalive, el servidor da la
        # sesion por muerta y corta. El ping la mantiene viva.
        if time.ticks_diff(time.ticks_ms(), ultimo_ping) > PING_MS:
            cliente.ping()
            ultimo_ping = time.ticks_ms()

        time.sleep_ms(200)

if __name__ == "__main__":
    main()
```

Por qué hace falta el ping

Publicar mantiene viva la conexión sin que uno se entere: cada dato que sale es tráfico. Un programa que sólo escucha no manda nada, y al cabo del *keepalive* —sesenta segundos, en el módulo del laboratorio 7.6— el servidor da la sesión por perdida y la cierra. La placa se queda escuchando un feed al que ya no está suscrita, sin ningún aviso.

El `ping()` cada treinta segundos es lo que lo evita: la mitad del *keepalive*, que es la regla habitual. Es la falla más desconcertante de este laboratorio, porque el programa funciona el primer minuto y después deja de obedecer sin escribir un solo error.

QUÉ DEBE OBSERVARSE

Al arrancar, la consola escribe el estado que tenía el interruptor la vez anterior: eso es el mensaje retenido llegando. Desde ahí, cada vez que se mueve el interruptor del tablero el relé chasquea en menos de un segundo, esté el tablero abierto en la computadora del taller o en el teléfono con datos móviles.

La prueba que muestra de qué se trata el laboratorio: dejar el tablero abierto en dos dispositivos a la vez. Al mover el interruptor en uno, el otro se actualiza solo. Ninguno de los dos habla con la placa: los tres hablan con el feed.

SI ALGO NO SALE

Síntoma	Causa habitual
Conecta pero el relé nunca se mueve	Falta <code>check_msg()</code> dentro de la repetición, o la suscripción usa un nombre de feed distinto del que tiene el bloque.
Obedece un minuto y después no	Falta el <code>ping()</code> . Ver la nota.
<code>MQTTException: 5</code>	Usuario o clave mal copiados en <code>config.py</code> . La clave es la de la llave amarilla, no la contraseña de ingreso.
El bloque manda ON y OFF en lugar de 1 y 0	Es la configuración del bloque interruptor. Cambiar los dos valores allí, o aceptar <code>ON</code> y <code>OFF</code> en <code>atender()</code> .
Al arrancar no recupera el estado anterior	Falta la suscripción al tópico terminado en <code>/get</code> , o la publicación vacía que la dispara.

El mismo algoritmo en...

En el área eléctrica y electrónica, es el telemando de una instalación sin ir hasta ella: el riego de un invernadero, la bomba de un tanque elevado, el alumbrado de un playón. Y con el 7.6 publicando la medición y éste recibiendo la orden, el lazo se cierra: la misma cuenta muestra el nivel del tanque y enciende la bomba.

En el área mecánica, es el precalentador o el bloqueo remoto de la bomba de combustible, que es como funciona un corte antirrobo comercial. Conviene mirarlo con ojo crítico: lo que en este laboratorio es un foco, ahí es un vehículo, y la clave de la cuenta es todo lo que separa al dueño de cualquier otro.

EJERCICIOS PROPUESTOS

1. Calcule cada cuántos segundos puede publicar un equipo con seis feeds en una cuenta gratuita de Adafruit IO. ¿Y con diez?
2. Modifique el laboratorio 7.6 para que publique sólo cuando la medición cambió respecto de la anterior. ¿Cuántos datos por minuto ahorra en un ambiente estable? ¿Qué se pierde?
3. En el laboratorio 7.7, agregue una etiqueta a la línea de InfluxDB — `ambiente,lugar=taller temperatura=23.4` — y úsela en Grafana para distinguir dos placas en el mismo panel.
4. Escriba una tabla comparando las cuatro rutas del capítulo según: alcance, histórico, dependencias, costo y qué pasa si se cae internet.
5. Un ensayo de motor de seis horas registrando cuatro magnitudes cada segundo. ¿Cuántos datos son? ¿Cuál de las cuatro rutas lo soporta?
6. Tome el tablero local del laboratorio 6.6 y agréguele la publicación de este capítulo. Debería ser una cuarta responsabilidad junto a leer, decidir y mostrar, sin tocar las otras tres. Si tuvo que tocarlas, ¿dónde estaba mal separado el programa original?
7. Escriba la versión equivocada del laboratorio 7.8 —la que busca `/on` en la petición entera— y compruebe el error con el navegador. Después abra una pestaña nueva y pulse APAGAR. Explique por qué ahí sí funciona.
8. Al laboratorio 7.8 agréguele una dirección `/estado` que devuelva sólo `1` o `0`, sin HTML. ¿Qué podría hacer otra placa con esa dirección?
9. Junte los laboratorios 7.6 y 7.9 en un solo programa: que publique la temperatura y además obedezca al interruptor. Cuide que la publicación no deje de atender los mensajes que llegan, y diga en qué parte del programa estaba el riesgo.
10. El laboratorio 7.9 hace `ping()` cada 30 s con un *keepalive* de 60. ¿Qué pasaría con 90 s? ¿Y con 1 s? Justifique los dos extremos.

A.1 La cuenta y la clave

Archivo `config.py`

```
AIO_USUARIO = "jmarcelo"  
AIO_CLAVE = "aio 000000000000000000000000000000000000"
```

Esa clave da acceso a toda la cuenta

No es una contraseña de un feed: con ella se puede leer, escribir y borrar cualquier cosa de la cuenta. Por eso no va dentro del programa, no se comparte y no se muestra en una diapositiva.

Si se publicó por error, desde la misma llave amarilla se la puede **regenerar**, y la anterior deja de servir en el acto. Es la ventaja de estas claves frente a la del WiFi.

A.2 Los feeds

Un **feed** es donde se guarda una magnitud. Hace falta uno por cada cosa que se quiera publicar: uno para la temperatura, otro para la humedad. Si el programa publica en un feed que no existe, el servidor acepta la conexión y **descarta el dato sin avisar**, de modo que crear los feeds primero ahorra un buen rato de desconcierto.

Paso	Qué hacer
------	-----------

- | | |
|---|--|
| 1 | En el menú superior, entrar en Feeds . |
| 2 | Pulsar New Feed y ponerle nombre. Para el laboratorio 7.6, <code>temperatura</code> y <code>humedad</code> ; para el 7.9, <code>foco</code> . |
| 3 | El nombre tiene que escribirse igual que en el programa, con las mismas minúsculas y sin acentos. |

Dos límites de la cuenta gratuita

Hay un tope de **cinco feeds** y otro de **treinta datos por minuto**, contando todos los feeds juntos. El segundo es el que aparece en el recuadro del laboratorio 7.6, con la cuenta que permite elegir el intervalo. El primero se nota antes: un medidor eléctrico que publique tensión, corriente, potencia, factor de potencia y energía ya ocupa los cinco.

A.3 El tablero

El tablero —*dashboard*— es la pantalla donde se ven los datos. Se arma con bloques, y cada bloque se conecta a uno o más feeds.

Paso	Qué hacer
1	En el menú superior, Dashboards → New Dashboard . Ponerle nombre y pulsar <i>Create</i> .
2	Entrar al tablero recién creado. Arriba a la derecha hay un engranaje: Dashboard Settings → Create New Block .
3	Elegir el tipo de bloque. Para ver la evolución en el tiempo, el gráfico de líneas ; para un valor suelto, el indicador circular; para mandar una orden en lugar de mirar un dato, el interruptor (<i>toggle</i>) del laboratorio 7.9.
4	En Connect Feeds , marcar los feeds que ese bloque va a mostrar. Un gráfico de líneas admite varios a la vez, y poner temperatura y humedad juntas permite compararlas.
5	Next step , ajustar el título y los límites del eje, y crear.

El bloque interruptor manda, no informa

Los bloques de gráfico e indicador **leen** un feed. El interruptor lo **escribe**: cada vez que se lo mueve, publica un valor, y quien esté suscrito a ese feed lo recibe. Es lo que usa el laboratorio 7.9 para encender el relé.

Al crearlo pregunta qué valor manda en cada posición. Conviene poner 1 y 0, que es lo que espera el programa del libro; si se dejan ON y OFF, hay que cambiar la función `atender()` en lugar del bloque.

Desde ese momento, cada vez que la placa publique, el bloque se actualiza solo. El tablero se puede compartir con un enlace público desde *Dashboard Privacy*, que es lo que permite que el resto del curso vea la medición sin tener cuenta.

SI ALGO NO SALE

Síntoma	Causa habitual
El programa conecta pero el feed queda vacío	El feed no existe, o el nombre está escrito distinto. El servidor no avisa.
Avisos de <i>throttle</i>	Se pasó de treinta datos por minuto. Subir el intervalo.
<code>MQTTEException: 5</code>	Usuario o clave mal copiados. La clave es la de la llave amarilla, no la contraseña de ingreso.
El bloque no muestra nada aunque el feed tenga datos	El rango de tiempo del gráfico es anterior a los datos, o el bloque quedó conectado a otro feed.

Los módulos que se copian a la placa

MicroPython trae lo básico, pero varios laboratorios necesitan módulos que no vienen incluidos. Son archivos `.py` que se copian **dentro de la placa**, una sola vez, igual que se copia un programa. Una vez allí quedan disponibles para cualquier programa que los importe.

Archivo	Para qué	Laboratorios
<code>config.py</code>	Guarda las claves fuera del programa. Se copia de <code>config_ejemplo.py</code> y se completa.	7.1 en adelante
<code>iot.py</code>	Conexión a la red y al servidor. Se escribe en el 7.1 y se completa en el 7.6; el del repositorio es el completo.	7.1 y 7.6 a 7.9
<code>ssd1306.py</code>	Controlador de la pantalla OLED.	6.1, 6.2, 6.6
<code>ble_uart.py</code>	Puerto serie sobre Bluetooth. Se escribe en el laboratorio 7.4.	7.4
<code>pzem.py</code>	Protocolo Modbus del medidor eléctrico.	6.5
<code>umqtt/</code>	Cliente MQTT. Viene con algunas versiones del firmware; si no, se instala aparte.	7.6, 7.9

CÓMO COPIARLOS

Con Thonny, la manera más simple es abrir el archivo en el computador, y después *Archivo* → *Guardar como...* eligiendo «**dispositivo MicroPython**» en lugar de «este computador». El nombre debe quedar exactamente igual, incluida la extensión.

Para comprobar que están, desde la consola:

```
>>> import os
>>> os.listdir()
['boot.py', 'config.py', 'iot.py', 'ssd1306.py', 'main.py']
```

Dónde conseguirlos

`config.py`, `iot.py` y `ble_uart.py` están escritos completos en este libro. `ssd1306.py` y `umqtt` son parte de la biblioteca oficial de MicroPython. Todos, junto con los programas de los laboratorios, están también en el repositorio que acompaña al libro.

Los pines de cada laboratorio

Tabla de consulta rápida. Sirve para armar un laboratorio sin releer su capítulo y, sobre todo, para ver de un vistazo qué montajes pueden convivir en la misma placa y cuáles se pisan.

Lab.	Montaje	Pines
1.5	LED de la placa	GPIO2
2.1	LED externo	GPIO23
2.2	Semáforo	GPIO23, GPIO21, GPIO17
2.3	Pulsador y LED	GPIO16, GPIO5
2.4	PIR y zumbador	GPIO4, GPIO5, VIN
2.5	Módulo de relé y pulsador	GPIO19, GPIO16
3.1	Potenciómetro	GPIO34, 3V3
3.2	Divisor de 5 V	GPIO35, VIN
3.3	LM35	GPIO32, VIN
3.4	Termistor NTC	GPIO33, 3V3
3.5	Calibración	GPIO34, 3V3
4.1 - 4.3	Puente generador-contador	GPIO25 → GPIO26
4.4	Tacómetro con L298N	GPIO19, GPIO18, GPIO5, GPIO17, VIN
5.2	Brillo con potenciómetro	GPIO34, GPIO18, 3V3
5.3 - 5.4	Servomotor	GPIO4, VIN
5.5	Zumbador pasivo	GPIO5
6.1 - 6.2	Pantalla OLED (I ² C)	GPIO22 (SCL), GPIO21 (SDA), 3V3
6.3	DHT11	GPIO14, 3V3
6.4	Presión HX710B	GPIO16 (OUT), GPIO4 (SCK), 3V3
6.5	Medidor PZEM-004T	GPIO16 (RX), GPIO17 (TX), 3V3
6.6	Tablero local	GPIO22, GPIO21, GPIO14, GPIO33, 3V3

7.2 – 7.4	DHT11 con red o Bluetooth	GPIO14, 3V3
7.5	ESP-NOW	GPIO4 en cada placa
7.8 – 7.9	Relé gobernado a distancia	GPIO19

Los pines que este libro no usa nunca

GPIO 6 a 11 están conectados a la memoria del módulo: usarlos impide arrancar. **GPIO 1 y 3** son la consola del USB. **GPIO 34, 35, 36 y 39** sólo pueden ser entrada. Y ningún sensor analógico de este libro va al ADC2 —GPIO 0, 2, 4, 12 a 15 y 25 a 27—, porque deja de medir con el WiFi encendido.

El plano completo, con los colores que agrupan estos casos, está en la figura 1.1, que es el de la placa de **30 pines**: la que usan todas las figuras del libro. Si la suya tiene diecinueve agujeros por lado en vez de quince, es la de 38 y los pines están en otro orden —los nombres de esta tabla siguen valiendo, la posición en el borde no.

QUÉ SE PISA CON QUÉ

Conflicto	Dónde aparece
El bus I ² C ocupa GPIO21 y GPIO22	El semáforo del 2.2 usa GPIO21 y el medidor del 6.5 usa GPIO17: ninguno de los dos puede convivir con la pantalla sin recablear.
GPIO5 aparece en cuatro laboratorios	2.3, 2.4, 4.4 y 5.5. Al combinar montajes es el primero que hay que revisar.
GPIO4 es SCK del HX710B y señal del servo	El 6.4 y el 5.3 no pueden estar armados a la vez.
GPIO16 y GPIO17 son el puerto serie del PZEM	El 6.5 se lleva los dos; el 2.3, el 2.5 y el 6.4 también usan GPIO16.
GPIO19 gobierna el relé y también el puente H	El 2.5, el 7.8 y el 7.9 lo usan para el relé; el 4.4 lo usa para ENA del L298N. No pueden convivir.